universität koblenz
Mathematik / Naturwissenschaften

Physics-Informed Neural Networks for Inverse Navier–Stokes Problems: Spatial Wake Analysis and High-Reynolds Number Extension

Fachbereich Mathematik / Naturwissenschaften
Universität Koblenz

Master of Science
in **Mathematical Modeling, Simulation and Optimization**

Submitted by:
Ravikumar Amrutlal Bharadva
Student ID: 222202539

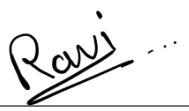
First Supervisor: Prof. Dr. Martin Siebenborn
Second Reviewer: Dr. Christian Kahle

Koblenz, 26th October 2026

Declaration

I hereby declare that the thesis submitted is my own unaided work. All direct or indirect sources used are acknowledged as references. I am aware that the thesis in digital form can be examined for the use of unauthorised aid and in order to determine whether the thesis as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future theses submitted.

Koblenz, 26th October 2026



Ravikumar Amrutlal Bharadva

Abstract

This thesis investigates the use of physics-informed neural networks (PINNs) for inverse problems in computational fluid dynamics, with a focus on identifying the parameters of the incompressible Navier–Stokes equations. Building on the cylinder-wake benchmark of Raissi et al., the network has to identify the convection coefficient λ_1 and the viscosity coefficient λ_2 from velocity data alone, without any pressure measurements. The main contribution is a systematic study of how the location and size of the measurement window in the wake, measurement noise and a higher Reynolds number affect this identification.

First, a complete workflow was built, consisting of mesh generation in Gmsh, high-fidelity spectral/hp element simulation in Nektar++, data extraction in Python, and PINN training. The original TensorFlow 1 implementation was ported to TensorFlow 2, and the benchmark at $Re = 100$ was reproduced. The PINN was then trained on data from different regions of the wake instead of only the region directly behind the cylinder, to examine whether accurate identification is still possible under this restriction. In the mid-wake and far-wake windows, λ_1 was recovered with errors of 0.009% and 0.11% and λ_2 with errors of 0.10% and 2.96%, respectively, and the pressure field was reconstructed although no pressure data were used. With 1% noise in the training velocities of these two windows, the error of λ_1 stayed below 0.1%, while the error of λ_2 was 3.1% in the mid wake and 0.63% in the far wake. In smaller windows that cover only one side of the vortex street, λ_1 was still recovered to within 0.16%, while the error of λ_2 rose to 5.2% and 3.2%.

Second, the study was extended to a higher Reynolds number, $Re = 500$, for which the mesh was refined and the time step reduced in the simulation. At $Re = 500$, λ_1 was still identified with an error of 3.5%, whereas the identification of λ_2 failed, with an error of 178%, and the velocity field was reproduced much less accurately than at $Re = 100$. Unlike at $Re = 100$, the training data at $Re = 500$ also contain the start-up of the flow, so this result does not isolate the effect of the Reynolds number.

The results show that PINN-based parameter identification works well for two-dimensional, time-periodic cylinder wakes at $Re = 100$, also from limited parts of the flow, while the identification of the viscosity already fails at $Re = 500$ with the present network and training.

Acknowledgements

I would like to express my sincere gratitude to my supervisors, Prof. Dr. Martin Siebenborn and Dr. Christian Kahle, for their invaluable guidance and support throughout this research. Their advice and suggestions have been an essential part of this thesis. I also thank my family and friends for their encouragement.

Contents

Declaration i

Abstract ii

Acknowledgements iii

List of Figures vii

List of Tables viii

List of Symbols and Abbreviations ix

1 Introduction 1

2 Literature Review 4

3 Theoretical Background 7

3.1 History of Neural Networks 7

3.2 Neural Network Architecture 8

3.2.1 The Artificial Neuron 8

3.2.2 The Feed-Forward Network 8

3.2.3 Activation Functions 9

3.3 Physics-Informed Neural Networks 10

3.3.1 General Formulation 10

3.3.2 Forward and Inverse Problems 11

3.4 The Spectral/hp Element Solver Nektar++ 11

4 Mathematical Formulation 12

4.1 Navier–Stokes Equations 12

4.1.1 Kinematics: Flow Map, Material Derivative and Transport Theorem . 12

4.1.2 Conservation of Mass 14

4.1.3 Conservation of Linear Momentum 14

4.1.4 Constitutive Law of a Newtonian Fluid 15

4.2 The Incompressible Navier–Stokes Equations 16

4.2.1 Convention Used from Here On . . . 16
4.2.2 Role of the Pressure . . . 17
4.2.3 Initial and Boundary Conditions . . . 17
4.3 The Reynolds Number . . . 17
4.3.1 Non-Dimensionalisation . . . 17
4.3.2 High Reynolds Numbers . . . 18
4.4 The Stream Function . . . 18
4.4.1 Pressure from the Stream Function . . . 19
4.4.2 Stream Function in the Inverse Problem . . . 19
4.5 PINN Mathematical Formulation . . . 19
4.5.1 Network Ansatz . . . 19
4.5.2 Backpropagation . . . 20
4.5.3 Enforcing Incompressibility . . . 20
4.5.4 Automatic Differentiation . . . 20
4.5.5 PDE Residual and Loss Function . . . 21
4.5.6 Optimisation . . . 22

5 The Spectral/hp Element Method in Nektar++ 26
5.1 Weak Form . . . 26
5.2 Spectral/hp Element Discretisation . . . 26
5.2.1 Convergence . . . 27
5.3 Velocity Correction Scheme . . . 27

6 Methodology and Implementation 29
6.1 The Cylinder-Wake Benchmark . . . 29
6.2 Mesh Generation . . . 30
6.3 Flow Simulation . . . 32
6.3.1 Validation of the Reference Simulation . . . 33
6.4 Data Extraction . . . 34
6.5 PINN Implementation in TensorFlow 2 . . . 35
6.5.1 Porting from TensorFlow 1 to TensorFlow 2 . . . 35
6.5.2 Network and Training Settings . . . 36
6.5.3 Error Measures . . . 37
6.5.4 Noise Model . . . 37
6.6 Overview of the Experiments . . . 37

7 Results and Discussion **39**

7.1 Reproduction of the Benchmark at $Re = 100$ 39

7.2 Identification from Different Regions of the Wake 41

7.2.1 Mid Wake 41

7.2.2 Far Wake 43

7.2.3 Off-Centre Windows 45

7.2.4 Comparison of the Windows 47

7.3 Higher Reynolds Number: $Re = 500$ 47

7.4 Discussion 50

8 Conclusion and Future Work **52**

8.1 Conclusion 52

8.2 Future Work 52

Bibliography **54**

A Mesh and Simulation Set-up **60**

A.1 Gmsh Geometry and Size Fields 60

A.2 Nektar++ Session File 61

B Data Format and Extraction **64**

B.1 From Nektar++ Output to Training Data 64

B.2 Structure of the Data Files 64

C The L-BFGS Two-Loop Recursion **65**

List of Figures

3.1 A single artificial neuron. 8
3.2 A fully connected feed-forward network with two hidden layers. 9

6.1 Schematic of the cylinder-wake benchmark of Raissi et al. 29
6.2 Spectral/hp element meshes generated with Gmsh for the cylinder wake. . . . 31
6.3 Vorticity of the cylinder wake at $Re = 100$ with the training regions. 35

7.1 Reference and PINN-predicted fields at $Re = 100$ in the near wake. 40
7.2 Training history at $Re = 100$ in the near wake. 41
7.3 Predicted and reference fields at $Re = 100$ in the mid wake. 42
7.4 Training history at $Re = 100$ in the mid wake. 42
7.5 Predicted and reference fields at $Re = 100$ in the far wake. 44
7.6 Training history at $Re = 100$ in the far wake. 44
7.7 Predicted and reference fields for the off-centre windows. 46
7.8 Predicted and reference fields at $Re = 500$ in the near wake. 48
7.9 Training loss at $Re = 500$. 49

List of Tables

2.1 Studies on parameter identification in the incompressible Navier–Stokes equations that are most relevant to this thesis. 6

5.1 Coefficients of the stiffly stable scheme for orders $K = 1, 2, 3$. 28

6.1 Properties of the meshes used for the two Reynolds numbers. 31
6.2 Numerical settings of the Nektar++ simulations. 32
6.3 Shedding frequency and force coefficients of the reference simulations. 34
6.4 Network and training settings. 36
6.5 Overview of the numerical experiments. 38

7.1 Results at $Re = 100$ in the near wake. 39
7.2 Results at $Re = 100$ in the mid wake. 41
7.3 Results at $Re = 100$ in the far wake. 43
7.4 Results at $Re = 100$ for the off-centre windows. 45
7.5 Results at $Re = 500$ in the near wake. 47

B.1 Arrays in the data files. 64

List of Symbols and Abbreviations

Bold lower-case symbols denote vectors and bold upper-case symbols denote matrices or second-order tensors. Symbols that are used only inside one derivation are defined where they appear.

Flow quantities

d	spatial dimension ($d = 2$ in this thesis)
Ω, $\partial\Omega$, $\mathbf{n}$	flow domain, its boundary and the outward unit normal
$\mathbf{u} = (u, v)$	velocity vector in $\mathbb{R}^d$ and its components
p	pressure (kinematic pressure in non-dimensional form)
ρ	density
μ, ν	dynamic and kinematic viscosity, $\nu = \mu/\rho$
ζ	second (dilatational) viscosity
μ_1, μ_2	coefficients of the isotropic tensor (4.17), $2\mu = \mu_1 + \mu_2$
$\mathbf{I}$	identity matrix
$\boldsymbol{\sigma}$, $\boldsymbol{\tau}$	Cauchy stress tensor and viscous stress tensor in $\mathbb{R}^{d\times d}$
$\mathbf{T}$	traction (surface force per unit area), $\mathbf{T} \in \mathbb{R}^d$
$\mathbf{S}$	rate-of-strain tensor in $\mathbb{R}^{d\times d}$
$\dot{\gamma}$	shear rate of a simple shear flow
$\mathbf{X}$, Ω_0, Ω_t	initial (material) position of a particle, initial region and material region at time t
$\boldsymbol{\Phi}$, $\mathbf{F}$, J	flow map, deformation gradient $\mathbf{F} = \partial\boldsymbol{\Phi}/\partial\mathbf{X}$ and $J = \det\mathbf{F}$
$\mathbf{f}$, Π	body force per unit mass and its potential, $\mathbf{f} = -\nabla\Pi$
Q	generic smooth scalar field (Section 4.1.1)
ω	vorticity in 2D, $\omega = \partial_x v - \partial_y u$
ψ	stream function, $u = \psi_y$, $v = -\psi_x$
D, U_∞	cylinder diameter and free-stream velocity
L_{ref}, U_{ref}	reference length and reference velocity of the non-dimensionalisation
Re	Reynolds number, $Re = U_\infty D/\nu$
St, f_s	Strouhal number and shedding frequency, $St = f_s D/U_\infty$
F_x, F_y, C_D, C_L	streamwise and transverse force per unit span on the cylinder and the drag and lift coefficients (6.1)
t_{f}	final time of a simulation or of a time interval $[0, t_{\mathrm{f}}]$

PINN quantities

λ_1, λ_2	unknown coefficients of the convective and viscous terms (true values 1 and $1/Re$)
$\hat{\lambda}_1$, $\hat{\lambda}_2$	identified values of λ_1, λ_2
$\mathcal{F}_\theta$	generic feed-forward network with parameters θ (Chapter 3)
θ	all weights and biases of the network, $\theta \in \mathbb{R}^{n_\theta}$
Θ	all trainable quantities $(\theta, \lambda_1, \lambda_2) \in \mathbb{R}^{n_\Theta}$, $n_\Theta = n_\theta + 2$
L, n_l	number of hidden layers and width of layer l
n_{in}, n_{out}	input and output dimension of a network
$\mathbf{W}^{(l)}$, $\mathbf{b}^{(l)}$	weight matrix in $\mathbb{R}^{n_l \times n_{l-1}}$ and bias vector in $\mathbb{R}^{n_l}$ of layer l
$\mathbf{z}^{(l)}$, $\mathbf{a}^{(l)}$	pre-activation and activation of layer l
$\boldsymbol{\delta}^{(l)}$	error (sensitivity of the loss) of layer l in backpropagation
ϕ	activation function (here $\phi = \tanh$); sig denotes the logistic sigmoid
R_θ^u, R_θ^v	residuals of the x- and y-momentum equations
$\mathcal{L}$	loss function
w_{c}, w_{b}	weights of the residual and boundary terms in the general loss (3.10)
N_d, N_c, N_b	number of data, collocation and boundary points
N_x, N_t	number of spatial points and snapshots in a data set
$\mathbf{e}_1$	first unit vector, $\mathbf{e}_1 = (1, 0)^\top$
$\mathbf{c}^{(r)}$, $\bar{\mathbf{c}}^{(r)}$, $\mathbf{D}_r$, M	intermediate result, adjoint, local Jacobian and number of steps in reverse-mode AD (4.46)
k	iteration index of the optimiser
$\mathbf{g}_k$, $\mathbf{m}_k$, $\mathbf{h}_k$	gradient and Adam moment estimates at iteration k, all in $\mathbb{R}^{n_\Theta}$
η, β_1, β_2, ϵ	Adam step size, decay rates and regularisation constant
$\tilde{\eta}_k$, $\hat{\epsilon}$	bias-corrected step size and regularisation constant of the TensorFlow form of Adam
$\mathbf{H}_k$	quasi-Newton approximation of the inverse Hessian, in $\mathbb{R}^{n_\Theta \times n_\Theta}$
$\mathbf{p}_k$, α_k	search direction in $\mathbb{R}^{n_\Theta}$ and step length of L-BFGS-B
c_1, c_2	constants of the Wolfe conditions, $0 < c_1 < c_2 < 1$
$\mathbf{s}_k$, $\mathbf{y}_k$, ϱ_k	BFGS difference vectors and $\varrho_k = 1/(\mathbf{y}_k^\top \mathbf{s}_k)$
m	number of stored pairs $(\mathbf{s}_i, \mathbf{y}_i)$ in L-BFGS (memory)
ϑ_k	scaling of the initial matrix $\mathbf{H}_k^0 = \vartheta_k \mathbf{I}$ in L-BFGS
f_{tol}, g_{tol}	stopping tolerances of L-BFGS-B for the loss and the gradient
ε_n	noise level of the training velocities
s_u, s_v, Z_i^u, Z_i^v	standard deviations of the clean velocities and standard normal random numbers in the noise model (6.5)
e_{λ_j}, e_q	relative parameter error ($j = 1, 2$) and relative L^2 error of $q \in \{u, v, p\}$

Numerical method

P	polynomial order of the spectral/hp expansion (NUMMODES = $P + 1$)
$P_i^{(1,1)}$	Jacobi polynomial of degree i

φ_i, φ_i^{1D}	two- and one-dimensional modal basis functions (Chapter 5)
$\hat{\Omega}$, $\boldsymbol{\xi}$	reference element and reference coordinates, $\boldsymbol{\xi} \in \hat{\Omega} \subset \mathbb{R}^2$
N_{el}, Ω^e	number of elements and element e
$\mathbf{x}^e$	map from the reference element to element e
N_e, $\hat{\mathbf{u}}^e$	number of modes and coefficient vector of element e, $\hat{\mathbf{u}}^e \in \mathbb{R}^{N_e}$
C, κ	constants of the exponential convergence estimate (5.5)
h_e	typical element size
K	order of the time-integration scheme
γ_0, a_r, χ_r	coefficients of the stiffly stable time integration (Table 5.1)
Δt	time step

Abbreviations

AD	automatic differentiation
CFD	computational fluid dynamics
IMEX	implicit–explicit time integration
L-BFGS(-B)	limited-memory Broyden–Fletcher–Goldfarb–Shanno method (with bounds)
MLP	multilayer perceptron
PDE	partial differential equation
PINN	physics-informed neural network
ReLU	rectified linear unit
TF1, TF2	TensorFlow version 1 and 2

Chapter 1

Introduction

Fluid mechanics has long faced a basic difficulty: the equations that describe viscous flow have been known for nearly two centuries, yet solving them in realistic settings remains very difficult. The incompressible Navier–Stokes equations describe flows as different as blood moving through an artery and air leaving a wind-turbine blade, but their nonlinearity and the multi-scale nature of turbulence mean that direct numerical simulation can require enormous computational resources [1]. Classical computational fluid dynamics (CFD), based on finite difference, finite element and spectral discretisations, has made real progress, but it comes with costs that are hard to ignore. For complex geometries, generating a suitable mesh is often one of the most time-consuming steps of a simulation, and it has been named as one of the main bottlenecks of current CFD practice [2]. Classical solvers do not include sparse or noisy measurements directly; this requires additional data-assimilation methods, such as variational (adjoint-based) or ensemble Kalman techniques, which have to be built around the solver [3, 4]. Inverse problems, in which physical parameters are recovered from observed flow data rather than prescribed in advance, usually require reformulating the problem and writing special-purpose code [4].

These difficulties have encouraged a steady shift towards data-driven and machine-learning methods in fluid mechanics. Brunton, Noack and Koumoutsakos [5] reviewed the growing role of machine learning in flow modelling, flow control and reduced-order modelling, while Duraisamy, Iaccarino and Xiao [6] examined how data can be used to correct systematic errors in turbulence closure models, a problem that purely analytical approaches have not solved for decades. Both reviews show that machine learning is not simply a fashionable addition to fluid mechanics: classical CFD has real structural limitations that data-driven methods are well placed to address.

Physics-informed neural networks (PINNs) are one of the most systematic attempts to combine these two worlds. Rather than treating the neural network as a black-box function approximator trained only on data, a PINN incorporates the governing partial differential equations directly into the training objective, penalising the network both for a poor fit of the data and for violations of the physics [7]. The idea goes back to the 1990s. Dissanayake and Phan-Thien [8] showed that the approximation capacity of a neural network can be used to turn the solution of a PDE into an unconstrained minimisation problem, and Lagaris, Likas and Fotiadis [9] extended this to a more general framework for ordinary and partial differential equations with different types of boundary conditions. These early studies received little attention at the time, partly because training neural networks was computationally expensive. Beyond faster hardware, the main change since then has been the maturity of automatic differentiation, which makes it practical to evaluate PDE residuals through a network during training without spatial discretisation [10, 11].

Raissi, Perdikaris and Karniadakis [10] formalised the modern PINN framework, combining these ideas into a single methodology for both forward and inverse problems. Since then the field has grown quickly. The reviews by Karniadakis et al. [7] and Cuomo et al. [12] describe many variants, including conservative, extended, variational and Bayesian PINNs,

applied in solid mechanics, heat transfer, geophysics and biomedicine. PINNs are not a finished technology: training can be unstable, the gradients of different loss terms can be badly balanced, and large problems remain expensive [13, 14]. On the other hand, for problems with sparse data and known governing laws, PINNs offer a feature that no purely data-driven approach can provide: the prediction is constrained by the governing equations also between and away from the measurement locations, although only approximately, because the physics enters the loss as a penalty [7].

The application of PINNs that is most relevant to this thesis is parameter identification for the incompressible Navier–Stokes equations. Raissi et al. [10] demonstrated it on a problem that has since become a standard benchmark: the two-dimensional flow past a circular cylinder at Reynolds number $Re = 100$. The idea is simple. The momentum equations are formulated with two unknown coefficients: λ_1, which multiplies the convective terms, and λ_2, which multiplies the viscous terms. A neural network is then trained to approximate the sparsely observed velocity field while also satisfying the PDE residuals, with λ_1 and λ_2 treated as additional trainable variables. The result is not only a fitted flow field but also an estimate of the governing physics. The pressure field is never observed; it is obtained as a by-product of the physics constraint [10].

This inverse capability has since been extended, for example to recover the pressure without pressure data [15], to infer velocity and pressure from the concentration of a passive scalar [16], and to identify structural parameters of a vibrating cylinder [17]. These studies are discussed in Chapter 2.

The same inverse capability has been used outside fluid mechanics, for example to identify material properties in solid mechanics [18], to infer unknown thermal boundary conditions in heat transfer [19], and to recover arterial blood pressure and near-wall blood flow from sparse medical measurements [20, 21]. These applications cover very different physical settings but share one structure: a known PDE, sparse observations, and unknown parameters or hidden fields that have to be recovered.

Two questions are left open by the original cylinder study. The first concerns the source of the data. Raissi et al. [10] sampled their training points from one fixed rectangle directly behind the cylinder. In practice, measurements are often available only in a limited part of the flow, for example in one camera window of a particle image velocimetry experiment, and the wake does not look the same everywhere: close to the cylinder the vortices are strong and the gradients steep, while further downstream the vortices are weaker and more diffused [22]. It is therefore not obvious that λ_1 and λ_2 can be identified equally well across the wake.

The second question involves the Reynolds number. The cylinder study was conducted at $Re = 100$, where vortices are shed in a regular, periodic manner. As the Reynolds number increases, the vortex dynamics become more complex and the velocity gradients steeper, which is known to make PINN training more difficult. Several studies have shown that PINN training becomes harder under exactly these conditions, because the loss landscape becomes ill-conditioned and the gradients of the different loss terms become unbalanced [14, 13], and because high-speed and turbulent flows are considerably more demanding than laminar ones [23, 24]; these difficulties are reviewed in Chapter 2. The viscosity of the cylinder wake has also been identified with other approaches, for example by sparse regression of the vorticity equation [25] and by adjoint-based optimisation [26], and PINNs have been used to infer the viscosity of two-dimensional turbulent flows [27]. To the author's knowledge, however, no study has examined systematically how the location of the measurement window in the cylinder wake, and a Reynolds number above 100, affect the PINN-based identification of λ_1 and λ_2.

This thesis takes the parameter-identification framework of Raissi et al. [10] as its starting point. The overall goal is the same: to recover λ_1 and λ_2 from velocity observations without measuring the pressure and without knowing the viscosity in advance. Within this goal, the thesis pursues four objectives. The first is to reproduce the cylinder-wake benchmark at $Re = 100$ with a complete pipeline of mesh generation in Gmsh, simulation in Nektar++ and PINN training, and to port the original TensorFlow 1 implementation of Raissi et al. to the current TensorFlow 2 interface. The second is to train the PINN on data from different regions of the wake, including windows that lie off the wake axis, and to find out whether λ_1 and λ_2 can still be identified when the data come only from the mid or far wake or from one side of the vortex street. The third is to test how strongly measurement noise in the velocity data affects the identification in these windows. The fourth is to extend the identification problem to a higher Reynolds number, $Re = 500$, and to examine whether the same network and training procedure still recover the parameters when the flow becomes more complex.

The thesis is organised as follows. Chapter 2 reviews the literature on PINNs for fluid mechanics and their known training difficulties. Chapter 3 introduces feed-forward neural networks, the general PINN framework and the solver Nektar++. Chapter 4 derives the incompressible Navier–Stokes equations and the stream-function formulation and states the PINN formulation, loss function and optimisers used in this work. Chapter 5 summarises the spectral/hp element method used to generate the reference data. Chapter 6 describes the computational workflow and the numerical experiments, and Chapter 7 presents and discusses their results. Chapter 8 summarises the findings and outlines future work.

Chapter 2

Literature Review

This chapter reviews the literature on physics-informed neural networks for fluid mechanics. It starts with the original PINN framework and its use for inverse problems, then discusses applications to more complex flows, the known training difficulties, domain decomposition and alternative identification methods, and ends with the research gap addressed in this thesis. Many of the studies below use the flow past a circular cylinder as test case; because its behaviour at moderate Reynolds numbers is so well known [22], it is a natural benchmark for data-driven methods.

Raissi, Perdikaris and Karniadakis [10] introduced the term physics-informed neural network, framing the solution of a partial differential equation as a supervised learning problem with an unusual kind of label. In addition to fitting measured or simulated field data, they added a second loss term obtained by inserting the network output into the governing PDE and evaluating the residual at a set of collocation points, with the required derivatives computed by automatic differentiation. A network that minimises both the data misfit and this residual is forced to behave like a PDE solution even where no data are available. The same paper showed that the construction can be reversed to solve inverse problems: if the equation coefficients are left as trainable parameters alongside the network weights, they can be recovered from sparse and possibly noisy observations without explicitly inverting the forward operator. This inverse formulation, which the authors applied to the two-dimensional Navier–Stokes equations behind a cylinder, is the direct starting point for the parameter-identification problem in this thesis. Lu, Meng, Mao and Karniadakis [28] later packaged the method in the open-source DeepXDE library, which helped to standardise how PINNs are set up and trained.

Karniadakis et al. [7] placed this idea in a broader programme called physics-informed machine learning and argued that building prior physical knowledge into the learning process reduces the amount of data needed and improves generalisation outside the training data. Cuomo et al. [12] surveyed the many variants that followed, from variational and conservative reformulations to new activation functions and architectures, and noted that most of this work has focused on the loss function and the optimisation procedure, while the underlying idea has changed little since 2019. Raissi, Yazdani and Karniadakis [16] introduced Hidden Fluid Mechanics, which infers the full velocity and pressure fields of an unsteady flow from nothing more than the concentration field of a passive scalar, again by enforcing the Navier–Stokes equations as a soft constraint. The appeal of this result is that the pressure, which is difficult to measure directly in an experiment, emerges as a by-product of the physics constraint. A related study by Raissi, Wang, Triantafyllou and Karniadakis [17] applied the same idea to vortex-induced vibration, coupling the flow network to a structural model of a freely oscillating cylinder and recovering the unmeasured pressure and unknown structural parameters from partial observations. Both studies used the flow past a circular cylinder as their main test case, showing how central this geometry has become for physics-informed inverse problems.

PINNs have also been tested as forward solvers for high-speed and compressible flows. Mao, Jagtap and Karniadakis [23] extended the formulation to high-speed flows with shocks and steep gradients, and Jagtap, Mao, Adams and Karniadakis [29] used a similar approach

for inverse problems in supersonic flow, recovering unknown flow quantities from sparse, Schlieren-type data. Closer to the incompressible setting of this thesis, Jin, Cai, Li and Karniadakis [15] developed NSFnets and compared velocity–pressure and velocity–vorticity formulations against direct numerical simulation data for laminar and turbulent channel flow. They found that the weighting between the loss terms strongly affects accuracy and that simulating turbulent channel flow required restricting the network to a part of the domain, with boundary data taken from the direct simulation, because training over the full domain was too expensive. This is one of the few studies that directly addresses the cost of pushing PINNs to higher Reynolds numbers. Cai, Mao, Wang, Yin and Karniadakis [30] reviewed this literature and observed a consistent pattern: PINNs work reliably for smooth flows at low to moderate Reynolds numbers but lose accuracy as the flow becomes multi-scale, whether through turbulence, sharp gradients or long time horizons.

As PINNs were applied to harder problems, several studies explained why the basic formulation often fails to converge or converges to a solution that satisfies the residual but is physically wrong. Wang, Teng and Perdikaris [13] traced one failure mode to an imbalance between the back-propagated gradients of the loss terms: the data loss and the residual loss can differ strongly in size and curvature, so whichever term produces the larger gradient dominates training. Their learning-rate annealing scheme rebalances the loss weights during training and improved accuracy by one to two orders of magnitude on several benchmarks. A follow-up study used the neural tangent kernel to show that different loss components converge at very different rates and that this, rather than an architectural weakness, explains many observed failures [31]. McClenny and Braga-Neto [32] proposed a self-adaptive scheme in which point-wise loss weights are learned during training, so that the network concentrates on the collocation points where the residual is hardest to reduce, while Yu, Lu, Meng and Karniadakis [33] showed that also penalising the gradient of the residual improves accuracy for problems with sharp local features.

Further studies identified failure modes that are specific to time-dependent problems. Krishnapriyan, Gholami, Zhe, Kirby and Mahoney [14] showed that PINNs can fail completely on simple time-dependent PDEs once the problem becomes stiff enough, because the soft-penalty formulation makes the optimisation landscape harder to navigate. Wang, Sankaran and Perdikaris [34] offered a complementary explanation: the standard loss treats all collocation points in space and time equally, so the optimiser may fit late times before the solution at early times has converged, violating the causal structure of the problem. They proposed a temporal weighting in which the error at early times must decrease before later times contribute strongly. Both studies are relevant to the higher-Reynolds-number case in this thesis, since steeper gradients and faster dynamics are exactly the conditions under which these difficulties have been reported.

Domain decomposition offers a different way to reduce the difficulty of training one network over a large domain. Jagtap, Kharazmi and Karniadakis [35] introduced conservative PINNs (cPINNs), which split the spatial domain into subdomains, assign a separate network to each, and enforce continuity of the solution and its flux across the interfaces. Jagtap and Karniadakis [36] generalised this to extended PINNs (XPINNs), which allow decomposition in space and time and arbitrary subdomain shapes at the cost of a weaker interface condition, and Shukla, Jagtap and Karniadakis [37] developed a parallel implementation of both methods with good multi-GPU scaling. This work is related to the region-based training used in this thesis, where the network is trained on data from one bounded region of the wake at a time. The motivation is different, however: here the regions represent physically different measurement windows, and the aim is to find out which part of the wake carries enough information to identify the parameters, rather than to reduce the load on a single network. An alternative that avoids retraining altogether is operator learning, for example DeepONet [38],

which learns a mapping from whole families of input conditions to the corresponding solutions; this is a possible direction for generalising the present results across flow conditions.

PINNs are not the only way to identify the parameters of a flow. In classical data assimilation the unknown parameters or fields are found by minimising the misfit between a numerical solution and the measurements, with the gradient computed from an adjoint problem, or by sequential estimation such as ensemble Kalman filtering [3, 4]. Zhang, Alla and Karniadakis [26] compared adjoint-based optimisation with PINNs on several inverse problems, among them the identification of a scalar viscosity in the two-dimensional cylinder wake. They found that the better choice depends mainly on how the unknown is represented: the discrete adjoint is efficient for low-dimensional parameters on a grid, PINNs are attractive for unknowns represented by networks and for time-dependent problems in which storing the adjoint trajectory is costly, and a PINN result refined by the adjoint method reached the accuracy of the adjoint at about half its cost.

A second alternative is sparse regression. Rudy et al. [25] built a library of candidate terms from numerically differentiated data and selected the few terms that describe the dynamics; applied to the vorticity of the cylinder wake at $Re = 100$, this recovered the vorticity transport equation including its viscous coefficient. Because the derivatives are computed from the data directly, the method is sensitive to noise [25], which is one motivation for the physics-informed alternative, in which the derivatives are taken from a smooth fitted network. Noise has also been treated explicitly by Bayesian PINNs [39], which estimate not only the unknown parameters but also their uncertainty. Closest to the higher-Reynolds-number part of this thesis, Parfenyev, Blumenau and Nikitin [27] used a PINN to reconstruct a moderately turbulent two-dimensional flow from sparse and possibly noisy velocity data and inferred the viscosity and a friction coefficient together with the pressure. Table 2.1 summarises the studies that are closest to this thesis.

Table 2.1: Studies on parameter identification in the incompressible Navier–Stokes equations that are most relevant to this thesis.

Study	Flow	Data	Identified	Method
[10]	cylinder wake, $Re = 100$	velocity, one window	λ_1, λ_2, p	PINN
[25]	cylinder wake, $Re = 100$	vorticity on a grid	equation terms	sparse regression
[16]	several flows	passive scalar	$\mathbf{u}$, p	PINN
[27]	2D turbulence	sparse velocity	viscosity, friction	PINN
[26]	cylinder wake	velocity	viscosity	adjoint vs. PINN
This thesis	cylinder wake, $Re = 100, 500$	velocity, five windows	λ_1, λ_2, p	PINN

Comparatively few of the studies above consider PINNs at Reynolds numbers above the classical benchmark value. NSFnets is one of the few that treat a turbulent case directly, and even there the domain was restricted and boundary data from the direct simulation were required [15]. The literature on gradient imbalance and causality offers a plausible reason why this is difficult: a wake at higher Reynolds number contains a wider range of active length and time scales than a wake at $Re = 100$, which is exactly the situation in which unbalanced gradients and violated causality have been shown to disturb training [13, 34]. Cai et al. [30] reach a similar conclusion and identify extending PINNs to more complex flows as an open problem. This thesis contributes to this question by testing parameter identification for the cylinder wake at $Re = 500$ and by studying the influence of the training region.

Chapter 3

Theoretical Background

This chapter introduces the deep-learning tools that underpin physics-informed neural networks (PINNs). It begins with a short history of neural networks, which explains why the standard choices of the field are what they are. It then treats the feed-forward architecture and the activation functions and introduces the general idea of the physics-informed framework; the detailed formulation for the Navier–Stokes problem follows in Chapter 4. The chapter closes with a short description of the spectral/hp element solver Nektar++, which is used to generate the reference data [40].

3.1 History of Neural Networks

The idea of computing with networks of simple units predates the modern computer. In 1943, McCulloch and Pitts proposed a mathematical model of the neuron as a threshold logic unit and showed that suitably connected units can, in principle, represent any logical proposition [41]. Their model could not learn; Rosenblatt's perceptron [42] added a learning rule that lets a single layer of such units adjust its weights from examples, and it created considerable early optimism about machine learning.

This optimism was followed by disappointment, for a specific reason. Minsky and Papert [43] showed that a single-layer perceptron can only separate classes that are linearly separable. This is illustrated by the exclusive-or (XOR) function, which is not linearly separable and therefore cannot be represented by a single perceptron [44, Sec. 6.1]. Stacking several layers removes this restriction, but at the time no practical method existed for training the weights of the hidden layers. This changed in 1986, when Rumelhart, Hinton and Williams [45] popularised backpropagation, an efficient application of the chain rule that computes the gradient of the error with respect to every weight of a multilayer network. Shortly afterwards, Cybenko [46] and Hornik, Stinchcombe and White [47] proved that a feed-forward network with one hidden layer and enough neurons can approximate any continuous function on a compact set to arbitrary accuracy, and Leshno et al. [48] showed that this holds for every (locally bounded, piecewise continuous) activation function that is not a polynomial [44, Sec. 6.4.1]. These theorems concern the approximation of function values. In a PINN the derivatives of the network enter the loss as well, so approximation in a stronger norm is needed. Hornik [49] showed that networks with a smooth, bounded and non-constant activation function can approximate a function together with its derivatives, and De Ryck, Lanthaler and Mishra [50] derived explicit error bounds in Sobolev norms for tanh networks. All these results are existence statements: they guarantee that a suitable network exists, but not that training finds it. This is why the optimisation procedure (Section 4.5.6) matters so much in practice.

3.2 Neural Network Architecture

3.2.1 The Artificial Neuron

The smallest building block is a neuron. It receives an input vector $\mathbf{x} \in \mathbb{R}^{n_{\text{in}}}$, forms a weighted sum with a weight vector $\mathbf{w} \in \mathbb{R}^{n_{\text{in}}}$ and a bias $b \in \mathbb{R}$, and passes the result through a scalar nonlinearity $\phi\colon \mathbb{R} \to \mathbb{R}$, the activation function (Figure 3.1) [44, Sec. 6.3]:

$$z = \sum_{i=1}^{n_{\text{in}}} w_i x_i + b = \mathbf{w}^\top \mathbf{x} + b \in \mathbb{R}, \qquad a = \phi(z) \in \mathbb{R}. \tag{3.1}$$

The scalar z is the pre-activation (or net input) and a is the activation, that is, the output of the neuron. The weights set the relative importance of each input, and the bias shifts the threshold at which the neuron responds. A single neuron with a step activation is exactly Rosenblatt's perceptron [42].

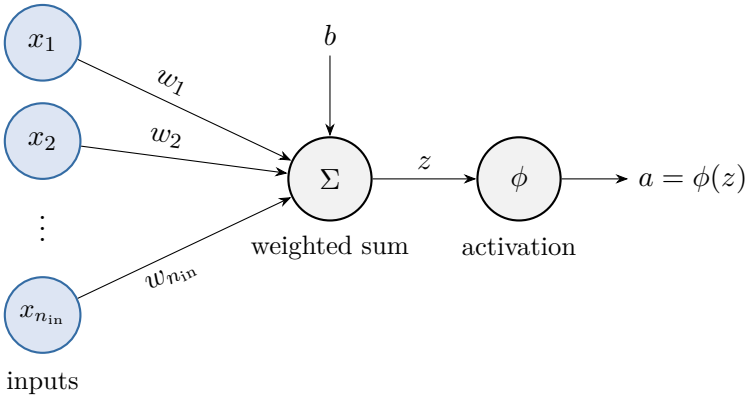


Figure 3.1: A single artificial neuron. The inputs $x_1, \ldots, x_{n_{\text{in}}}$ are multiplied by the weights $w_1, \ldots, w_{n_{\text{in}}}$ and summed together with the bias b, which gives the pre-activation z of (3.1). The activation function ϕ then gives the output $a = \phi(z)$.[51]

3.2.2 The Feed-Forward Network

A feed-forward network, or multilayer perceptron (MLP), arranges neurons in successive layers: the outputs of one layer are the inputs of the next, and information flows in one direction only [44, Ch. 6]. Let $n_0 = n_{\text{in}}$ be the input dimension, let n_l be the number of neurons (the width) of the hidden layer $l = 1, \ldots, L$, and let $n_{L+1} = n_{\text{out}}$ be the output dimension. Collecting the weights of layer l in a matrix $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ and the biases in a vector $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$, the vector form of (3.1) for a whole layer is

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \in \mathbb{R}^{n_l}, \qquad \mathbf{a}^{(l)} = \phi\big(\mathbf{z}^{(l)}\big) \in \mathbb{R}^{n_l}, \qquad l = 1, \ldots, L, \tag{3.2}$$

with $\mathbf{a}^{(0)} = \mathbf{x}$ the input and ϕ applied component-wise. The output layer is left linear, $\mathbf{z}^{(L+1)} = \mathbf{W}^{(L+1)} \mathbf{a}^{(L)} + \mathbf{b}^{(L+1)} \in \mathbb{R}^{n_{\text{out}}}$, so that the output can take any real value. Composing all layers gives the parametric map $\mathcal{F}_\theta\colon \mathbb{R}^{n_{\text{in}}} \to \mathbb{R}^{n_{\text{out}}}$,

$$\mathcal{F}_\theta(\mathbf{x}) = \mathbf{W}^{(L+1)} \phi\Big(\mathbf{W}^{(L)} \phi(\cdots \phi(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}) \cdots) + \mathbf{b}^{(L)}\Big) + \mathbf{b}^{(L+1)} \in \mathbb{R}^{n_{\text{out}}}, \tag{3.3}$$

whose trainable parameters are $\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L+1}$. Collected in one vector, they form $\theta \in \mathbb{R}^{n_\theta}$ with

$$n_\theta = \sum_{l=1}^{L+1} n_l \, (n_{l-1} + 1). \tag{3.4}$$

The number L of hidden layers is the depth of the network. Figure 3.2 shows a small example.

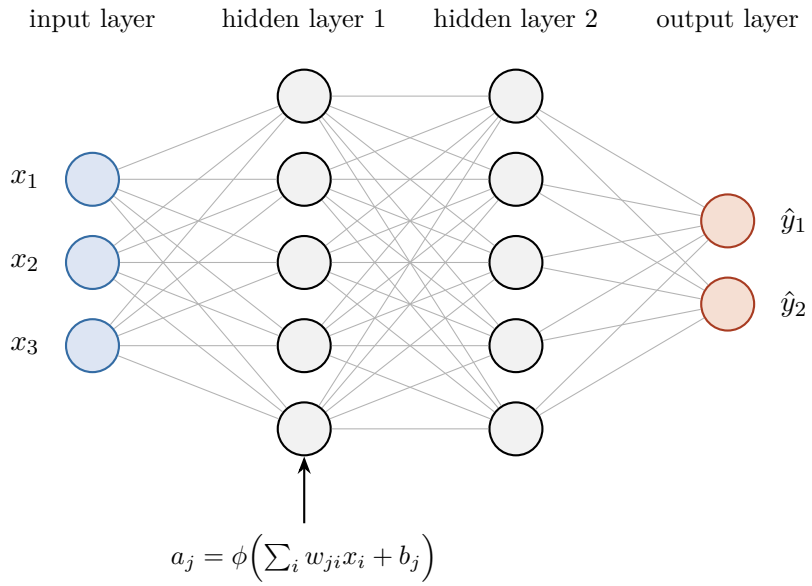


Figure 3.2: A fully connected feed-forward network with two hidden layers ($L = 2$, $n_{\text{in}} = 3$, $n_{\text{out}} = 2$). Each edge carries a weight and each neuron evaluates (3.1). Information propagates from left to right, and the parameters are learned by backpropagation.[51]

3.2.3 Activation Functions

The activation ϕ is the only source of nonlinearity in (3.3). Without it, the composition of affine maps would be a single affine map, and the network could represent only affine functions [44, Sec. 6.3]. The choice of ϕ controls how gradients flow during training and, for PINNs, whether the higher derivatives required by the PDE are well behaved. The most relevant functions are introduced below.

Sigmoid. The logistic sigmoid $\mathrm{sig}\colon \mathbb{R} \to (0, 1)$ maps any real input into the open interval $(0, 1)$,

$$\mathrm{sig}(x) = \frac{1}{1 + e^{-x}}, \qquad \mathrm{sig}'(x) = \mathrm{sig}(x)\big(1 - \mathrm{sig}(x)\big). \tag{3.5}$$

It is smooth and was historically the standard choice, but it saturates: for large $|x|$ the output flattens and the derivative tends to zero. In a deep network these small derivatives are multiplied during backpropagation, so the gradient reaching the early layers can become vanishingly small; this is the vanishing-gradient problem [52].

Hyperbolic tangent. The hyperbolic tangent $\tanh\colon \mathbb{R} \to (-1, 1)$ has the same S-shape but is centred at zero,

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}, \qquad \frac{\mathrm{d}}{\mathrm{d}x}\tanh(x) = 1 - \tanh^2(x). \tag{3.6}$$

Because its outputs are centred on zero, the signals passed to the next layer are not biased towards one sign, which in practice makes tanh train more reliably than the sigmoid [53, Sec. 4.4], [52], [44, Sec. 6.3.2]. It still saturates for large $|x|$, but it is infinitely differentiable, which is important for PINNs, whose loss contains derivatives of the network output [10].

Rectified linear unit (ReLU). The rectified linear unit $\mathrm{ReLU}\colon \mathbb{R} \to [0,\infty)$ sets negative inputs to zero,

$$\mathrm{ReLU}(x) = \max(0,x), \qquad \mathrm{ReLU}'(x) = \begin{cases} 1, & x > 0, \\ 0, & x < 0. \end{cases} \tag{3.7}$$

It was popularised for restricted Boltzmann machines by Nair and Hinton [54], although the function itself is much older [55], and analysed for deep networks by Glorot et al. [56]; see also [44, Sec. 6.3.1]. It does not saturate for $x > 0$, where its derivative is exactly one, so gradients pass through without shrinking and very deep networks can be trained. Its drawbacks are that it is not differentiable at the origin and that its second derivative is zero wherever it exists. This is harmless for image classification, but it rules ReLU out for the PINNs of this thesis: the residuals of Section 4.5.5 contain second derivatives of the velocity, that is, third derivatives of the network output, and these would vanish almost everywhere for a ReLU network.

Smooth alternatives such as the Swish function $x\,\mathrm{sig}(x)$ [57] and adaptive activation functions designed for PINNs [58] also exist. This thesis follows Raissi et al. [10] and uses $\phi = \tanh$.

3.3 Physics-Informed Neural Networks

A PINN uses a feed-forward network $u_\theta\colon \mathbb{R}^{d+1} \to \mathbb{R}$ (a network $\mathcal{F}_\theta$ with $n_{\mathrm{in}} = d+1$ and $n_{\mathrm{out}} = 1$) as a mesh-free approximation of the solution of a PDE and trains it so that it (i) matches the available measurements and (ii) satisfies the governing equation [10]. The key observation is that u_θ is an explicit, differentiable composition of functions, so every derivative in the PDE can be evaluated exactly by the same automatic differentiation that is used for backpropagation, but with respect to the input coordinates instead of the parameters [11]. The PDE is written as a residual and penalised in the loss, which turns the solution of the PDE into an optimisation problem over θ. In this section u denotes a generic scalar unknown; from Chapter 4 onwards, u is the streamwise velocity component and $\mathbf{u}_\theta = (u_\theta, v_\theta)$ the velocity predicted by the network.

3.3.1 General Formulation

Consider a general PDE for a scalar unknown $u\colon \Omega \times [0,t_{\mathrm{f}}] \to \mathbb{R}$ on a spatial domain $\Omega \subset \mathbb{R}^d$ and a time interval $[0,t_{\mathrm{f}}]$ [10],

$$\partial_t u + \mathcal{N}[u;\boldsymbol{\lambda}] = 0, \qquad \mathbf{x} \in \Omega,\ t \in [0,t_{\mathrm{f}}], \tag{3.8}$$

subject to the boundary condition $\mathcal{B}[u] = g$ on $\partial\Omega$ and the initial condition $u(\mathbf{x},0) = u_0(\mathbf{x})$. Here $\mathcal{N}[\,\cdot\,;\boldsymbol{\lambda}]$ is a (generally nonlinear) differential operator in space, and $\boldsymbol{\lambda} \in \mathbb{R}^{n_\lambda}$ collects the physical parameters. The solution is approximated by a network $u_\theta(\mathbf{x},t)$ with input dimension $d+1$ and output dimension one, and the PDE residual is obtained by inserting the network into (3.8):

$$R_\theta(\mathbf{x},t) = \partial_t u_\theta(\mathbf{x},t) + \mathcal{N}[u_\theta;\boldsymbol{\lambda}](\mathbf{x},t). \tag{3.9}$$

The derivatives $\partial_t u_\theta, \partial_x u_\theta, \partial_{xx} u_\theta, \ldots$ are computed by automatic differentiation. If u_θ were the exact solution, R_θ would vanish everywhere; training therefore drives R_θ towards zero at a set of collocation points in the interior of the domain.

The network is trained on a composite loss with three contributions [7, 10],

$$\mathcal{L}(\theta,\boldsymbol{\lambda}) = \mathcal{L}_{\mathrm{data}} + w_{\mathrm{c}}\,\mathcal{L}_{\mathrm{PDE}} + w_{\mathrm{b}}\,\mathcal{L}_{\mathrm{BC}}, \tag{3.10}$$

with

$$\mathcal{L}_{\text{data}} = \frac{1}{N_d} \sum_{i=1}^{N_d} \left| u_\theta(\mathbf{x}_d^i, t_d^i) - u^i \right|^2, \tag{3.11}$$

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_c} \sum_{i=1}^{N_c} \left| R_\theta(\mathbf{x}_c^i, t_c^i) \right|^2, \tag{3.12}$$

$$\mathcal{L}_{\text{BC}} = \frac{1}{N_b} \sum_{i=1}^{N_b} \left| \mathcal{B}[u_\theta](\mathbf{x}_b^i, t_b^i) - g^i \right|^2. \tag{3.13}$$

Here $u^i \in \mathbb{R}$ are the observations at the N_d data points $(\mathbf{x}_d^i, t_d^i) \in \Omega \times [0, t_\text{f}]$, the N_c collocation points $(\mathbf{x}_c^i, t_c^i) \in \Omega \times [0, t_\text{f}]$ enforce the physics, and the N_b boundary points $(\mathbf{x}_b^i, t_b^i) \in \partial\Omega \times [0, t_\text{f}]$ enforce the boundary condition. An initial-condition term of the same form as (3.13) is added when initial data are prescribed; it is omitted here because it is not needed in the inverse problem of this thesis. The weights $w_\text{c}, w_\text{b} > 0$ balance the terms. Choosing them is delicate, because the gradients of the individual terms can differ by orders of magnitude [13]. The residual term acts as a soft constraint: the physics is encouraged through a penalty rather than imposed exactly as in a finite element scheme.

3.3.2 Forward and Inverse Problems

Forward problem. The parameters $\boldsymbol{\lambda}$ are known, and (3.10) is minimised over θ only. With enough boundary and initial data and collocation points, the trained network becomes a mesh-free surrogate of the PDE solution [10].

Inverse problem. Some entries of $\boldsymbol{\lambda}$ are unknown and only sparse, possibly noisy, measurements are available. The unknown entries are added to the trainable variables, and (3.10) is minimised jointly over $(\theta, \boldsymbol{\lambda})$. The residual term ties the recovered $\boldsymbol{\lambda}$ to values that are consistent with the observed data [10].

This thesis concerns the inverse problem for the incompressible Navier–Stokes equations, where the unknown parameters are the coefficient λ_1 of the convective term and the coefficient λ_2 of the viscous term, $\boldsymbol{\lambda} = (\lambda_1, \lambda_2)$ [10]. Chapter 4 derives the governing equations, and Section 4.5 gives the complete PINN formulation for this problem, including the network, the residuals and the loss.

3.4 The Spectral/hp Element Solver Nektar++

The reference data of this thesis are computed with Nektar++, an open-source framework for solving partial differential equations with the spectral/hp element method, developed at Imperial College London and the University of Utah [40, 59].

The method divides the domain into elements, which may be curved to follow boundaries such as the cylinder surface, and represents the solution inside each element by a polynomial of order P. Its accuracy can be raised by refining the mesh or by raising P; for smooth flows the second option converges very quickly and introduces little numerical dissipation, which suits the long-distance transport of the shed vortices in the cylinder wake [60, Chs. 1–2]. The mathematical details of the discretisation are given in Chapter 5.

Chapter 4

Mathematical Formulation

This chapter presents the mathematical formulation of the incompressible Navier–Stokes equations, which follow from mass conservation and the balance of linear momentum and describe how the velocity and pressure fields evolve in a fluid domain [61, Ch. 3], [62, Sec. 1.3]. It also states the initial and boundary conditions that complete the problem. The chapter then explains the stream-function formulation for two-dimensional incompressible flow, which satisfies the continuity equation automatically and is the representation used by the PINN. It ends with the mathematical formulation of the PINN used in this thesis.

Throughout, $d \in \{2, 3\}$ is the spatial dimension, $\Omega \subset \mathbb{R}^d$ is the flow domain with boundary $\partial\Omega$ and outward unit normal $\mathbf{n} \in \mathbb{R}^d$. In Sections 4.1 to 4.4, bold lower-case symbols denote vectors in $\mathbb{R}^d$ and bold upper-case symbols denote second-order tensors, i.e. matrices in $\mathbb{R}^{d\times d}$; in Section 4.5 the dimension of every vector and matrix is stated where it is introduced. Repeated indices are summed from 1 to d, and $\partial_j = \partial/\partial x_j$.

4.1 Navier–Stokes Equations

The Navier–Stokes equations follow from the basic balance laws of continuum mechanics. These laws are first written for a material volume that moves with the fluid and then converted into local differential equations using the transport and divergence theorems [61, Ch. 3], [62, Sec. 1.1], [63, Chs. III–V]. The final form of the equations depends on the constitutive law used for the stress tensor.

Let $\rho(\mathbf{x}, t) \in \mathbb{R}$ be the density, $\mathbf{u}(\mathbf{x}, t) \in \mathbb{R}^d$ the velocity and $p(\mathbf{x}, t) \in \mathbb{R}$ the pressure, for $\mathbf{x} \in \Omega$ and $t \geq 0$.

4.1.1 Kinematics: Flow Map, Material Derivative and Transport Theorem

Flow map and material derivative. Each fluid particle is labelled by the position $\mathbf{X} \in \Omega_0 \subset \mathbb{R}^d$ that it occupies at $t = 0$. Its position at time t is $\mathbf{x} = \mathbf{\Phi}(\mathbf{X}, t) \in \mathbb{R}^d$, where the flow map $\mathbf{\Phi}$ solves the ordinary differential equation [62, Sec. 1.1]

$$\frac{\partial \mathbf{\Phi}}{\partial t}(\mathbf{X}, t) = \mathbf{u}\big(\mathbf{\Phi}(\mathbf{X}, t), t\big), \qquad \mathbf{\Phi}(\mathbf{X}, 0) = \mathbf{X}. \tag{4.1}$$

In words, a particle moves with the local fluid velocity. If $\mathbf{u}$ is continuous in t and Lipschitz continuous in $\mathbf{x}$, the Picard–Lindelöf theorem guarantees a unique local solution of (4.1) for every $\mathbf{X}$; if $\mathbf{u}$ is in addition bounded, this solution exists for all $t \geq 0$. The map $\mathbf{\Phi}(\cdot, t)$ is then invertible, because its inverse is obtained by integrating (4.1) backwards in time, and if $\mathbf{u}$ is smooth, so is $\mathbf{\Phi}(\cdot, t)$ [62, Sec. 1.1]. A material region $\Omega_t = \mathbf{\Phi}(\Omega_0, t)$ always consists of the same fluid particles.

Let $Q\colon \Omega \times [0,\infty) \to \mathbb{R}$ be any smooth scalar field, for example the temperature or one component of $\mathbf{u}$. The rate of change of Q seen by a moving particle follows from the chain rule applied to $Q(\mathbf{\Phi}(\mathbf{X},t),t)$ [61, Ch. 2]:

$$\frac{\mathrm{D}Q}{\mathrm{D}t} := \frac{\mathrm{d}}{\mathrm{d}t} Q(\mathbf{\Phi}(\mathbf{X},t),t) = \frac{\partial Q}{\partial t} + \frac{\partial Q}{\partial x_j}\frac{\partial \Phi_j}{\partial t} = \frac{\partial Q}{\partial t} + (\mathbf{u}\cdot\nabla)Q. \tag{4.2}$$

This is the material derivative. Its two terms have the following meaning:

- $\partial_t Q$ is the local rate of change, that is, how fast Q changes at a fixed point in space;
- $(\mathbf{u}\cdot\nabla)Q = u_j \partial_j Q$ is the convective rate of change: Q changes for the particle simply because the particle is carried to a place where Q has a different value.

Applying (4.2) to each velocity component gives the acceleration of a fluid particle [62, Sec. 1.1],

$$\mathbf{a} = \frac{\mathrm{D}\mathbf{u}}{\mathrm{D}t} = \partial_t \mathbf{u} + (\mathbf{u}\cdot\nabla)\mathbf{u} \in \mathbb{R}^d, \qquad a_i = \partial_t u_i + u_j \partial_j u_i. \tag{4.3}$$

The term $(\mathbf{u}\cdot\nabla)\mathbf{u}$ is quadratic in the unknown $\mathbf{u}$. It is the only nonlinear term of the incompressible Navier–Stokes equations and the source of most of their mathematical difficulty [62, Sec. 1.3].

Volume change of a fluid element. Let $\mathbf{F} \in \mathbb{R}^{d\times d}$ with entries $F_{iJ} = \partial \Phi_i / \partial X_J$ be the deformation gradient, and let $J = \det \mathbf{F} > 0$ be the local ratio of current to initial volume. Differentiating F_{iJ} in time and using (4.1) gives

$$\frac{\partial F_{iJ}}{\partial t} = \frac{\partial}{\partial X_J} u_i(\mathbf{\Phi},t) = (\partial_k u_i)\, F_{kJ}, \qquad \text{i.e.} \quad \dot{\mathbf{F}} = (\nabla \mathbf{u})\, \mathbf{F}, \tag{4.4}$$

where $\nabla\mathbf{u} \in \mathbb{R}^{d\times d}$ is the velocity gradient with entries $(\nabla\mathbf{u})_{ik} = \partial_k u_i$. Jacobi's formula for the derivative of a determinant, $\frac{\mathrm{d}}{\mathrm{d}t}\det\mathbf{F} = \det\mathbf{F}\ \mathrm{tr}(\mathbf{F}^{-1}\dot{\mathbf{F}})$, together with $\mathrm{tr}(\mathbf{F}^{-1}(\nabla\mathbf{u})\mathbf{F}) = \mathrm{tr}(\nabla\mathbf{u}) = \nabla\cdot\mathbf{u}$, yields Euler's expansion formula [62, Sec. 1.1]

$$\frac{\mathrm{D}J}{\mathrm{D}t} = J\,\nabla\cdot\mathbf{u}. \tag{4.5}$$

Thus $\nabla\cdot\mathbf{u}$ is the relative rate at which a small fluid element expands.

Reynolds transport theorem. For a smooth scalar field Q and a material region Ω_t [62, Sec. 1.1], [63, Ch. III],

$$\frac{\mathrm{d}}{\mathrm{d}t}\int_{\Omega_t} Q\,\mathrm{d}\mathbf{x} = \int_{\Omega_t} \big(\partial_t Q + \nabla\cdot(Q\mathbf{u})\big)\,\mathrm{d}\mathbf{x} = \int_{\Omega_t} \partial_t Q\,\mathrm{d}\mathbf{x} + \int_{\partial\Omega_t} Q\,\mathbf{u}\cdot\mathbf{n}\,\mathrm{d}S. \tag{4.6}$$

Proof. Changing variables to the fixed reference region gives $\int_{\Omega_t} Q\,\mathrm{d}\mathbf{x} = \int_{\Omega_0} Q\big(\mathbf{\Phi}(\mathbf{X},t),t\big)\,J\,\mathrm{d}\mathbf{X}$. The domain of integration no longer moves, so the time derivative can be taken under the integral sign, and (4.5) gives

$$\frac{\mathrm{d}}{\mathrm{d}t}\int_{\Omega_0} QJ\,\mathrm{d}\mathbf{X} = \int_{\Omega_0} \Big(\frac{\mathrm{D}Q}{\mathrm{D}t} + Q\,\nabla\cdot\mathbf{u}\Big)J\,\mathrm{d}\mathbf{X} = \int_{\Omega_t} \big(\partial_t Q + \mathbf{u}\cdot\nabla Q + Q\,\nabla\cdot\mathbf{u}\big)\,\mathrm{d}\mathbf{x}. \tag{4.7}$$

Since $\mathbf{u}\cdot\nabla Q + Q\,\nabla\cdot\mathbf{u} = \nabla\cdot(Q\mathbf{u})$, the first form of (4.6) follows, and the divergence theorem gives the second. □

The meaning of (4.6) is that the total amount of Q inside a moving blob of fluid changes for two reasons: because Q itself changes inside the blob (first term on the right) and because Q is carried across the boundary by the flow (second term). For a vector-valued Q the identity holds component by component.

4.1.2 Conservation of Mass

Mass conservation states that the mass of a material volume does not change [61, Ch. 2]:

$$\frac{\mathrm{d}}{\mathrm{d}t}\int_{\Omega_t}\rho\,\mathrm{d}\mathbf{x}=0. \tag{4.8}$$

Applying the Reynolds transport theorem (4.6) and using that Ω_t is arbitrary gives the local continuity equation

$$\frac{\partial\rho}{\partial t}+\nabla\cdot(\rho\mathbf{u})=0, \qquad \text{or equivalently} \qquad \frac{\mathrm{D}\rho}{\mathrm{D}t}+\rho\,\nabla\cdot\mathbf{u}=0. \tag{4.9}$$

For an incompressible flow the density of each fluid particle stays constant, $\mathrm{D}\rho/\mathrm{D}t=0$, and since $\rho>0$, (4.9) gives

$$\nabla\cdot\mathbf{u}=0. \tag{4.10}$$

The incompressibility condition is thus the mass-conservation law for a fluid whose particles keep their density [61, Ch. 2]. In this thesis the fluid is, moreover, assumed to be homogeneous, i.e. ρ is the same constant everywhere. This stronger assumption is needed in Section 4.2, where the momentum equation is divided by ρ and the density is absorbed into the kinematic pressure.

4.1.3 Conservation of Linear Momentum

Two kinds of force act on a material region Ω_t: a body force $\rho\mathbf{f}$ per unit volume, with $\mathbf{f}(\mathbf{x},t)\in\mathbb{R}^d$ the body force per unit mass (for example gravity), and a surface force exerted across $\partial\Omega_t$ by the surrounding fluid. Let $\mathbf{T}(\mathbf{x},\mathbf{n},t)\in\mathbb{R}^d$ be the surface force per unit area, the traction, on a surface element with outward unit normal $\mathbf{n}$. Cauchy's stress theorem states that $\mathbf{T}$ depends linearly on $\mathbf{n}$ [63, Ch. V]:

$$\mathbf{T}=\boldsymbol{\sigma}\mathbf{n}, \qquad T_i=\sigma_{ij}n_j, \tag{4.11}$$

where $\boldsymbol{\sigma}(\mathbf{x},t)\in\mathbb{R}^{d\times d}$ is the Cauchy stress tensor; σ_{ij} is the i-th component of the force per unit area on a surface whose normal points in direction j.

Newton's second law for the material region states that the rate of change of momentum equals the total force [61, Ch. 3]:

$$\frac{\mathrm{d}}{\mathrm{d}t}\int_{\Omega_t}\rho u_i\,\mathrm{d}\mathbf{x}=\int_{\Omega_t}\rho f_i\,\mathrm{d}\mathbf{x}+\int_{\partial\Omega_t}\sigma_{ij}n_j\,\mathrm{d}S, \qquad i=1,\dots,d. \tag{4.12}$$

Applying (4.6) to ρu_i on the left and expanding the products gives

$$\begin{aligned}\int_{\Omega_t}\big[\partial_t(\rho u_i)+\partial_j(\rho u_i u_j)\big]\,\mathrm{d}\mathbf{x}&=\int_{\Omega_t}\Big[u_i\big(\partial_t\rho+\partial_j(\rho u_j)\big)+\rho\big(\partial_t u_i+u_j\partial_j u_i\big)\Big]\,\mathrm{d}\mathbf{x}\\&=\int_{\Omega_t}\rho\frac{\mathrm{D}u_i}{\mathrm{D}t}\,\mathrm{d}\mathbf{x},\end{aligned} \tag{4.13}$$

where the first bracket vanishes by (4.9). On the right of (4.12), the divergence theorem turns the surface integral into $\int_{\Omega_t}\partial_j\sigma_{ij}\,\mathrm{d}\mathbf{x}$. Since Ω_t is arbitrary, the integrands must be equal, which gives Cauchy's equation of motion [61, Ch. 3], [63, Ch. V]

$$\rho\frac{\mathrm{D}u_i}{\mathrm{D}t}=\partial_j\sigma_{ij}+\rho f_i, \qquad \text{i.e.} \qquad \rho(\partial_t\mathbf{u}+(\mathbf{u}\cdot\nabla)\mathbf{u})=\nabla\cdot\boldsymbol{\sigma}+\rho\mathbf{f}, \tag{4.14}$$

where $(\nabla \cdot \boldsymbol{\sigma})_i = \partial_j \sigma_{ij}$. Term by term, (4.14) reads: (mass per unit volume) × (particle acceleration) = (net surface force per unit volume) + (body force per unit volume). The balance of angular momentum, in the absence of internal couples, additionally requires the stress tensor to be symmetric, $\sigma_{ij} = \sigma_{ji}$ [63, Ch. V].

Equation (4.14) holds for any continuum, whether water, steel or rubber. It is not yet a closed system: the unknowns are $\mathbf{u}$ (d functions), ρ, and the symmetric tensor $\boldsymbol{\sigma}$ ($d(d+1)/2$ functions), while only $d+1$ equations are available. The missing information distinguishes one material from another and is supplied by a constitutive law [61, Ch. 3].

4.1.4 Constitutive Law of a Newtonian Fluid

Rate of strain. Near a point $\mathbf{x}_0$, a Taylor expansion gives $\mathbf{u}(\mathbf{x}_0 + \Delta\mathbf{x}) \approx \mathbf{u}(\mathbf{x}_0) + (\nabla\mathbf{u})\,\Delta\mathbf{x}$ for a small displacement $\Delta\mathbf{x} \in \mathbb{R}^d$. The deformation of a small fluid element is described by the symmetric part of the velocity gradient, the rate-of-strain tensor [61, Ch. 2]

$$\mathbf{S} = \tfrac{1}{2}\big(\nabla\mathbf{u} + (\nabla\mathbf{u})^\top\big) \in \mathbb{R}^{d\times d}, \qquad S_{ij} = \tfrac{1}{2}(\partial_j u_i + \partial_i u_j). \tag{4.15}$$

It describes how the element is stretched and sheared, and $\operatorname{tr}\mathbf{S} = \nabla \cdot \mathbf{u}$. The remaining, antisymmetric part of $\nabla\mathbf{u}$ only rotates the element rigidly and does not change its shape [61, Ch. 2].

Stokes's assumptions. In a fluid at rest no direction is preferred, so the stress is a pure pressure, $\boldsymbol{\sigma} = -p\mathbf{I}$, where $\mathbf{I} \in \mathbb{R}^{d\times d}$ is the identity. In a moving fluid one writes

$$\boldsymbol{\sigma} = -p\mathbf{I} + \boldsymbol{\tau}, \tag{4.16}$$

where $\boldsymbol{\tau} \in \mathbb{R}^{d\times d}$ is the viscous stress, the part caused by internal friction. Following Stokes [64] (see also [61, Ch. 3]), three assumptions are made:

1. $\boldsymbol{\tau}$ depends on the flow only through the local velocity gradient, and only through $\mathbf{S}$: a pure rigid rotation ($\mathbf{S} = \mathbf{0}$) produces no internal friction.
2. The dependence is linear: $\tau_{ij} = C_{ijkl} S_{kl}$, with coefficients C_{ijkl} that do not depend on $\mathbf{S}$.
3. The fluid is isotropic: the coefficients C_{ijkl} have the same values in every rotated (and reflected) Cartesian frame.

A classical result of tensor algebra states that every isotropic fourth-order tensor $(C_{ijkl})_{i,j,k,l=1}^{3}$ in three dimensions has the form [65, Ch. 2]

$$C_{ijkl} = \zeta\,\delta_{ij}\delta_{kl} + \mu_1\,\delta_{ik}\delta_{jl} + \mu_2\,\delta_{il}\delta_{jk}, \tag{4.17}$$

with three scalars ζ, μ_1, μ_2, where δ_{ij} is the Kronecker delta. The coefficient ζ is the second (or dilatational) viscosity; it is often written λ in the literature, but ζ is used here to avoid confusion with the PINN parameters λ_1 and λ_2. Inserting this form into assumption 2 and using the symmetry $S_{lk} = S_{kl}$ gives

$$\tau_{ij} = \zeta\,\delta_{ij} S_{kk} + \mu_1 S_{ij} + \mu_2 S_{ji} = \zeta\,(\operatorname{tr}\mathbf{S})\,\delta_{ij} + (\mu_1 + \mu_2)\,S_{ij}. \tag{4.18}$$

Writing $2\mu := \mu_1 + \mu_2$ yields the Newtonian constitutive law [61, Ch. 3], [66, §15]

$$\boldsymbol{\sigma} = -p\mathbf{I} + 2\mu\mathbf{S} + \zeta\,(\nabla \cdot \mathbf{u})\,\mathbf{I}. \tag{4.19}$$

Two-dimensional flows are treated as three-dimensional flows that do not depend on x_3 and have $u_3 = 0$, so (4.19) applies to them unchanged. The three terms mean:

- $-p\mathbf{I}$ is the isotropic pressure, which is present even without motion;
- $2\mu\mathbf{S}$ is the friction caused by shearing and stretching. For the simple shear flow $\mathbf{u} = (\dot{\gamma}x_2, 0, 0)$ with constant shear rate $\dot{\gamma}$ one finds $S_{12} = \dot{\gamma}/2$ and hence $\sigma_{12} = \mu\dot{\gamma}$, which is Newton's law of viscosity: the shear stress is proportional to the velocity gradient, with the dynamic viscosity μ as the constant of proportionality;
- $\zeta(\nabla \cdot \mathbf{u})\mathbf{I}$ is the friction caused by volume change (compression or expansion). For incompressible flow it vanishes by (4.10), so the second viscosity plays no role in this thesis.

4.2 The Incompressible Navier–Stokes Equations

When (4.19) is inserted into Cauchy's equation (4.14) with constant μ, the identity

$$\partial_j(2\mu S_{ij}) = \mu\,\partial_j(\partial_j u_i + \partial_i u_j) = \mu\nabla^2 u_i + \mu\,\partial_i(\nabla \cdot \mathbf{u}) \tag{4.20}$$

shows that all terms that contain $\nabla \cdot \mathbf{u}$ vanish for constant density by (4.10). Dividing by ρ gives the incompressible Navier–Stokes equations [61, Ch. 3], [62, Sec. 1.3]

$$\partial_t\mathbf{u} + (\mathbf{u}\cdot\nabla)\mathbf{u} = -\frac{1}{\rho}\nabla p + \nu\nabla^2\mathbf{u} + \mathbf{f}, \qquad \nabla\cdot\mathbf{u} = 0, \tag{4.21}$$

with the kinematic viscosity $\nu = \mu/\rho$. The terms are:

- $\partial_t\mathbf{u}$: local acceleration (unsteadiness);
- $(\mathbf{u}\cdot\nabla)\mathbf{u}$: convective acceleration (nonlinear inertia);
- $-\rho^{-1}\nabla p$: acceleration caused by the pressure gradient, which pushes fluid from high to low pressure;
- $\nu\nabla^2\mathbf{u}$: viscous diffusion of momentum, which smooths velocity differences in the same way as heat conduction smooths temperature differences;
- $\mathbf{f}$: body force per unit mass.

4.2.1 Convention Used from Here On

If the body force is conservative, $\mathbf{f} = -\nabla\Pi$ with a scalar potential Π (as for gravity), it can be absorbed into the pressure. It is then convenient to redefine p as the kinematic pressure $p/\rho + \Pi$ [61, Ch. 3]. With this convention, or simply when no body force acts,

$$\partial_t\mathbf{u} + (\mathbf{u}\cdot\nabla)\mathbf{u} = -\nabla p + \nu\nabla^2\mathbf{u}, \qquad \nabla\cdot\mathbf{u} = 0. \tag{4.22}$$

In two dimensions, with $\mathbf{u} = (u, v)$ and $\mathbf{x} = (x, y)$, this reads

$$u_t + uu_x + vu_y = -p_x + \nu(u_{xx} + u_{yy}), \tag{4.23}$$

$$v_t + uv_x + vv_y = -p_y + \nu(v_{xx} + v_{yy}), \tag{4.24}$$

$$u_x + v_y = 0, \tag{4.25}$$

that is, three scalar equations for the three unknown functions u, v and p. Subscripts t, x, y denote partial derivatives.

4.2.2 Role of the Pressure

No evolution equation exists for p. Taking the divergence of (4.22) and using $\nabla \cdot \mathbf{u} = 0$ (so that $\partial_t(\nabla \cdot \mathbf{u}) = 0$ and $\nabla \cdot \nabla^2 \mathbf{u} = \nabla^2(\nabla \cdot \mathbf{u}) = 0$) together with

$$\partial_i(u_j \partial_j u_i) = \partial_i u_j \, \partial_j u_i + u_j \, \partial_j(\partial_i u_i) = \partial_i u_j \, \partial_j u_i \tag{4.26}$$

gives the pressure Poisson equation [62, Sec. 1.3]

$$-\nabla^2 p = \partial_i u_j \, \partial_j u_i. \tag{4.27}$$

At each instant, therefore, p is determined by the current velocity field. It acts as a Lagrange multiplier that adjusts instantaneously to preserve $\nabla \cdot \mathbf{u} = 0$ [62, Sec. 1.3].

4.2.3 Initial and Boundary Conditions

System (4.22) is completed by an initial condition $\mathbf{u}(\mathbf{x}, 0) = \mathbf{u}_0(\mathbf{x}) \in \mathbb{R}^d$ with $\nabla \cdot \mathbf{u}_0 = 0$ and by boundary conditions on $\partial\Omega$ [67, Ch. III]. At a solid wall the standard condition is no-slip, $\mathbf{u} = \mathbf{u}_{\text{wall}} \in \mathbb{R}^d$, the velocity of the wall: the fluid sticks to the wall [61, Ch. 1]. At inflow boundaries the velocity is prescribed; at outflow boundaries various conditions are used, for example a zero pressure together with a zero normal derivative of the velocity. Since $\nabla \cdot \mathbf{u} = 0$, the divergence theorem imposes the compatibility condition $\int_{\partial\Omega} \mathbf{u} \cdot \mathbf{n} \, \mathrm{d}S = 0$ when the velocity is prescribed on the whole boundary, and in that case p is determined only up to an additive constant [67, Ch. III].

4.3 The Reynolds Number

4.3.1 Non-Dimensionalisation

Choose a reference length L_{ref} (for example the cylinder diameter D) and a reference velocity U_{ref} (for example the free-stream speed U_∞), and define dimensionless variables, marked by stars [61, Ch. 4]:

$$\mathbf{x} = L_{\text{ref}} \, \mathbf{x}^*, \qquad t = \frac{L_{\text{ref}}}{U_{\text{ref}}} \, t^*, \qquad \mathbf{u} = U_{\text{ref}} \, \mathbf{u}^*, \qquad p = U_{\text{ref}}^2 \, p^*. \tag{4.28}$$

By the chain rule, $\partial_t = (U_{\text{ref}}/L_{\text{ref}}) \, \partial_{t^*}$ and $\nabla = L_{\text{ref}}^{-1} \nabla^*$, so the terms of (4.22) transform as

$$\begin{aligned} \partial_t \mathbf{u} &= \frac{U_{\text{ref}}^2}{L_{\text{ref}}} \, \partial_{t^*} \mathbf{u}^*, \quad (\mathbf{u} \cdot \nabla)\mathbf{u} = \frac{U_{\text{ref}}^2}{L_{\text{ref}}} \, (\mathbf{u}^* \cdot \nabla^*)\mathbf{u}^*, \\ \nabla p &= \frac{U_{\text{ref}}^2}{L_{\text{ref}}} \, \nabla^* p^*, \qquad \nu \nabla^2 \mathbf{u} = \frac{\nu U_{\text{ref}}}{L_{\text{ref}}^2} \, \nabla^{*2} \mathbf{u}^*. \end{aligned} \tag{4.29}$$

Dividing by the common factor $U_{\text{ref}}^2 / L_{\text{ref}}$ gives

$$\partial_{t^*} \mathbf{u}^* + (\mathbf{u}^* \cdot \nabla^*)\mathbf{u}^* = -\nabla^* p^* + \frac{1}{Re} \nabla^{*2} \mathbf{u}^*, \qquad \nabla^* \cdot \mathbf{u}^* = 0, \tag{4.30}$$

where the only remaining dimensionless parameter is the Reynolds number [68], [61, Ch. 4]

$$Re = \frac{U_{\text{ref}} L_{\text{ref}}}{\nu} = \frac{\rho \, U_{\text{ref}} L_{\text{ref}}}{\mu}. \tag{4.31}$$

Hence all flows with different U_{ref}, L_{ref}, ρ and μ collapse onto one family parametrised by Re (and by the shape of the domain and the dimensionless boundary data). This is the principle of dynamic similarity: two flows in geometrically similar domains with the same Re have the same dimensionless solution [66, §19]. From here on the stars are dropped, and all equations are understood in dimensionless form. For the cylinder wake in this thesis, $D = 1$ and $U_\infty = 1$, so $\nu = 1/Re$.

4.3.2 High Reynolds Numbers

When $Re \to \infty$, the viscous term formally disappears and (4.30) becomes the Euler equations. The limit is singular, because the highest derivative is multiplied by the small number $1/Re$; thin boundary layers therefore form near walls, and at sufficiently large Re the flow becomes turbulent [61, Ch. 5]. For the circular cylinder, periodic vortex shedding starts at about $Re \approx 47$, and in a real (three-dimensional) flow the wake becomes three-dimensional at about $Re \approx 190$ [22].

4.4 The Stream Function

Let $\Omega \subset \mathbb{R}^2$ be simply connected and let $\mathbf{u} = (u, v)$ satisfy $\partial_x u + \partial_y v = 0$. Then there is a scalar function $\psi\colon \Omega \times [0, t_{\text{f}}] \to \mathbb{R}$, $\psi = \psi(x, y, t)$, which at each time t is unique up to an additive constant, such that [61, Ch. 2]

$$u = \frac{\partial \psi}{\partial y}, \qquad v = -\frac{\partial \psi}{\partial x}. \tag{4.32}$$

The differential form $u\,\mathrm{d}y - v\,\mathrm{d}x$ is closed: by Green's theorem, its integral over the boundary of any region $A \subset \Omega$ equals $\iint_A (\partial_x u + \partial_y v)\,\mathrm{d}x\,\mathrm{d}y = 0$. On a simply connected domain a closed form is exact (Poincaré lemma), so for a fixed base point $\mathbf{x}_0 \in \Omega$ the line integral

$$\psi(\mathbf{x}, t) = \int_{\mathbf{x}_0}^{\mathbf{x}} (u\,\mathrm{d}y - v\,\mathrm{d}x) \tag{4.33}$$

is independent of the path, and $\mathrm{d}\psi = \psi_x\,\mathrm{d}x + \psi_y\,\mathrm{d}y = -v\,\mathrm{d}x + u\,\mathrm{d}y$, which is (4.32). Changing the base point $\mathbf{x}_0$ changes ψ only by a constant.

Both sign conventions for ψ occur in the literature, so the convention should always be stated; (4.32) is the one used by Raissi et al. [10]. For flow around an obstacle the domain is not simply connected, but ψ is still single-valued, because the net flux through any closed curve around the obstacle equals the flux through the impermeable obstacle, which is zero [61, Ch. 2]. In the simulations of Chapter 6 the free-stream velocity is prescribed on the lateral boundaries $y = \pm 8$, so the cylinder is the only obstacle to simple connectedness, and ψ is single-valued in the whole domain. In any case, the PINN represents ψ only in a simply connected window inside the domain.

Continuity is satisfied automatically, since $\partial_x(\psi_y) + \partial_y(-\psi_x) = 0$ for every twice continuously differentiable ψ. Working with ψ therefore removes the equation $\nabla \cdot \mathbf{u} = 0$ and reduces the number of unknown velocity functions from two to one. Moreover, the curves $\psi = \text{const}$ are streamlines, since $\mathbf{u} \cdot \nabla\psi = u\psi_x + v\psi_y = \psi_y\psi_x - \psi_x\psi_y = 0$.

4.4.1 Pressure from the Stream Function

The stream function also determines the pressure. Inserting (4.32) into the right-hand side of (4.27), with $u_x = \psi_{xy}$, $v_y = -\psi_{xy}$, $u_y = \psi_{yy}$ and $v_x = -\psi_{xx}$, gives

$$\partial_i u_j \, \partial_j u_i = u_x^2 + v_y^2 + 2u_y v_x = 2\psi_{xy}^2 - 2\psi_{xx}\psi_{yy}, \tag{4.34}$$

and hence

$$\nabla^2 p = 2(\psi_{xx}\psi_{yy} - \psi_{xy}^2) = 2\det\begin{pmatrix} \psi_{xx} & \psi_{xy} \\ \psi_{xy} & \psi_{yy} \end{pmatrix}, \tag{4.35}$$

that is, twice the determinant of the 2×2 Hessian matrix of ψ. The pressure therefore follows from ψ by solving one Poisson equation (with boundary data from the momentum equation). This is why a PINN that represents only ψ and p can recover a pressure field that is consistent with the velocity, even though no pressure data are given [10].

4.4.2 Stream Function in the Inverse Problem

This formulation is exactly what physics-informed neural networks exploit when identifying flow parameters from data. Raissi, Perdikaris and Karniadakis [10] represent the velocity of the cylinder wake through two network outputs (ψ, p) with $u = \psi_y$, $v = -\psi_x$, so that (4.25) holds identically, and learn the unknown coefficients λ_1 and λ_2 in

$$u_t + \lambda_1(uu_x + vu_y) = -p_x + \lambda_2(u_{xx} + u_{yy}) \tag{4.36}$$

(and the analogous v-equation) from data. In the dimensionless form (4.30), the exact values are $\lambda_1 = 1$ and $\lambda_2 = 1/Re$; for the wake at $Re = 100$ this means $\lambda_2 = 0.01$, and at $Re = 500$ it means $\lambda_2 = 0.002$.

4.5 PINN Mathematical Formulation

4.5.1 Network Ansatz

Following the PINN framework of Raissi, Perdikaris and Karniadakis [10], which the authors applied to the cylinder wake in the same work, the feed-forward network of Section 3.2 is used to represent the stream function ψ and the pressure p as functions of position and time [44, Ch. 6]. It is the network (3.3) with $\phi = \tanh$, input dimension $n_0 = 3$ (the coordinates x, y, t) and output dimension $n_{L+1} = 2$ (the values of ψ and p). For later reference, the layer-by-layer definition of this network $\mathbb{R}^3 \to \mathbb{R}^2$ reads

$$\mathbf{a}^{(0)} := (x, y, t)^\top \in \mathbb{R}^3, \tag{4.37}$$

$$\mathbf{z}^{(l)} := \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \in \mathbb{R}^{n_l}, \qquad \mathbf{a}^{(l)} := \tanh(\mathbf{z}^{(l)}) \in \mathbb{R}^{n_l}, \qquad l = 1, \dots, L, \tag{4.38}$$

$$(\psi_\theta, p_\theta)^\top := \mathbf{z}^{(L+1)} = \mathbf{W}^{(L+1)}\mathbf{a}^{(L)} + \mathbf{b}^{(L+1)} \in \mathbb{R}^2, \tag{4.39}$$

where $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ and $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$ are the weight matrix and bias vector of layer l, tanh is applied element-wise, and the output layer is linear, so that ψ_θ and p_θ can take any real value. The collection $\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L+1} \in \mathbb{R}^{n_\theta}$, with n_θ given by (3.4), contains every trainable weight and bias of the network. For the network used here ($L = 8$, $n_l = 20$) this gives $n_\theta = 20 \cdot 4 + 7 \cdot 20 \cdot 21 + 2 \cdot 21 = 3062$. The physical parameters $\lambda_1, \lambda_2 \in \mathbb{R}$ are trained together with θ but are not part of it. The activation is tanh rather than, for example, ReLU, because tanh is infinitely differentiable and the residual needs third derivatives of ψ_θ (Section 3.2). In practice the inputs are first scaled linearly to $[-1, 1]$; this affine map is part of the first layer and does not change the formulation.

4.5.2 Backpropagation

Training requires the gradient of the scalar loss $\mathcal{L}$ with respect to every weight and bias in (4.38). Backpropagation computes this gradient exactly with one forward and one backward pass through the network, regardless of the number of weights [45], [44, Sec. 6.5].

The recursion is first written for a loss that depends on the network output at one input point $\mathbf{a}^{(0)} \in \mathbb{R}^3$. For the PINN loss (4.49), which sums over many points and also contains derivatives of the output with respect to the inputs, the same principle is applied to the larger computational graph that includes these derivatives (Section 4.5.4), and the contributions of all points are added. For each layer $l = 1, \dots, L+1$, the error of the layer is defined as the sensitivity of the loss to the pre-activation of that layer,

$$\boldsymbol{\delta}^{(l)} := \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{n_l}. \tag{4.40}$$

At the output layer, $\boldsymbol{\delta}^{(L+1)} = \partial \mathcal{L} / \partial(\psi_\theta, p_\theta) \in \mathbb{R}^2$. For a hidden layer, the loss depends on $\mathbf{z}^{(l)}$ only through $\mathbf{a}^{(l)} = \tanh(\mathbf{z}^{(l)})$ and then only through the next pre-activation $\mathbf{z}^{(l+1)} = \mathbf{W}^{(l+1)}\mathbf{a}^{(l)} + \mathbf{b}^{(l+1)}$, so the chain rule gives, component-wise for $i = 1, \dots, n_l$,

$$\frac{\partial \mathcal{L}}{\partial z_i^{(l)}} = \sum_{j=1}^{n_{l+1}} \frac{\partial \mathcal{L}}{\partial z_j^{(l+1)}} \frac{\partial z_j^{(l+1)}}{\partial a_i^{(l)}} \frac{\partial a_i^{(l)}}{\partial z_i^{(l)}} = \sum_{j=1}^{n_{l+1}} \delta_j^{(l+1)} \, W_{ji}^{(l+1)} \, \tanh'(z_i^{(l)}), \tag{4.41}$$

which in matrix–vector form is the backward recursion

$$\boldsymbol{\delta}^{(l)} = \left(\left(\mathbf{W}^{(l+1)}\right)^\top \boldsymbol{\delta}^{(l+1)} \right) \odot \tanh'(\mathbf{z}^{(l)}), \qquad l = L, L-1, \dots, 1, \tag{4.42}$$

with $\odot$ the element-wise product of two vectors in $\mathbb{R}^{n_l}$ and $\tanh'(z) = 1 - \tanh^2(z)$. Given $\boldsymbol{\delta}^{(l)}$, the gradients with respect to the weights and biases of that layer follow at once: since $z_i^{(l)} = \sum_j W_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$,

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(l)}} = \delta_i^{(l)} a_j^{(l-1)} \iff \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} \left(\mathbf{a}^{(l-1)}\right)^\top \in \mathbb{R}^{n_l \times n_{l-1}}, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)} \in \mathbb{R}^{n_l}. \tag{4.43}$$

4.5.3 Enforcing Incompressibility

Instead of penalising $\nabla \cdot \mathbf{u} \neq 0$ in the loss, the velocity components are obtained directly from the stream-function output by differentiation [10],

$$u_\theta = \frac{\partial \psi_\theta}{\partial y}, \qquad v_\theta = -\frac{\partial \psi_\theta}{\partial x}. \tag{4.44}$$

Because $\partial^2 \psi_\theta / \partial x \partial y = \partial^2 \psi_\theta / \partial y \partial x$ for the smooth network output, (4.44) satisfies $\partial_x u_\theta + \partial_y v_\theta \equiv 0$ for every choice of the weights θ. The continuity equation therefore never has to be learned; only the momentum residuals of Section 4.5.5 enter the loss.

4.5.4 Automatic Differentiation

Evaluating (4.44) and the PDE residual requires derivatives of the network output with respect to its inputs (x, y, t), whereas backpropagation differentiates with respect to the weights θ. Both are obtained by the same mechanism, automatic differentiation (AD) [11], [69, Ch. 3], which applies the chain rule to the sequence of operations in (4.38) and is therefore exact

up to rounding errors, unlike a finite-difference approximation. With $\psi_\theta = \mathbf{e}_1^\top \mathbf{z}^{(L+1)}$, where $\mathbf{e}_1 = (1,0)^\top \in \mathbb{R}^2$, and $\mathbf{a}^{(0)} = (x,y,t)^\top \in \mathbb{R}^3$,

$$\frac{\partial \psi_\theta}{\partial \mathbf{a}^{(0)}} = \mathbf{e}_1^\top \underbrace{\frac{\partial \mathbf{z}^{(L+1)}}{\partial \mathbf{a}^{(L)}}}_{2\times n_L} \underbrace{\frac{\partial \mathbf{a}^{(L)}}{\partial \mathbf{z}^{(L)}}}_{n_L \times n_L} \cdots \underbrace{\frac{\partial \mathbf{z}^{(1)}}{\partial \mathbf{a}^{(0)}}}_{n_1 \times 3} \in \mathbb{R}^{1\times 3}, \tag{4.45}$$

whose three entries are $\partial\psi_\theta/\partial x$, $\partial\psi_\theta/\partial y$ and $\partial\psi_\theta/\partial t$. Here $\partial \mathbf{z}^{(L+1)}/\partial \mathbf{a}^{(L)} = \mathbf{W}^{(L+1)}$, $\partial \mathbf{a}^{(l)}/\partial \mathbf{z}^{(l)} = \operatorname{diag}(\tanh'(\mathbf{z}^{(l)}))$ and $\partial \mathbf{z}^{(1)}/\partial \mathbf{a}^{(0)} = \mathbf{W}^{(1)}$. As in backpropagation, this product of Jacobian matrices is evaluated from left to right, i.e. backwards through the network (reverse-mode AD). Writing the whole forward computation as a generic chain $\mathbf{c}^{(0)} \to \cdots \to \mathbf{c}^{(M)}$ of intermediate results $\mathbf{c}^{(r)} \in \mathbb{R}^{m_r}$ and defining the adjoint row vector $\bar{\mathbf{c}}^{(r)} := \partial y/\partial \mathbf{c}^{(r)} \in \mathbb{R}^{1\times m_r}$ for a scalar output $y = \mathbf{c}^{(M)}$ ($m_M = 1$), the chain rule gives the general backward recursion [69, Ch. 3]

$$\bar{\mathbf{c}}^{(M)} = 1, \qquad \bar{\mathbf{c}}^{(r-1)} = \bar{\mathbf{c}}^{(r)}\,\mathbf{D}_r, \qquad \mathbf{D}_r = \frac{\partial \mathbf{c}^{(r)}}{\partial \mathbf{c}^{(r-1)}} \in \mathbb{R}^{m_r \times m_{r-1}}, \qquad r = M, \dots, 1. \tag{4.46}$$

Seeding (4.46) at the loss and stopping at the weights gives backpropagation (4.42); seeding it at the output ψ_θ and continuing to the input layer gives (4.45). The cost of one backward sweep is a small constant multiple of the cost of one forward pass, regardless of the number of inputs or weights [69, Ch. 3]. This is why reverse-mode AD, rather than a finite-difference stencil or forward-mode AD, is used for networks with many weights. Higher derivatives such as $u_{\theta,xx} = \psi_{\theta,yxx}$, needed for the viscous term, are obtained by applying (4.46) again to the computational graph of the lower derivative.

4.5.5 PDE Residual and Loss Function

Inserting the network output into the momentum equations (4.23)–(4.24), with λ_1 in front of the convective terms and λ_2 in place of ν, gives two scalar residuals, one per velocity component [10]:

$$R_\theta^u = u_{\theta,t} + \lambda_1\big(u_\theta u_{\theta,x} + v_\theta u_{\theta,y}\big) + p_{\theta,x} - \lambda_2\big(u_{\theta,xx} + u_{\theta,yy}\big), \tag{4.47}$$

$$R_\theta^v = v_{\theta,t} + \lambda_1\big(u_\theta v_{\theta,x} + v_\theta v_{\theta,y}\big) + p_{\theta,y} - \lambda_2\big(v_{\theta,xx} + v_{\theta,yy}\big). \tag{4.48}$$

Both residuals are functions of (x,y,t), and the subscripts after the comma denote partial derivatives. They vanish exactly when u_θ, v_θ and p_θ solve the momentum equations; away from a solution they measure how strongly the equations are violated at a given point. Given N_d labelled data points $(x_i, y_i, t_i, u_i, v_i) \in \mathbb{R}^5$, $i = 1, \dots, N_d$, and N_c collocation points $(x_j, y_j, t_j) \in \mathbb{R}^3$, $j = 1, \dots, N_c$, the network is trained to minimise the combined data and physics loss [10]

$$\begin{aligned}\mathcal{L}(\theta, \lambda_1, \lambda_2) = \frac{1}{N_d}\sum_{i=1}^{N_d}\Big[\big(u_\theta(x_i,y_i,t_i) - u_i\big)^2 + \big(v_\theta(x_i,y_i,t_i) - v_i\big)^2\Big] \\ + \frac{1}{N_c}\sum_{j=1}^{N_c}\Big[R_\theta^u(x_j,y_j,t_j)^2 + R_\theta^v(x_j,y_j,t_j)^2\Big].\end{aligned} \tag{4.49}$$

In the implementation of Raissi et al. and in this thesis the collocation points coincide with the data points, $N_c = N_d$ [10]. The first term fits the network to the observed velocities; the second penalises departures from the physics. Because λ_1 and λ_2 are also trainable, minimising $\mathcal{L}$ simultaneously fits the flow field and identifies the coefficients that make it consistent with the Navier–Stokes equations [10].

In contrast to the general loss (3.10), no boundary or initial-condition terms appear in (4.49). The training window lies inside the flow domain, and the velocity and pressure on its boundary are unknown; the solution is constrained only by the velocity data and the momentum residuals. This is possible because the data are distributed over the whole window and over all snapshots, and not only on its boundary [10].

The gradients of the loss with respect to the two physical parameters follow directly from the chain rule, since λ_1 and λ_2 enter the residuals linearly (all quantities in the brackets are evaluated at (x_j, y_j, t_j)):

$$\frac{\partial \mathcal{L}}{\partial \lambda_1} = \frac{2}{N_c} \sum_{j=1}^{N_c} \Big[R_\theta^u \big(u_\theta u_{\theta,x} + v_\theta u_{\theta,y} \big) + R_\theta^v \big(u_\theta v_{\theta,x} + v_\theta v_{\theta,y} \big) \Big], \tag{4.50}$$

$$\frac{\partial \mathcal{L}}{\partial \lambda_2} = -\frac{2}{N_c} \sum_{j=1}^{N_c} \Big[R_\theta^u \big(u_{\theta,xx} + u_{\theta,yy} \big) + R_\theta^v \big(v_{\theta,xx} + v_{\theta,yy} \big) \Big]. \tag{4.51}$$

These expressions suggest why λ_2 is harder to identify than λ_1. The parameter λ_2 enters the residuals only through the term $\lambda_2 \nabla^2 \mathbf{u}_\theta$, with $\mathbf{u}_\theta = (u_\theta, v_\theta) \in \mathbb{R}^2$, whose size is $1/Re$ times the second derivatives of the fitted velocity. Any error in these second derivatives, which are the least accurate derivatives of the network, is therefore comparable to the term that determines λ_2. The convective term, in contrast, is not multiplied by a small factor, so λ_1 is constrained much more strongly by the data. The size of the gradient alone does not explain the difference, because Adam (Section 4.5.6) rescales the step of every parameter by its own gradient history; what matters is how sensitively the residuals react to a change of each parameter compared with the fitting error of the network.

4.5.6 Optimisation

The PINN is trained by minimising the loss (4.49) with respect to both the network parameters and the two unknown physical coefficients. All trainable quantities are collected in one vector

$$\Theta = (\theta, \lambda_1, \lambda_2) \in \mathbb{R}^{n_\Theta}, \qquad n_\Theta = n_\theta + 2, \tag{4.52}$$

where $\theta \in \mathbb{R}^{n_\theta}$ contains all weights and biases (Section 4.5.1), so that training solves the unconstrained minimisation problem

$$\min_{\Theta \in \mathbb{R}^{n_\Theta}} \mathcal{L}(\Theta), \qquad \mathcal{L} \colon \mathbb{R}^{n_\Theta} \to [0, \infty). \tag{4.53}$$

Because tanh is smooth, $\mathcal{L}$ is infinitely differentiable, and its gradient $\nabla_\Theta \mathcal{L} \in \mathbb{R}^{n_\Theta}$ is computed exactly by reverse-mode automatic differentiation (Section 4.5.4). Following Raissi et al. [10], training is carried out in two stages: Adam first reduces the loss with adaptive first-order steps, and L-BFGS-B then refines the solution with approximate curvature information. In both stages $k = 0, 1, 2, \ldots$ denotes the iteration index, $\Theta_k \in \mathbb{R}^{n_\Theta}$ the current iterate, and

$$\mathbf{g}_k := \nabla_\Theta \mathcal{L}(\Theta_k) \in \mathbb{R}^{n_\Theta} \tag{4.54}$$

the gradient at this iterate.

Adam. Adam is a first-order method introduced by Kingma and Ba [70]. It keeps exponentially weighted estimates of the first and second moments of the gradient and uses them to scale the step of every component of Θ separately. This gives each trainable quantity its own effective step size at the cost of only two additional vectors of length n_Θ, which makes the method suitable for problems with many parameters [70, Secs. 1–2].

Starting from $\mathbf{m}_0 = \mathbf{h}_0 = \mathbf{0} \in \mathbb{R}^{n_\Theta}$, Adam forms for $k = 0, 1, 2, \ldots$ the moving averages

$$\mathbf{m}_{k+1} = \beta_1 \mathbf{m}_k + (1 - \beta_1)\, \mathbf{g}_k, \qquad \mathbf{h}_{k+1} = \beta_2 \mathbf{h}_k + (1 - \beta_2)\, \mathbf{g}_k \odot \mathbf{g}_k, \tag{4.55}$$

where $\odot$ denotes the component-wise product and the decay rates $\beta_1, \beta_2 \in [0, 1)$ determine how quickly older gradients are forgotten [70, Alg. 1]. The vector $\mathbf{m}_{k+1} \in \mathbb{R}^{n_\Theta}$ is a smoothed estimate of the gradient, and $\mathbf{h}_{k+1} \in \mathbb{R}^{n_\Theta}$ records, component by component, the typical size of the squared gradient.

Bias correction. Because both averages start at zero, their early values are biased towards zero. Unrolling (4.55) gives

$$\mathbf{m}_{k+1} = (1 - \beta_1) \sum_{i=0}^{k} \beta_1^{k-i}\, \mathbf{g}_i. \tag{4.56}$$

If the gradients have a common expectation, $\mathbb{E}[\mathbf{g}_i] = \mathbb{E}[\mathbf{g}]$ for $i = 0, \ldots, k$, the geometric sum $\sum_{i=0}^{k} \beta_1^{k-i} = (1 - \beta_1^{k+1})/(1 - \beta_1)$ gives $\mathbb{E}[\mathbf{m}_{k+1}] = (1 - \beta_1^{k+1})\, \mathbb{E}[\mathbf{g}]$, and in the same way $\mathbb{E}[\mathbf{h}_{k+1}] = (1 - \beta_2^{k+1})\, \mathbb{E}[\mathbf{g} \odot \mathbf{g}]$ [70, Sec. 3]. Adam therefore uses the corrected estimates

$$\hat{\mathbf{m}}_{k+1} = \frac{\mathbf{m}_{k+1}}{1 - \beta_1^{k+1}}, \qquad \hat{\mathbf{h}}_{k+1} = \frac{\mathbf{h}_{k+1}}{1 - \beta_2^{k+1}}, \tag{4.57}$$

which are unbiased under this assumption.

Update. The trainable vector is then updated according to [70, Alg. 1]

$$\Theta_{k+1} = \Theta_k - \eta\, \hat{\mathbf{m}}_{k+1} \oslash \Big(\sqrt{\hat{\mathbf{h}}_{k+1}} + \epsilon \Big), \tag{4.58}$$

where $\eta > 0$ is the learning rate, $\epsilon > 0$ prevents division by zero, and the square root and the division $\oslash$ act component by component. Kingma and Ba recommend $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$ [70, Alg. 1]. TensorFlow implements the slightly reordered form that Kingma and Ba give at the end of their Section 2, in which the two bias corrections are folded into the step size [70, 71]:

$$\tilde{\eta}_{k+1} = \eta\, \frac{\sqrt{1 - \beta_2^{k+1}}}{1 - \beta_1^{k+1}}, \qquad \Theta_{k+1} = \Theta_k - \tilde{\eta}_{k+1}\, \mathbf{m}_{k+1} \oslash \Big(\sqrt{\mathbf{h}_{k+1}} + \hat{\epsilon} \Big). \tag{4.59}$$

Here $\tilde{\eta}_{k+1}$ is the bias-corrected step size, $\mathbf{m}_{k+1}$ and $\mathbf{h}_{k+1}$ are the uncorrected moment estimates of (4.55), and $\hat{\epsilon}$ is the regularisation constant of this form. Inserting (4.57) into (4.58) shows that both forms give the same step when $\hat{\epsilon} = \epsilon \sqrt{1 - \beta_2^{k+1}}$, so they differ only in how the small constant enters. TensorFlow's default $\hat{\epsilon} = 10^{-7}$ is used in this thesis (Table 6.4).

Two properties of (4.58) matter for the present inverse problem [70, Sec. 2.1]. First, the step of each component is approximately bounded by the learning rate, $|(\Theta_{k+1} - \Theta_k)_i| \lesssim \eta$, independently of the size of the gradient. Second, if the loss is multiplied by a constant $c > 0$, then $\mathbf{g}_k$, $\mathbf{m}_k$ and $\sqrt{\mathbf{h}_k}$ are all multiplied by c, so the update is unchanged apart from the influence of ϵ. Both properties are useful here, because the network weights and the physical coefficients λ_1 and λ_2 produce gradients of very different size.

L-BFGS-B. After the Adam stage, the optimisation is continued with L-BFGS-B (limited-memory Broyden–Fletcher–Goldfarb–Shanno with bounds). Byrd, Lu, Nocedal and Zhu [72] developed this method for large nonlinear problems with simple lower and upper bounds on the variables. It combines the quasi-Newton idea with the limited-memory storage of Liu and Nocedal [73], so that curvature information is used without forming an $n_\Theta \times n_\Theta$ matrix.

Quasi-Newton step. Near Θ_k, the loss is approximated by the quadratic model

$$\mathcal{L}(\Theta_k + \mathbf{p}) \approx \mathcal{L}(\Theta_k) + \mathbf{g}_k^\top \mathbf{p} + \tfrac{1}{2}\mathbf{p}^\top \nabla^2 \mathcal{L}(\Theta_k)\, \mathbf{p}, \qquad \mathbf{p} \in \mathbb{R}^{n_\Theta}, \tag{4.60}$$

where $\nabla^2 \mathcal{L}(\Theta_k) \in \mathbb{R}^{n_\Theta \times n_\Theta}$ is the Hessian. If the Hessian is positive definite, the model is minimised by the Newton direction $\mathbf{p} = -[\nabla^2 \mathcal{L}(\Theta_k)]^{-1} \mathbf{g}_k$ [74, Sec. 2.2]. Computing and inverting the Hessian is expensive, and away from a minimum it is in general not positive definite [74, Sec. 3.4]. Quasi-Newton methods therefore replace the inverse Hessian by a symmetric positive definite matrix $\mathbf{H}_k \in \mathbb{R}^{n_\Theta \times n_\Theta}$ that is built from gradient differences alone, and take the step

$$\Theta_{k+1} = \Theta_k + \alpha_k\, \mathbf{p}_k, \qquad \mathbf{p}_k = -\mathbf{H}_k \mathbf{g}_k \in \mathbb{R}^{n_\Theta} \tag{4.61}$$

[74, Sec. 6.1]. Since $\mathbf{H}_k$ is positive definite, $\mathbf{g}_k^\top \mathbf{p}_k = -\mathbf{g}_k^\top \mathbf{H}_k \mathbf{g}_k < 0$ whenever $\mathbf{g}_k \neq \mathbf{0}$, so $\mathbf{p}_k$ is a descent direction. The step length $\alpha_k > 0$ is chosen by a line search that satisfies the Wolfe conditions [74, Sec. 3.1, Eq. (3.6)]

$$\mathcal{L}(\Theta_k + \alpha_k \mathbf{p}_k) \leq \mathcal{L}(\Theta_k) + c_1\, \alpha_k\, \mathbf{g}_k^\top \mathbf{p}_k, \qquad \nabla_\Theta \mathcal{L}(\Theta_k + \alpha_k \mathbf{p}_k)^\top \mathbf{p}_k \geq c_2\, \mathbf{g}_k^\top \mathbf{p}_k, \tag{4.62}$$

with constants $0 < c_1 < c_2 < 1$: the first condition demands a sufficient decrease of the loss, the second excludes steps that are too short.

BFGS update. For two consecutive iterates define

$$\mathbf{s}_k := \Theta_{k+1} - \Theta_k \in \mathbb{R}^{n_\Theta}, \qquad \mathbf{y}_k := \mathbf{g}_{k+1} - \mathbf{g}_k \in \mathbb{R}^{n_\Theta}. \tag{4.63}$$

The vector $\mathbf{s}_k$ records the parameter displacement and $\mathbf{y}_k$ the corresponding change of the gradient. The new matrix is required to satisfy the secant equation $\mathbf{H}_{k+1}\mathbf{y}_k = \mathbf{s}_k$, which reproduces the observed curvature along the last step. Among all symmetric matrices with this property, BFGS chooses the one closest to $\mathbf{H}_k$ in a weighted norm, which gives [74, Sec. 6.1, Eq. (6.17)]

$$\mathbf{H}_{k+1} = \left(\mathbf{I} - \varrho_k\, \mathbf{s}_k \mathbf{y}_k^\top\right) \mathbf{H}_k \left(\mathbf{I} - \varrho_k\, \mathbf{y}_k \mathbf{s}_k^\top\right) + \varrho_k\, \mathbf{s}_k \mathbf{s}_k^\top, \qquad \varrho_k := \frac{1}{\mathbf{y}_k^\top \mathbf{s}_k}, \tag{4.64}$$

where $\mathbf{I}$ is the $n_\Theta \times n_\Theta$ identity matrix. The second Wolfe condition in (4.62) implies the curvature condition $\mathbf{y}_k^\top \mathbf{s}_k \geq (c_2 - 1)\, \alpha_k\, \mathbf{g}_k^\top \mathbf{p}_k > 0$, so ϱ_k is well defined and $\mathbf{H}_{k+1}$ stays symmetric positive definite [74, Sec. 6.1].

Limited memory (L). Storing the dense matrix $\mathbf{H}_k$ would require n_Θ^2 numbers. L-BFGS therefore keeps only the m most recent pairs $(\mathbf{s}_i, \mathbf{y}_i)$, $i = k-m, \ldots, k-1$ [73]. Values between 3 and 20 are usually sufficient [74, Sec. 7.2]; this thesis uses $m = 50$, as in the code of Raissi et al. [10]. The matrix $\mathbf{H}_k$ is defined implicitly by starting from the scaled identity $\mathbf{H}_k^0 = \vartheta_k \mathbf{I}$ with $\vartheta_k = \mathbf{s}_{k-1}^\top \mathbf{y}_{k-1} / (\mathbf{y}_{k-1}^\top \mathbf{y}_{k-1}) > 0$ and applying the update (4.64) with these m pairs; it is never formed explicitly. The product $\mathbf{H}_k \mathbf{g}_k$ can be computed directly from the stored pairs by the two-loop recursion of Appendix C [74, Sec. 7.2, Alg. 7.4], which needs $2m\, n_\Theta$ stored numbers and $\mathcal{O}(m\, n_\Theta)$ operations per iteration instead of $\mathcal{O}(n_\Theta^2)$. The L-BFGS-B code used here does not apply the two-loop recursion itself but the mathematically equivalent compact representation of Byrd, Nocedal and Schnabel [75], which writes the limited-memory matrix as a low-rank correction of $\vartheta_k^{-1}\mathbf{I}$ and is better suited to the handling of bounds [72]. Both forms define the same matrix; the two-loop recursion is shown because it explains the idea most directly.

Bounds (B). L-BFGS-B extends L-BFGS to problems with simple lower and upper bounds on the variables by projecting the steps onto the feasible box [72]. In this thesis no bounds are imposed on the network parameters or on λ_1 and λ_2, so L-BFGS-B reduces to the L-BFGS method described above. In particular, λ_2 is not restricted to positive values, so that the

parameters are identified only through the data and the residual and not through prior knowledge of their sign. L-BFGS-B is used because its SciPy implementation [76], which is based on the Fortran code of Zhu et al. [77], is well tested and was also used by Raissi et al. [10].

Stopping criteria. The iteration stops when the relative decrease of the loss becomes small,

$$\frac{\mathcal{L}(\Theta_k) - \mathcal{L}(\Theta_{k+1})}{\max\{|\mathcal{L}(\Theta_k)|,\, |\mathcal{L}(\Theta_{k+1})|,\, 1\}} \leq f_{\mathrm{tol}}, \tag{4.65}$$

when the largest absolute component of the gradient $\mathbf{g}_k$ falls below g_{tol} (without bounds the projected gradient used by the code equals the gradient), or when the maximum number of iterations or function evaluations is reached [76, 77]. The values used are listed in Table 6.4.

Role of the two stages. The two optimisers play complementary roles. Adam makes robust progress from the random initialisation, where the quadratic model (4.60) is a poor description of $\mathcal{L}$. Near a minimum the model becomes accurate, and L-BFGS-B uses the observed changes of the gradient to take larger and better scaled steps, which in the experiments of Chapter 7 is where most of the final accuracy of λ_2 is reached.

Chapter 5

The Spectral/hp Element Method in Nektar++

The reference data are computed by solving the non-dimensional Navier–Stokes equations (4.30) with $\nu = 1/Re$ in Nektar++ [40, 59]. This chapter summarises only the three ingredients of the solver that are needed to understand the data and the settings in Table 6.2: the weak form, the spectral/hp element basis, and the splitting scheme in time. Further details can be found in [60, Chs. 2–4 and 8].

5.1 Weak Form

Nektar++ discretises the weak (Galerkin) form of (4.22) [60, Ch. 2]. The momentum equation is multiplied by a test function $\mathbf{w}\colon \Omega \to \mathbb{R}^d$ that vanishes on the Dirichlet part of the boundary, the continuity equation by a test function $q\colon \Omega \to \mathbb{R}$, and both are integrated over Ω. Integrating the viscous and pressure terms by parts gives

$$\int_\Omega \mathbf{w} \cdot \left(\partial_t \mathbf{u} + (\mathbf{u}\cdot\nabla)\mathbf{u}\right) \mathrm{d}\mathbf{x} + \nu \int_\Omega \nabla\mathbf{w} : \nabla\mathbf{u}\, \mathrm{d}\mathbf{x} - \int_\Omega (\nabla\cdot\mathbf{w})\, p\, \mathrm{d}\mathbf{x} = 0, \tag{5.1}$$

$$\int_\Omega q\, (\nabla\cdot\mathbf{u})\, \mathrm{d}\mathbf{x} = 0, \tag{5.2}$$

where $\nabla\mathbf{w} : \nabla\mathbf{u} = \partial_j w_i\, \partial_j u_i \in \mathbb{R}$ is the double contraction of the two $d \times d$ gradient matrices. The integration by parts produces the boundary term $\int_{\partial\Omega} \mathbf{w} \cdot (\nu\, \partial_n \mathbf{u} - p\, \mathbf{n})\, \mathrm{d}S$. It vanishes on the Dirichlet boundaries, i.e. on the inflow, the cylinder and the lateral boundaries $y = \pm 8$, because $\mathbf{w} = \mathbf{0}$ there, and on the outflow because $p = 0$ and $\partial_n \mathbf{u} = \mathbf{0}$ are imposed there. Only first derivatives of $\mathbf{u}$ remain, so a continuous, piecewise-polynomial (C^0) basis can be used [60, Ch. 2].

5.2 Spectral/hp Element Discretisation

The domain is divided into N_{el} elements Ω^e, each the image of a reference element $\hat{\Omega}$ (the square $[-1,1]^2$ or a reference triangle) under a polynomial map $\mathbf{x}^e\colon \hat{\Omega} \to \Omega^e$ [60, Ch. 4]. On each element every solution component is expanded as

$$u_h^e(\mathbf{x}^e(\boldsymbol{\xi})) = \sum_{i=0}^{N_e-1} \hat{u}_i^e\, \varphi_i(\boldsymbol{\xi}), \qquad \boldsymbol{\xi} \in \hat{\Omega} \subset \mathbb{R}^2, \tag{5.3}$$

with coefficients $\hat{\mathbf{u}}^e = (\hat{u}_0^e, \ldots, \hat{u}_{N_e-1}^e)^\top \in \mathbb{R}^{N_e}$, where $N_e = (P+1)^2$ on a quadrilateral and $N_e = (P+1)(P+2)/2$ on a triangle for polynomial order P. The two-dimensional basis

functions φ_i are built from the one-dimensional modified basis $\varphi_i^{\mathrm{1D}}\colon [-1,1] \to \mathbb{R}$ of order P [60, Ch. 2],

$$\varphi_i^{\mathrm{1D}}(\xi) = \begin{cases} \dfrac{1-\xi}{2}, & i = 0, \\ \dfrac{1-\xi}{2}\,\dfrac{1+\xi}{2}\,P_{i-1}^{(1,1)}(\xi), & 1 \le i \le P-1, \\ \dfrac{1+\xi}{2}, & i = P, \end{cases} \tag{5.4}$$

where $P_{i-1}^{(1,1)}$ is a Jacobi polynomial of degree $i-1$. The two linear vertex modes connect neighbouring elements, while the interior ('bubble') modes vanish at $\xi = \pm 1$. On quadrilaterals the two-dimensional basis is the tensor product $\varphi_i^{\mathrm{1D}}(\xi_1)\,\varphi_j^{\mathrm{1D}}(\xi_2)$, $0 \le i, j \le P$; on triangles a collapsed coordinate system is used so that the same one-dimensional functions can be reused [78], [60, Ch. 3]. All integrals are evaluated with Gauss–Jacobi quadrature [60, App. B].

5.2.1 Convergence

The accuracy can be increased by refining the mesh (h-refinement) or by raising P (p-refinement). For a solution that is analytic in each element, p-refinement on a fixed mesh gives exponential convergence,

$$\|\mathbf{u} - \mathbf{u}_h\|_{H^1(\Omega)} \le C\, e^{-\kappa P}, \tag{5.5}$$

with constants $C, \kappa > 0$ independent of P, whereas h-refinement at fixed P only gives the algebraic rate $\mathcal{O}(h_e^P)$, where h_e is a typical element size [60, Ch. 2], [79]. Smoothness alone is not sufficient for the exponential rate; for solutions with finite regularity, raising P gives an algebraic rate that grows with the regularity of the solution [79]. For this reason the cylinder wakes of this thesis are computed with a relatively high polynomial order, $P = 10$, on meshes of a few thousand elements; the polynomial order was not varied (Section 7.4).

5.3 Velocity Correction Scheme

Solving (5.1)–(5.2) for $(\mathbf{u}, p)$ together would require a large saddle-point system in every time step, and the velocity and pressure spaces would have to satisfy the inf–sup condition, which equal-order expansions do not [80, Ch. 12]. Nektar++ instead uses the velocity correction scheme of Karniadakis, Israeli and Orszag [81], which splits each time step into $d+1$ scalar elliptic problems that can all use the basis (5.3).

Let $\Delta t > 0$ be the time step and $\mathbf{u}^n \approx \mathbf{u}(\cdot, n\Delta t)\colon \Omega \to \mathbb{R}^d$. The time derivative is approximated by a backward-differentiation formula of order K, and the nonlinear term $\mathbf{N}(\mathbf{u}) := (\mathbf{u}\cdot\nabla)\mathbf{u} \in \mathbb{R}^d$ is extrapolated explicitly [81]:

$$\left.\frac{\partial \mathbf{u}}{\partial t}\right|^{n+1} \approx \frac{\gamma_0 \mathbf{u}^{n+1} - \hat{\mathbf{u}}}{\Delta t}, \qquad \hat{\mathbf{u}} = \sum_{r=0}^{K-1} a_r \mathbf{u}^{n-r}, \qquad \mathbf{N}^{*,n+1} = \sum_{r=0}^{K-1} \chi_r\, \mathbf{N}(\mathbf{u}^{n-r}). \tag{5.6}$$

The coefficients in Table 5.1 follow from requiring that the time derivative in (5.6) is exact for polynomials of degree K and the extrapolation for polynomials of degree $K-1$ in time [81]. This thesis uses $K = 2$.

Each time step then consists of two elliptic solves. First, taking the divergence of the semi-discrete momentum equation and requiring $\nabla \cdot \mathbf{u}^{n+1} = 0$ gives a Poisson equation for the pressure $p^{n+1}\colon \Omega \to \mathbb{R}$,

$$\nabla^2 p^{n+1} = \nabla \cdot \left(\frac{\hat{\mathbf{u}}}{\Delta t} - \mathbf{N}^{*,n+1} \right) \quad \text{in } \Omega, \tag{5.7}$$

Table 5.1: Coefficients of the stiffly stable scheme (5.6) for orders $K = 1, 2, 3$ [81].

K	γ_0	a_0	a_1	a_2	χ_0	χ_1	χ_2
1	1	1	–	–	1	–	–
2	3/2	2	−1/2	–	2	−1	–
3	11/6	3	−3/2	1/3	3	−3	1

with the high-order Neumann condition [81]

$$\frac{\partial p^{n+1}}{\partial n} = -\Big[\frac{\partial \mathbf{u}}{\partial t}\Big|^{n+1} + \mathbf{N}^{*,n+1} + \nu(\nabla \times \boldsymbol{\omega})^{*,n+1}\Big] \cdot \mathbf{n} \quad \text{on } \partial\Omega, \qquad \boldsymbol{\omega} = \nabla \times \mathbf{u}, \tag{5.8}$$

where the superscript $*$ denotes extrapolation with the weights χ_r. The terms in the bracket are the normal components of the acceleration, of the convective term and of the viscous term, so (5.8) is the momentum equation projected onto the boundary normal. The viscous term is written in rotational form: the identity $\nabla^2\mathbf{u} = \nabla(\nabla \cdot \mathbf{u}) - \nabla \times \nabla \times \mathbf{u}$ shows that $\nabla^2\mathbf{u} = -\nabla \times \boldsymbol{\omega}$ for a divergence-free field. The rotational form depends only on the vorticity, so the small divergence of the extrapolated velocity does not enter the boundary condition; this is what preserves the time accuracy of the scheme, in contrast to the simple condition $\partial p/\partial n = 0$ [81]. In two dimensions $\boldsymbol{\omega} = (0, 0, \omega)$ and $\nabla \times \boldsymbol{\omega} = (\partial_y\omega, -\partial_x\omega)$. In Nektar++ the high-order condition must be requested explicitly for each boundary [82]; the simulations of this thesis used the simple condition (Section 6.3). Second, the velocity is obtained from one Helmholtz problem per component,

$$\Big(\nabla^2 - \frac{\gamma_0}{\nu\Delta t}\Big)\mathbf{u}^{n+1} = -\frac{1}{\nu}\Big(\frac{\hat{\mathbf{u}}}{\Delta t} - \mathbf{N}^{*,n+1} - \nabla p^{n+1}\Big). \tag{5.9}$$

Because $\gamma_0/(\nu\Delta t) > 0$, each of these problems is coercive on $H^1(\Omega)$ and therefore uniquely solvable by the Lax–Milgram theorem [80, Thm. 2.7.7]. The pressure problem is of Neumann type on the inflow, on the cylinder and on the lateral boundaries; prescribing the Dirichlet condition $p = 0$ on the outflow boundary removes the additive constant and makes its solution unique.

Chapter 6

Methodology and Implementation

This chapter describes the computational workflow used to generate the training data and to train the physics-informed neural network (PINN) introduced in Chapter 4. The workflow has five stages: (i) definition of the cylinder-wake problem following Raissi et al. [10]; (ii) mesh generation in Gmsh [83]; (iii) high-fidelity flow simulation with the spectral/hp element solver Nektar++ [40, 59]; (iv) extraction of the simulation output into PINN training data; and (v) construction and training of the PINN in TensorFlow 2 [84]. The chapter closes with an overview of the numerical experiments.

6.1 The Cylinder-Wake Benchmark

This thesis studies the two-dimensional incompressible flow past a circular cylinder, introduced as a PINN benchmark by Raissi, Perdikaris and Karniadakis [10]. Figure 6.1 shows a schematic of the set-up, drawn after the description in [10].

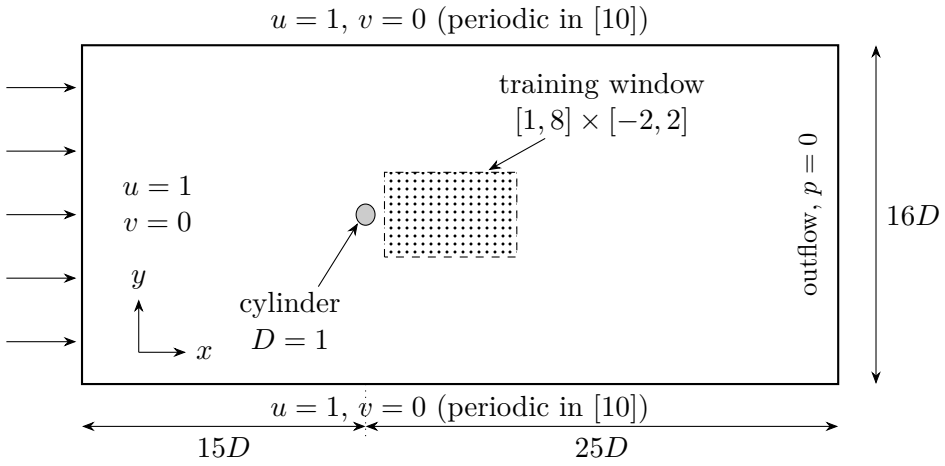


Figure 6.1: Schematic of the cylinder-wake benchmark of Raissi et al. [10]: computational domain $[-15, 25] \times [-8, 8]$ with a cylinder of diameter $D = 1$ at the origin, uniform inflow on the left and zero-pressure outflow on the right. At the top and bottom, Raissi et al. used periodic conditions, whereas the free-stream velocity is prescribed there in this thesis. The dotted rectangle is the region from which Raissi et al. took their training data. [51]

All quantities are non-dimensional, with the cylinder diameter $D = 1$ and the free-stream speed $U_\infty = 1$ as reference scales, so that $\nu = 1/Re$ [10]. The domain is $[-15, 25] \times [-8, 8]$ with the cylinder centred at the origin. The boundary conditions are shown in Figure 6.1: at $x = -15$ the velocity is set to $(1, 0)$; at $x = 25$ the pressure is fixed to zero; on the cylinder wall the fluid is at rest, $u = v = 0$; and the boundaries $y = \pm 8$ are connected periodically in the original study [10]. In the original study the flow was computed with Nektar++ on a mesh of 412 triangular elements with high-order polynomial expansions and, once the

periodic shedding regime had been reached, snapshots of the flow were stored at regular time intervals [10].

The figure also shows the most important choice made by Raissi et al.: the PINN is not trained on the whole domain but only on data from the rectangle $[1, 8] \times [-2, 2]$ directly behind the cylinder. From all points and snapshots inside this rectangle, $N_d = 5000$ space–time points are drawn at random, and only the velocity components u and v at these points are given to the network; the pressure is never used for training [10]. The same points serve as collocation points for the residuals (4.47)–(4.48). The unknown parameters λ_1 and λ_2 therefore have to be identified from a small velocity data set in a limited part of the wake. This rectangle is the starting point of the region study in this thesis, where the training window is moved to other parts of the wake (Section 6.6).

The set-up of this thesis follows Figure 6.1 with one difference: instead of periodic conditions, the free-stream velocity $(u, v) = (1, 0)$ is prescribed on the lateral boundaries $y = \pm 8$. The lateral boundaries confine the flow, and their position influences the computed wake [85]; with a domain height of 16 diameters this influence is small, but the reference data of this thesis are therefore not identical to those of Raissi et al. The domain and the non-dimensionalisation are the same, and the flow fields are recomputed with the author's own meshes and Nektar++ configurations, as described below.

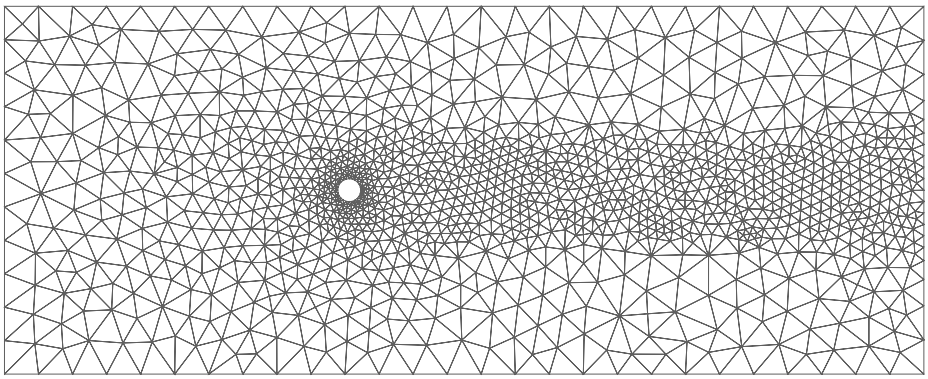
6.2 Mesh Generation

The computational domain around the cylinder was discretised with Gmsh [83], with local refinement in the boundary layer and in the near wake to resolve the velocity gradients that drive vortex shedding. Distance-based size fields were used to control the element size near the wall and in the wake, and the domain was triangulated with a Delaunay-type algorithm of Gmsh (Frontal–Delaunay for the $Re = 500$ mesh, Appendix A.1) [83, 86]. For $Re = 500$, a thin layer of quadrilateral elements was placed directly around the cylinder surface with a boundary-layer field of Gmsh [86]: the first element at the wall has a height of 0.005, the height grows by a factor of 1.15 from layer to layer, and the layer region has a total thickness of 0.15 (Figure 6.2c). These quadrilaterals resolve the steep velocity gradients normal to the wall at a lower cost than a uniformly refined triangular mesh. The target element sizes of the $Re = 500$ mesh are 0.05 at the cylinder, 0.35 in the wake band and 1.5 in the far field. The mesh was then converted into the Nektar++ input format with the utility NekMesh [40]. For $Re = 500$ the mesh had to be refined further, because the boundary layer on the cylinder becomes thinner as the Reynolds number increases; its thickness scales approximately like $Re^{-1/2}$ [61, Ch. 5].

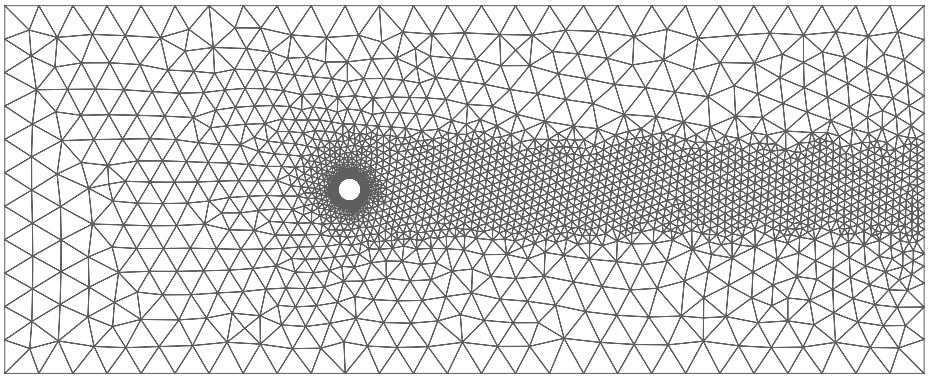
Figure 6.2 shows the resulting meshes, and Table 6.1 lists their main properties. The elements are smallest on the cylinder surface, where the boundary layer must be resolved, and grow gradually towards the outer boundaries, where the flow is nearly uniform. A band of finer elements extends from the cylinder into the wake, so that the shed vortices are resolved over the whole length of the training regions.

Table 6.1: Properties of the meshes used for the two Reynolds numbers.

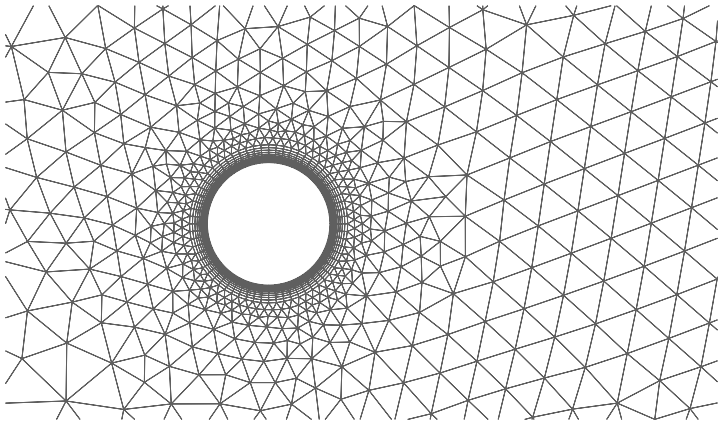
Property	$Re = 100$	$Re = 500$
Number of elements N_{el}	2665	4865
Element type	triangles	triangles and quadrilaterals
Polynomial order P	10	10
Height of the first quadrilateral layer	–	0.005
Growth ratio of the quadrilateral layers	–	1.15
Total thickness of the quadrilateral layers	–	0.15



(a) Whole computational domain, $Re = 100$.



(b) Whole computational domain, $Re = 500$ (refined mesh).



(c) Close-up of the boundary-layer refinement around the cylinder, $Re = 500$.

Figure 6.2: Spectral/hp element meshes generated with Gmsh for the cylinder wake.

6.3 Flow Simulation

The flow was computed with the incompressible Navier–Stokes solver of Nektar++, which implements the spectral/hp discretisation and the velocity correction scheme of Chapter 5 [40, 59]. The numerical settings are summarised in Table 6.2.

Table 6.2: Numerical settings of the Nektar++ simulations. The meaning of each setting is described in the Nektar++ user guide [82, Ch. 'Incompressible Navier–Stokes Solver'].

Setting	$Re = 100$	$Re = 500$	Meaning
Spatial discretisation	continuous Galerkin		Section 5.1
Basis	modified modal basis, $P = 10$		(5.4)
Splitting scheme	velocity correction		Section 5.3
Time integration	IMEX, order $K = 2$		Table 5.1
Viscosity ν	0.01	0.002	$\nu = 1/Re$
Time step Δt	0.005	0.0005	
Number of time steps	100 000	200 000	
Final time t_f	500	100	
Snapshot interval	0.1 (20 steps)	0.2 (400 steps)	time units
Lateral boundaries $y = \pm 8$	free stream, $(u, v) = (1, 0)$		Section 6.3
Pressure on velocity boundaries	$\partial p/\partial n = 0$		Section 6.3
Initial condition	$(u, v, p) = (1, 0, 0)$, uniform free stream		

The boundary conditions follow Figure 6.1: the velocity $(u, v) = (1, 0)$ is prescribed at the inflow, $p = 0$ and zero normal derivatives of u and v are prescribed at the outflow, the free-stream velocity $(u, v) = (1, 0)$ is prescribed on the lateral boundaries $y = \pm 8$, and $u = v = 0$ holds on the cylinder. On all boundaries where the velocity is prescribed, the homogeneous Neumann condition $\partial p/\partial n = 0$ was set for the pressure. Nektar++ applies the high-order condition (5.8) only on boundaries that are marked for it in the session file [82], which was not done in the present simulations. The simple condition reduces the time accuracy of the pressure near these boundaries [81]; the effect is largest at the cylinder wall, which lies outside all training windows, but it is a limitation of the reference data (Section 7.4).

Two points needed particular care. First, the nonlinear term $(\mathbf{u} \cdot \nabla)\mathbf{u}$ is the product of two polynomials of order P, i.e. a polynomial of higher order, and if it is integrated with too few quadrature points the resulting aliasing error can make a spectral/hp simulation unstable [60, Ch. 8]. At $Re = 500$ the first simulations on a coarser mesh with a larger time step became unstable. Because the convective term is treated explicitly (Section 5.3), the time step is limited by a condition of CFL type, which becomes stricter as the element size decreases and the polynomial order increases [60]. The final simulation at $Re = 500$ therefore uses the refined mesh of Section 6.2 and a ten times smaller time step, $\Delta t = 0.0005$ (Table 6.2). Its session file contains the solver property `Dealiasing` (Appendix A.2); in Nektar++ this property controls dealiasing in the Fourier direction of quasi-three-dimensional runs, whereas spectral/hp dealiasing of a two-dimensional run is switched on with `SPECTRALHPDEALIASING` [82]. The stable simulation at $Re = 500$ is therefore attributed to the refined mesh and the small time step.

Second, the simulation starts from the uniform free stream $(u, v, p) = (1, 0, 0)$. The flow first develops a symmetric pair of vortices behind the cylinder. Because the unstructured mesh is not exactly symmetric about $y = 0$ and round-off errors are always present, small asymmetric disturbances exist from the start; above $Re \approx 47$ the wake is unstable to such disturbances, so they grow and the flow develops into periodic vortex shedding [22]. The simulation at

$Re = 100$ was run up to $t_\mathrm{f} = 500$ and the one at $Re = 500$ up to $t_\mathrm{f} = 100$ (Table 6.2), but only a part of each simulation is used for the PINN: at $Re = 100$, after the periodic state had been reached, a time window of 20 time units (200 snapshots, near wake) or 40 time units (400 snapshots, all other windows) was extracted. At $Re = 500$, in contrast, all 501 snapshots from $t = 0$ to $t = 100$ (interval 0.2) were used. This interval includes the start-up of the flow from the uniform free stream: vortex shedding begins at about $t \approx 20$ and becomes periodic only at about $t \approx 40$ (Section 6.3.1). The training data at $Re = 500$ are therefore not restricted to the periodic state, unlike at $Re = 100$; the consequences are discussed in Section 7.3. The field errors at $Re = 500$ are evaluated at the snapshot $t = 20$ (Section 6.5.3), i.e. at the beginning of vortex shedding.

The stored snapshots were converted to the VTU format and inspected in ParaView [87] to confirm that vortex shedding had developed.

6.3.1 Validation of the Reference Simulation

Both simulations were checked with the aerodynamic forces on the cylinder, which Nektar++ computes during the run with the `AeroForces` filter (Appendix A.2) [82]. The drag and lift coefficients are

$$C_D = \frac{2F_x}{\rho U_\infty^2 D}, \qquad C_L = \frac{2F_y}{\rho U_\infty^2 D}, \tag{6.1}$$

where F_x and F_y are the streamwise and transverse force per unit span on the cylinder, ρ is the density, U_∞ the free-stream velocity and D the diameter; with $\rho = U_\infty = D = 1$ they reduce to $C_D = 2F_x$ and $C_L = 2F_y$ [22]. The lift oscillates with the shedding frequency f_s, which defines the Strouhal number

$$St = \frac{f_s D}{U_\infty}, \tag{6.2}$$

the dimensionless frequency of vortex shedding [22]. Here f_s was obtained from the dominant peak of the Fourier spectrum of $C_L(t)$ and, as a check, from the mean time between successive upward zero crossings of $C_L - \overline{C_L}$; both methods gave the same value to within 0.1%.

At $Re = 100$, vortex shedding develops within the first few tens of time units, and from about $t \approx 50$ the lift oscillates with constant amplitude. In the interval $t \in [125, 250]$, which contains 21 shedding periods, the amplitude of C_L is the same in the first and the last third of the interval to within 0.01%, so the flow is periodic. The resulting values are compared with the literature in Table 6.3. The Strouhal number $St = 0.171$ is 4.3% larger than the value $St = 0.164$ given by the empirical Strouhal–Reynolds relation of Williamson [88], and the mean drag coefficient $\overline{C_D} = 1.399$ and the lift amplitude 0.346 are 5.2% and 4.2% larger than the values 1.33 and 0.332 of the two-dimensional simulations of Park, Kwon and Choi [89]. All three quantities deviate by a similar amount and in the same direction. The most likely reason is the confinement of the flow by the lateral boundaries $y = \pm 8$, on which the free-stream velocity is prescribed; the position of these boundaries is known to influence the computed shedding frequency and forces [85]. The reference simulation at $Re = 100$ therefore reproduces the known behaviour of the cylinder wake to within a few percent. A further, indirect confirmation is that the PINN recovers $\lambda_2 = 1/Re$ to within a few percent from several independent windows of these data (Section 7.2), which is only possible if the data satisfy the Navier–Stokes equations with $\nu = 0.01$ in the chosen non-dimensionalisation.

At $Re = 500$ the lift is zero at first, because the flow starts symmetric. Vortex shedding begins at about $t \approx 20$, and the amplitude of C_L grows until about $t \approx 40$. In the interval $t \in [50, 100]$, which contains 11 shedding periods, the amplitude of C_L differs by only 0.08% between the first and the last third of the interval, so the flow is periodic there. A comparison

with experiments is not meaningful at this Reynolds number, because the real wake is already three-dimensional [22, 90], whereas the simulation is two-dimensional.

Table 6.3: Shedding frequency and force coefficients of the reference simulations, evaluated over $t \in [125, 250]$ at $Re = 100$ and $t \in [50, 100]$ at $Re = 500$, and reference values from the literature for $Re = 100$.

Quantity	$Re = 100$	Literature, $Re = 100$	$Re = 500$
Strouhal number St	0.171	0.164 [88]	0.229
Mean drag coefficient $\overline{C_D}$	1.399	1.33 [89]	1.466
Amplitude of C_D	0.010	–	0.142
Amplitude of C_L	0.346	0.332 [89]	1.197
Change of the C_L amplitude	$< 0.01\%$	–	0.08%

6.4 Data Extraction

The VTU files were read with a Python script based on the VTK library, and the velocity and pressure were written to MATLAB `.mat` files with the same structure as the data set of Raissi et al. [10]. Let N_x be the number of spatial points in the chosen region and N_t the number of stored snapshots. Each file contains

- the coordinates x_k and y_k of the spatial points, $k = 1, \ldots, N_x$ (two vectors in $\mathbb{R}^{N_x}$);
- the snapshot times t_n, $n = 1, \ldots, N_t$ (a vector in $\mathbb{R}^{N_t}$);
- the velocity components $u(x_k, y_k, t_n)$ and $v(x_k, y_k, t_n)$ (two matrices in $\mathbb{R}^{N_x \times N_t}$);
- the pressure $p(x_k, y_k, t_n)$ (a matrix in $\mathbb{R}^{N_x \times N_t}$), which is used only for evaluation and never for training.

The velocity and pressure are available at the quadrature points of every element (Appendix B). These points are not distributed uniformly: they are dense where the mesh is fine, that is, in the refined wake band, and sparse where it is coarse. For training, the $N_x N_t$ space–time points inside the chosen training region are arranged in one list, and $N_d = 5000$ of them are drawn at random without replacement, so regions with a finer mesh contribute proportionally more training points. Figure 6.3 shows the training regions used in this thesis on a snapshot of the vorticity field.

Only the points of one region are given to the PINN in each experiment. Before training, all extracted data were checked against the reference scales $D = 1$ and $U_\infty = 1$. This check is necessary because the PINN identifies the viscosity in the units of the data: if, for example, the mesh used a different cylinder diameter, the identified λ_2 would be the kinematic viscosity in those units and not $1/Re$, although the network would fit the data equally well.

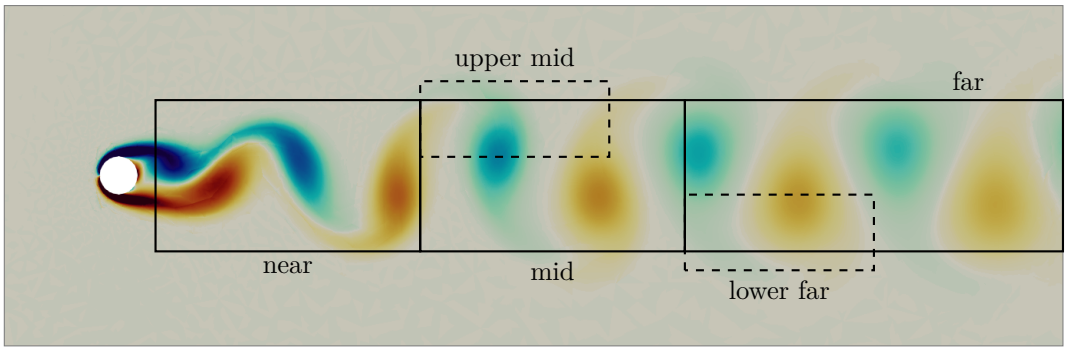


Figure 6.3: Vorticity $\omega = \partial_x v - \partial_y u$ of the cylinder wake at $Re = 100$ (section $x \in [-3, 25]$, $y \in [-4.5, 4.5]$) with the training regions. Solid: near wake $[1, 8] \times [-2, 2]$, mid wake $[8, 15] \times [-2, 2]$ and far wake $[15, 25] \times [-2, 2]$. Dashed: upper mid-wake window $[8, 13] \times [0.5, 2.5]$ and lower far-wake window $[15, 20] \times [-2.5, -0.5]$. Positive and negative values of ω mark the counter-rotating shed vortices.

6.5 PINN Implementation in TensorFlow 2

6.5.1 Porting from TensorFlow 1 to TensorFlow 2

The original code of Raissi et al. [10] was written for TensorFlow 1, which builds a static computational graph with placeholders and executes it in a session. It also uses `tf.contrib.opt.ScipyOptimizerInterface` for the L-BFGS-B stage. These interfaces no longer exist in TensorFlow 2, which executes operations eagerly by default [84]. The code was therefore rewritten for TensorFlow 2 while keeping the network, the loss and the training procedure of the original. The main changes were:

- **Derivatives.** The derivatives of ψ_θ and p_θ with respect to (x, y, t) are computed with nested `tf.GradientTape` objects. Because the residuals (4.47)–(4.48) need second derivatives of the velocity, i.e. third derivatives of ψ_θ, the tapes must be nested in the correct order and must be persistent where several derivatives are taken from the same tape.

- **Compiled training step.** The loss and gradient computation is wrapped in a `tf.function`, which compiles it into a graph and restores most of the speed of the original TensorFlow 1 code.

- **L-BFGS-B.** The second training stage uses `scipy.optimize.minimize` with the method `L-BFGS-B` [72, 76] (Section 4.5.6). All trainable variables are flattened into the vector $\Theta \in \mathbb{R}^{n_\Theta}$, and a wrapper function returns the loss and its gradient to SciPy. At $Re = 100$ the network is evaluated in single precision (`float32`), and the vector passed to SciPy is converted to double precision (`float64`), as SciPy requires. Because the loss is then evaluated in single precision, relative changes smaller than about 10^{-7} cannot be resolved; with the small value of f_{tol} in Table 6.4, L-BFGS-B therefore stops in practice when the loss no longer changes in single precision, when the line search fails, or at the iteration limit. The runs at $Re = 500$ use double precision (`float64`) throughout; there both L-BFGS-B stages stopped at the iteration limit (Section 7.3).

- **Loss scaling.** As in the original code, the squared errors are summed instead of averaged, i.e. the implemented loss is $N_d \mathcal{L}$ with $\mathcal{L}$ from (4.49) (recall $N_c = N_d$). Both versions have the same minimiser, and Adam is unaffected by a constant rescaling of the gradient (apart from ϵ) [70]; only the stopping tests of L-BFGS-B depend on the scaling.

6.5.2 Network and Training Settings

The network architecture follows Raissi et al. [10]: the inputs (x, y, t) are passed through 8 hidden layers with 20 neurons each and tanh activation to the two outputs (ψ, p). The weights are initialised with the Xavier (Glorot) normal scheme [52] and the biases with zeros. The inputs are scaled linearly to the interval $[-1, 1]$ using the bounds of the training data. The two unknown parameters are created as separate trainable variables. In the reference runs they are initialised with $\lambda_1 = \lambda_2 = 0$, as in the original code, so that the network receives no prior information about their true values. No additional constraint or penalty pulls the parameters towards their true values; they are identified only through the data and the physics residual.

Training starts with Adam [70] and continues with L-BFGS-B until one of its stopping criteria is met. From the space–time points of the training region, $N_d = 5000$ are sampled at random for training. All settings are summarised in Table 6.4.

Table 6.4: Network and training settings. They are used in all experiments unless stated otherwise in the table or in Chapter 7.

Setting	Value
Inputs / outputs	(x, y, t) / (ψ, p)
Hidden layers × neurons	8×20
Activation	tanh
Weight initialisation	Xavier (Glorot) normal [52]
Initial values (λ_1, λ_2)	$(0, 0)$
Number of parameters n_θ / n_Θ	3062 / 3064
Training points $N_d = N_c$	5000
Number of snapshots N_t	400 (near wake at $Re = 100$: 200; $Re = 500$: 501)
Adam: learning rate η	10^{-3}
Adam: iterations	30 000 (near-wake run of Section 7.1: 10 000; noisy runs at $Re = 100$: 200 000, Adam only; $Re = 500$: 300 000, noisy run at $Re = 500$: 200 000)
Adam: β_1, β_2, ϵ	0.9, 0.999, 10^{-7} (TensorFlow default)
L-BFGS-B: memory m	50
L-BFGS-B: max. iterations / function evaluations	15 000 / 50 000 (near-wake run of Section 7.1: 10 000 / 15 000; $Re = 500$: 100 000 / 300 000; noisy run at $Re = 500$: 50 000 / 100 000)
L-BFGS-B: max. line-search steps	50
L-BFGS-B: f_{tol} in (4.65)	machine precision ($\approx 2.2 \times 10^{-16}$)
L-BFGS-B: g_{tol}	10^{-5} (SciPy default)
Floating-point precision	$Re = 100$: `float32` (network), `float64` (SciPy); $Re = 500$: `float64`
Random seed (NumPy, TensorFlow)	1234
Framework	TensorFlow 2.16.1 [84], SciPy 1.18.1 [76]
Hardware	NVIDIA GeForce RTX 3050
Training time per run	$Re = 100$: 3–4 h; $Re = 500$: about 3 days

6.5.3 Error Measures

The identified parameters are compared with their true values $\lambda_1 = 1$ and $\lambda_2 = 1/Re$ by the relative error

$$e_{\lambda_j} = \frac{|\hat{\lambda}_j - \lambda_j|}{|\lambda_j|} \times 100\,\%, \qquad j = 1, 2, \tag{6.3}$$

where $\hat{\lambda}_j$ is the identified value. The predicted velocity and pressure fields are compared with the Nektar++ solution by the relative L^2 error

$$e_q = \frac{\|\mathbf{q}_\theta - \mathbf{q}\|_2}{\|\mathbf{q}\|_2} \times 100\,\%, \qquad q \in \{u, v, p\}, \tag{6.4}$$

where $\mathbf{q} \in \mathbb{R}^{N_x}$ contains the reference values of q at the N_x points of the training region at one snapshot, $\mathbf{q}_\theta \in \mathbb{R}^{N_x}$ the PINN prediction at the same points, and $\|\cdot\|_2$ is the Euclidean norm [10]. The errors reported in Chapter 7 are evaluated at the snapshot $t = 10$ for $Re = 100$ and $t = 20$ for $Re = 500$, which are also the snapshots shown in the figures. Because only the pressure gradient appears in the momentum equations, the PINN can determine the pressure only up to an additive constant. Before (6.4) is evaluated for p, the mean of the predicted pressure is therefore shifted to the mean of the reference pressure. The resulting e_p still depends on the mean level of the reference pressure, which is set by the outflow condition $p = 0$: a large mean pressure makes e_p look small. The pressure errors are therefore less informative than e_u and e_v and are compared mainly between windows of similar position.

6.5.4 Noise Model

To mimic measurement errors, the training velocities are perturbed in the same way as in Raissi et al. [10]. For each of the N_d training points,

$$\tilde{u}_i = u_i + \varepsilon_n\, s_u\, Z_i^u, \qquad \tilde{v}_i = v_i + \varepsilon_n\, s_v\, Z_i^v, \qquad i = 1, \ldots, N_d, \tag{6.5}$$

where $Z_i^u, Z_i^v \sim \mathcal{N}(0, 1)$ are independent standard normal random numbers, s_u and s_v are the standard deviations of the clean training values $\{u_i\}$ and $\{v_i\}$, and ε_n is the noise level. In this thesis $\varepsilon_n = 0.01$, i.e. the noise has a standard deviation of 1% of the natural variation of each velocity component. The noisy values $\tilde{u}_i, \tilde{v}_i$ replace u_i, v_i in the data term of (4.49); the training points and the network are the same as in the clean case.

At $Re = 100$ the noisy runs are trained with Adam only, for 200 000 iterations, without the L-BFGS-B stage. This is a deliberate deviation from Raissi et al., who used both stages also for noisy data [10]. L-BFGS-B drives the loss towards a very small value and could then also start to fit the noise, whereas Adam with a fixed number of iterations stops earlier and acts as a mild regularisation. At $Re = 500$, where the loss remains large (Section 7.3), the noisy run uses both stages as well: 200 000 Adam iterations followed by L-BFGS-B with the default limits of SciPy, 50 000 iterations and 100 000 function evaluations [76].

6.6 Overview of the Experiments

Four groups of experiments were carried out; they are summarised in Table 6.5.

1. **Reproduction at $Re = 100$.** The TensorFlow 2 implementation is trained on the author's own Nektar++ data in the original region $[1, 8] \times [-2, 2]$, and the result is compared with the values reported by Raissi et al. This checks the ported code and the simulation pipeline together.

2. **Different regions of the wake.** The training window is moved to different parts of the wake at $Re = 100$: the near wake directly behind the cylinder (the original window), a mid-wake window $[8, 15] \times [-2, 2]$ and a far-wake window $[15, 25] \times [-2, 2]$, which reaches the outflow boundary. In addition, two smaller windows that lie off the wake centreline are used: an upper mid-wake window $[8, 13] \times [0.5, 2.5]$ and a lower far-wake window $[15, 20] \times [-2.5, -0.5]$. They represent a measurement window that covers only one side of the vortex street. In each case, the PINN sees only data from that window.

3. **Higher Reynolds number.** The simulation and the PINN training are repeated at $Re = 500$, with a refined mesh and the true value $\lambda_2 = 1/500 = 0.002$.

4. **Noisy data.** The runs in the near-wake, mid-wake, far-wake and off-centre windows at $Re = 100$ and the run at $Re = 500$ are repeated with 1% noise in the training velocities, using the noise model of Section 6.5.4.

Table 6.5: Overview of the numerical experiments.

Experiment	Training region	Re	True (λ_1, λ_2)
1: Reproduction	$[1, 8] \times [-2, 2]$	100	$(1, 0.01)$
2: Regions, centred	near / mid / far wake	100	$(1, 0.01)$
2: Regions, off-centre	$[8, 13] \times [0.5, 2.5]$, $[15, 20] \times [-2.5, -0.5]$	100	$(1, 0.01)$
3: Higher Re	$[1, 8] \times [-2, 2]$	500	$(1, 0.002)$
4: Noisy data (1%)	all windows above	100, 500	as above

Chapter 7

Results and Discussion

This chapter presents the results of the experiments described in Section 6.6 and discusses what they show about PINN-based parameter identification for the cylinder wake. In all experiments the network receives only the velocity components u and v at $N_d = 5000$ randomly chosen points of the training region, which also serve as collocation points; the pressure and the viscosity are never given to it. All runs use the network, loss and two-stage training of Section 6.5 with the settings of Table 6.4 unless stated otherwise, and λ_1 and λ_2 start from zero. The true values are $\lambda_1 = 1$ in all cases and $\lambda_2 = 1/Re$, i.e. 0.01 at $Re = 100$ and 0.002 at $Re = 500$. The quality of the identification is measured with the relative parameter error (6.3), and the quality of the reconstructed flow fields with the relative L^2 error (6.4).

7.1 Reproduction of the Benchmark at $Re = 100$

The first step was to check that the TensorFlow 2 implementation behaves like the original TensorFlow 1 code of Raissi et al. [10]. Table 7.1 compares the values reported by Raissi et al. with those of the ported code on the author's Nektar++ data in the same region $[1, 8] \times [-2, 2]$, for clean data and for data with 1% noise.

Table 7.1: Identified parameters and relative L^2 errors of the reconstructed fields at $Re = 100$ in the near-wake window $[1, 8] \times [-2, 2]$ (true values $\lambda_1 = 1$, $\lambda_2 = 0.01$). The first two rows give the values reported by Raissi et al. for their TensorFlow 1 code and data set; the last two rows are the TensorFlow 2 port on the Nektar++ data. The clean run on the Nektar++ data used 10 000 Adam iterations. The field errors (6.4) are evaluated at $t = 10$; the pressure error after the constant shift of Section 6.5.3.

Run	$\hat{\lambda}_1$	e_{λ_1} [%]	$\hat{\lambda}_2$	e_{λ_2} [%]	e_u [%]	e_v [%]	e_p [%]
Raissi et al. [10], clean	0.999[a]	0.078	0.01047	4.67	–	–	–
Raissi et al. [10], 1% noise	0.998	0.17	0.01057	5.70	–	–	–
TF2 port, clean	0.99887	0.114	0.010267	2.67	4.51	1.25	2.00
TF2 port, 1% noise	0.99391	0.609	0.010641	6.41	–	–	–

[a] Raissi et al. report the identified values rounded to three significant digits but the errors computed from the unrounded values; this is why $\hat{\lambda}_1 = 0.999$ appears together with an error of 0.078%.

Figure 7.1 compares the predicted fields with the Nektar++ solution. The pressure is reconstructed although the network never saw any pressure data: it is obtained only because the momentum residuals (4.47)–(4.48) link the pressure gradient to the velocity field. As expected, the predicted pressure differs from the reference by a constant, because only ∇p enters the equations [10]. The predicted and reference fields in Figure 7.1 can hardly be distinguished: the recirculation zone directly behind the cylinder, the shear layer that rolls up into the first vortex, and the low-pressure cores of the shed vortices all appear at the correct positions and with the correct strength. The identified values are $\hat{\lambda}_1 = 0.9989$ and $\hat{\lambda}_2 = 0.01027$, i.e. errors of 0.11% and 2.7% (Table 7.1). The error in λ_1 is of the same order

as the 0.078% reported by Raissi et al., and the error in λ_2 is smaller than their 4.7% [10]. With 1% noise, the errors rise to 0.61% for λ_1 and 6.41% for λ_2, compared with 0.17% and 5.70% reported by Raissi et al. for the same noise level [10].

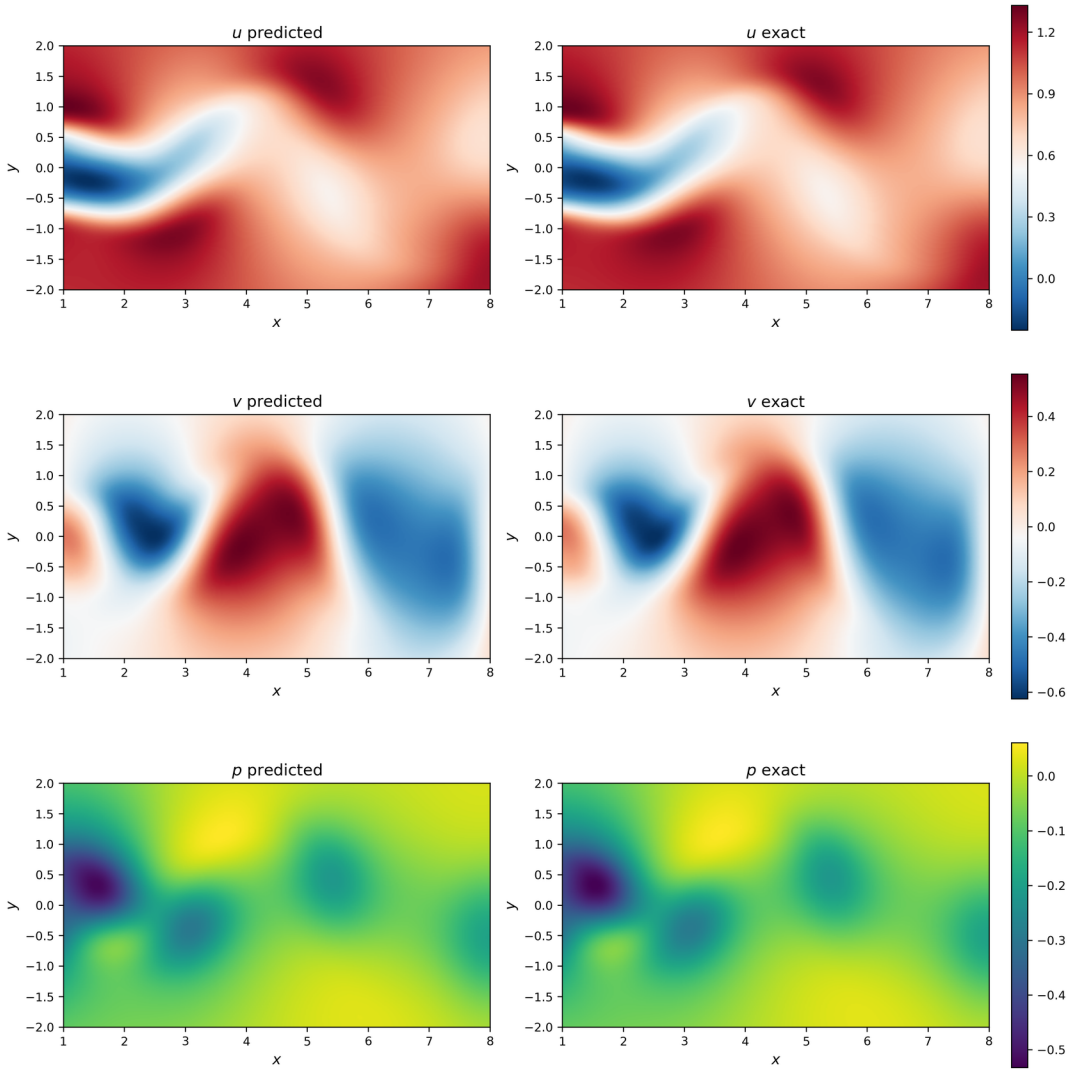


Figure 7.1: Predicted (left) and reference Nektar++ (right) velocity components u, v and pressure p at $Re = 100$ in the near-wake window $[1, 8] \times [-2, 2]$ at $t = 10$ ($\hat{\lambda}_1 = 0.9989$, $\hat{\lambda}_2 = 0.01027$). No pressure data were used in training; the predicted pressure has been shifted by a constant (Section 6.5.3).

Figure 7.2 shows the course of training. In this run the Adam stage was stopped after 10 000 iterations instead of the 30 000 iterations used for the other windows (Table 6.4); this run was not repeated with 30 000 iterations, so its parameter errors are not directly comparable with those of the other windows (Section 7.2.4). The loss first stays on a plateau for about 2500 iterations, while the network fits only the mean flow, and then drops by more than an order of magnitude as the vortex pattern is captured. At the same moment λ_1 jumps from almost zero to about 0.8, and it reaches about 0.97 at the end of the Adam stage. λ_2 shows two short spikes, up to about 0.034 in the first iterations and 0.024 at about iteration 2700, then rises to about 0.017 and decreases slowly towards the true value; at the switch to L-BFGS-B it is still about 0.012. During the first iterations λ_2 even takes small negative values. This is physically meaningless, but it is allowed, because no bound $\lambda_2 \geq 0$ is imposed (Section 4.5.6). In the L-BFGS-B stage the loss decreases by another one and a half orders of magnitude, λ_1 reaches one, and λ_2 settles at its final value within about 2000 iterations. As in all windows

studied in Section 7.2, most of the final accuracy of λ_2 is therefore reached in the L-BFGS-B stage, which uses curvature information of the loss (Section 4.5.6).

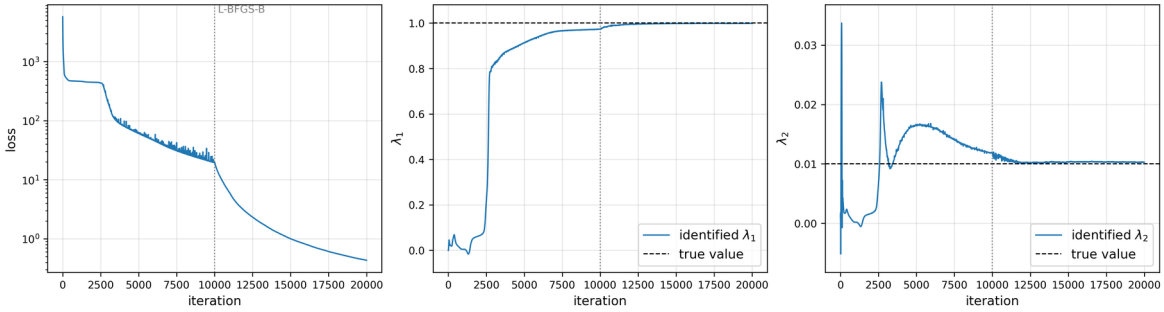


Figure 7.2: Training history at $Re = 100$ in the near-wake window: total loss (left) and evolution of the identified parameters λ_1 (middle) and λ_2 (right). The dotted vertical line marks the switch from Adam to L-BFGS-B after 10 000 iterations, and the dashed horizontal lines mark the true values.

Because the ported code was run on the author's own Nektar++ data, this comparison tests the rewrite and the simulation pipeline (mesh, session file, extraction and non-dimensionalisation) together. The close agreement with the values of Raissi et al. shows that the rewrite did not change the method and that the new data are consistent with the benchmark.

7.2 Identification from Different Regions of the Wake

In the second experiment the training window was moved downstream at $Re = 100$: besides the near-wake window of Section 7.1, a mid-wake window $[8, 15] \times [-2, 2]$ and a far-wake window $[15, 25] \times [-2, 2]$ were used, with the settings of Table 6.4 unchanged, so that only the location of the data differs. These windows are centred on the wake axis $y = 0$. For both windows the identification was also repeated with 1% noise in the training velocities, using the noise model of Section 6.5.4 and the same training points; the noisy runs were trained with Adam only. Two further, smaller windows that lie off the axis are discussed in Section 7.2.3, and the windows are compared in Section 7.2.4.

7.2.1 Mid Wake

Table 7.2 lists the identified parameters and field errors for the mid-wake window. With clean data, both parameters are identified very accurately: the convection coefficient to within 0.009% and the viscosity coefficient to within 0.10%. In this run, data from the region where the vortices are formed were therefore not needed to identify the parameters.

Table 7.2: Identified parameters and relative L^2 errors of the reconstructed fields at $Re = 100$ in the mid-wake window $[8, 15] \times [-2, 2]$ from clean data and from data with 1% noise (true values $\lambda_1 = 1$, $\lambda_2 = 0.01$). The field errors (6.4) are evaluated at $t = 10$; the pressure error after the constant shift of Section 6.5.3.

Data	$\hat{\lambda}_1$	e_{λ_1} [%]	$\hat{\lambda}_2$	e_{λ_2} [%]	e_u [%]	e_v [%]	e_p [%]
clean	0.99991	0.009	0.010010	0.10	0.32	1.45	2.31
1% noise	1.00002	0.002	0.009694	3.06	–	–	–

Figure 7.3 compares the predicted and reference fields. The alternating pattern of the von Kármán vortex street is reproduced: regions of reduced streamwise velocity u alternate with regions of positive and negative transverse velocity v as the vortices are carried downstream [22].

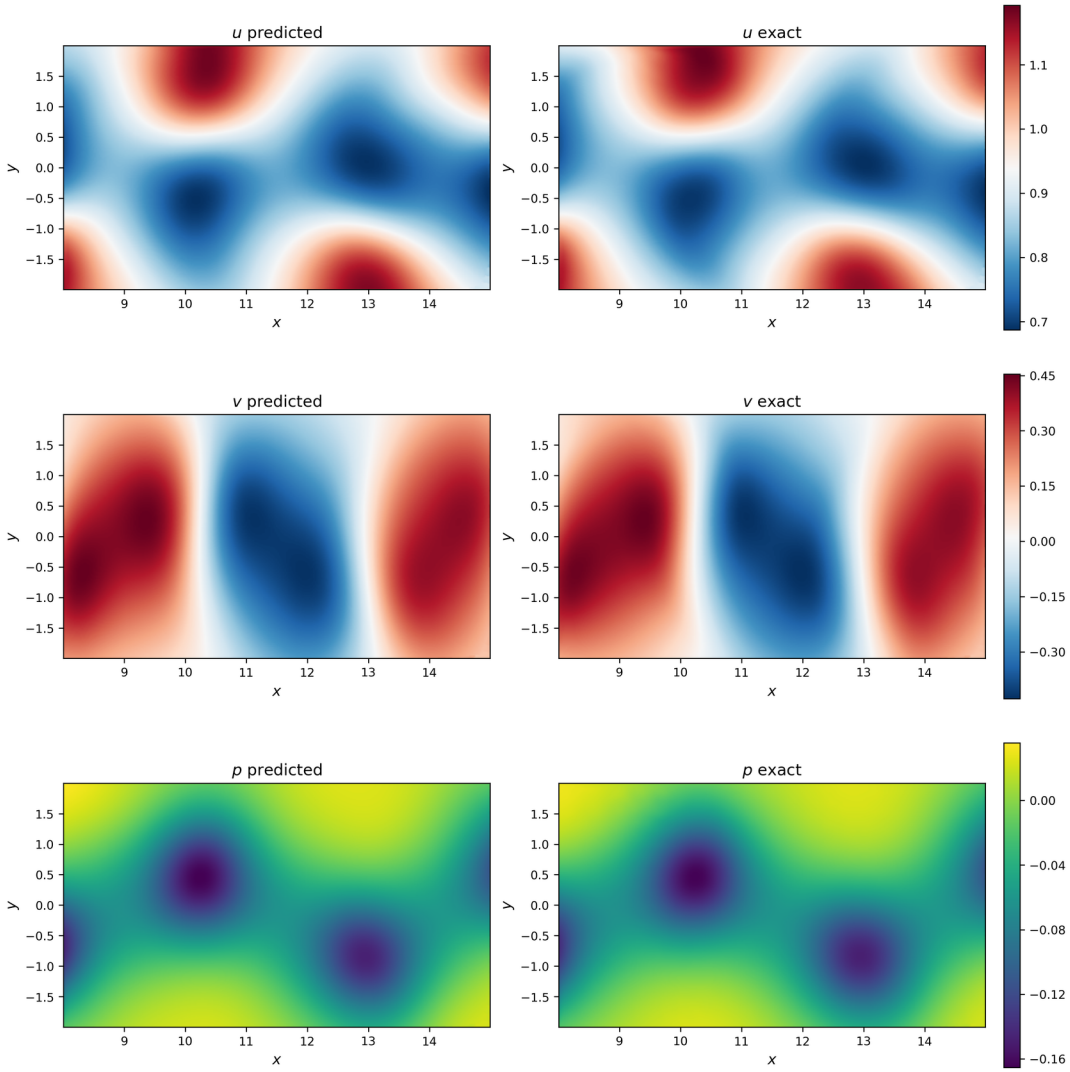


Figure 7.3: Predicted (left) and reference Nektar++ (right) velocity components u, v and pressure p at $Re = 100$ in the mid-wake window $[8, 15] \times [-2, 2]$ at $t = 10$, clean data ($\hat{\lambda}_1 = 0.99991$, $\hat{\lambda}_2 = 0.010010$). No pressure data were used in training; the predicted pressure has been shifted by a constant.

The velocity is reconstructed with relative errors (6.4) of $e_u = 0.32\%$ and $e_v = 1.45\%$. The pressure never enters the training data and is obtained only through the residuals (4.47)–(4.48) [10]; nevertheless, the low-pressure vortex cores appear at the correct positions, with $e_p = 2.31\%$.

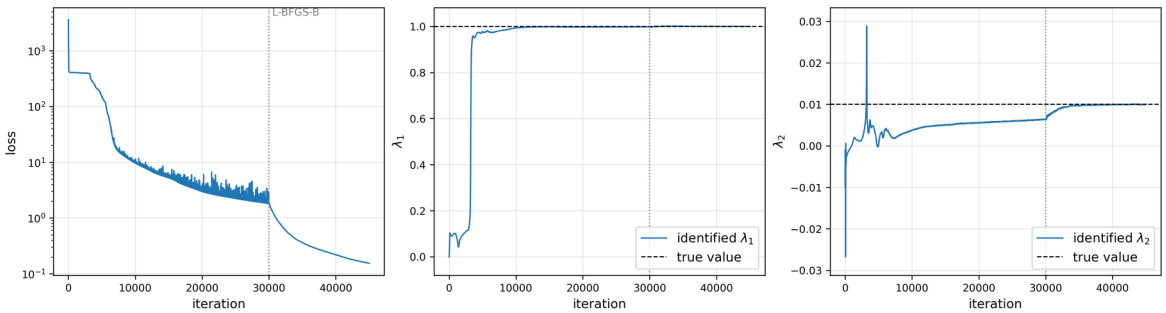


Figure 7.4: Training history at $Re = 100$ in the mid-wake window, clean data: total loss (left) and identified parameters λ_1 (middle) and λ_2 (right). The dotted vertical line marks the switch from Adam to L-BFGS-B after 30 000 iterations, and the dashed lines mark the true values.

The training history in Figure 7.4 shows the same behaviour as in the near wake. After a plateau of a few thousand iterations the loss drops sharply, and at the same moment λ_1 jumps close to one and reaches its true value within a few thousand Adam iterations. λ_2, in contrast, shows large spikes during the first 5000 iterations, when the network has not yet captured the vortex pattern, and then approaches the true value only slowly. It was still about 0.0063 at the switch to L-BFGS-B and was corrected mainly there: the loss drops by about another order of magnitude in this stage, and λ_2 moves to its final value within a few thousand iterations.

With 1% noise, the error of λ_1 stays very small (0.002%), while the error of λ_2 rises from 0.10% to 3.06% (Table 7.2). This is still below the 5.7% reported by Raissi et al. with the same noise level, although data, window and training differ [10]. The noise affects the two parameters very differently. The convection coefficient λ_1 is determined by the large convective terms, which are hardly changed by a perturbation of 1%. The viscosity coefficient λ_2, in contrast, is weighted in (4.51) by the second derivatives of the velocity. Compared with the smooth flow field, whose energy is concentrated at long wavelengths, the noise carries a large part of its energy at short wavelengths, and differentiation amplifies exactly these: a perturbation $A\sin(2\pi x/\Lambda)$ of amplitude A and wavelength Λ has a second derivative of size $A(2\pi/\Lambda)^2$, which grows rapidly as the wavelength decreases. If the network followed the noise, its second derivatives, and hence $\hat{\lambda}_2$, would be strongly disturbed. The physics residual counteracts this, because a function that follows random noise cannot satisfy the Navier–Stokes equations; the residual therefore acts as a regulariser that keeps the fitted field smooth [10]. The remaining error in $\hat{\lambda}_2$ reflects the balance between fitting the noisy data and satisfying the equations.

Two points must be kept in mind when comparing the noisy and the clean results. First, the noisy run uses Adam only, whereas the clean run uses Adam followed by L-BFGS-B, so part of the difference may come from the optimiser and not from the noise. Second, the noisy result is based on one random realisation of the noise; another realisation would give slightly different values.

7.2.2 Far Wake

Table 7.3 lists the identified parameters and field errors for the far-wake window, which reaches the outflow boundary. With clean data, the convection coefficient is identified to within 0.11% and the viscosity coefficient to within 2.96%. Both parameters can therefore still be identified from a part of the wake that lies 15 to 25 diameters behind the cylinder.

Table 7.3: Identified parameters and relative L^2 errors of the reconstructed fields at $Re = 100$ in the far-wake window $[15, 25] \times [-2, 2]$ from clean data and from data with 1% noise (true values $\lambda_1 = 1$, $\lambda_2 = 0.01$). The field errors (6.4) are evaluated at $t = 10$; the pressure error after the constant shift of Section 6.5.3.

Data	$\hat{\lambda}_1$	e_{λ_1} [%]	$\hat{\lambda}_2$	e_{λ_2} [%]	e_u [%]	e_v [%]	e_p [%]
clean	1.00112	0.112	0.010296	2.96	0.34	1.71	7.25
1% noise	1.00094	0.094	0.009937	0.63	–	–	–

Figure 7.5 shows that the vortex street is reproduced also in the far wake, where its structures are wider and weaker than in the mid wake, because the vortices have been spread out by viscous diffusion [22]. The velocity errors, $e_u = 0.34\%$ and $e_v = 1.71\%$, are close to those in the mid wake, but the pressure error is about three times larger ($e_p = 7.25\%$). Two effects may contribute. First, the far window reaches the outflow boundary $x = 25$, where $p = 0$ and zero normal derivatives of the velocity are imposed. These conditions keep the simulation stable but are not exact properties of the wake, so close to the outflow the reference fields

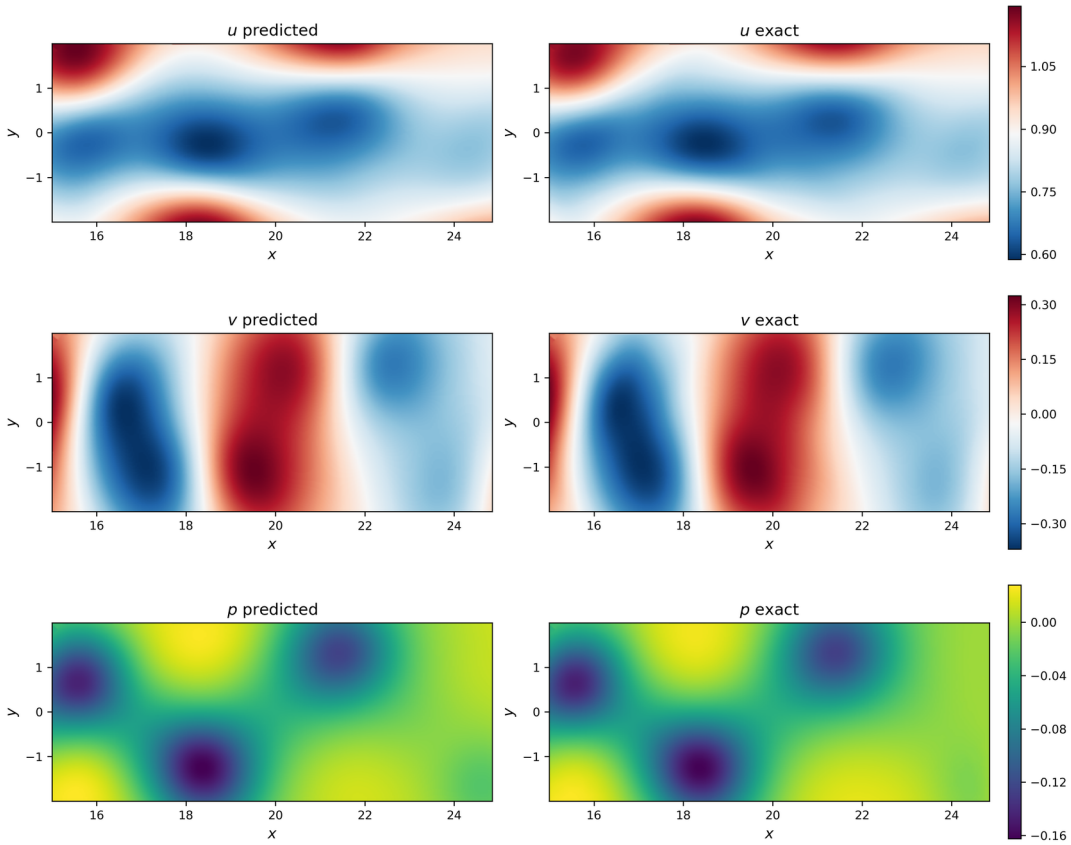


Figure 7.5: Predicted (left) and reference Nektar++ (right) velocity components u, v and pressure p at $Re = 100$ in the far-wake window $[15, 25] \times [-2, 2]$ at $t = 10$, clean data ($\hat{\lambda}_1 = 1.0011$, $\hat{\lambda}_2 = 0.01030$). No pressure data were used in training; the predicted pressure has been shifted by a constant.

are influenced by the boundary condition [91]. Second, the far window is longer (10 instead of 7 units), so the same 5000 points are spread over a larger area; since the pressure enters the residual only through its gradient, errors in the fitted pressure gradient can accumulate over the longer window. The first effect may also contribute to the larger error of λ_2 in this window compared with the mid wake.

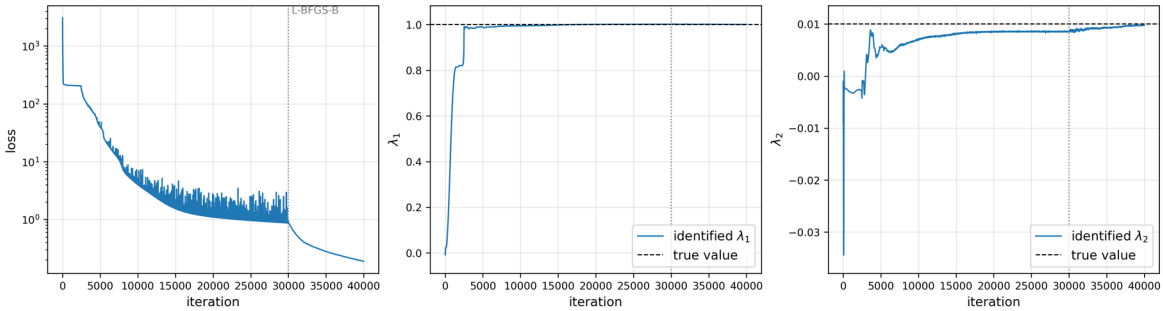


Figure 7.6: Training history at $Re = 100$ in the far-wake window, clean data: total loss (left) and identified parameters λ_1 (middle) and λ_2 (right). The dotted vertical line marks the switch from Adam to L-BFGS-B after 30 000 iterations, and the dashed lines mark the true values.

The training history in Figure 7.6 is similar to that in the mid wake: λ_1 converges early in the Adam stage, while λ_2 reaches its final value mainly in the L-BFGS-B stage.

With 1% noise, the error of λ_1 remains below 0.1% (0.094%), and the error of λ_2 is 0.63%, i.e. smaller than with clean data (2.96%, Table 7.3). This should not be read as an improvement caused by the noise. The noisy run was trained with Adam only, its result depends on

the single realisation of the noise and on the random initialisation, and each configuration was trained only once; a difference of this size between two single runs therefore cannot be attributed to the noise. The same two remarks about the optimiser and the single noise realisation as in Section 7.2.1 apply.

7.2.3 Off-Centre Windows

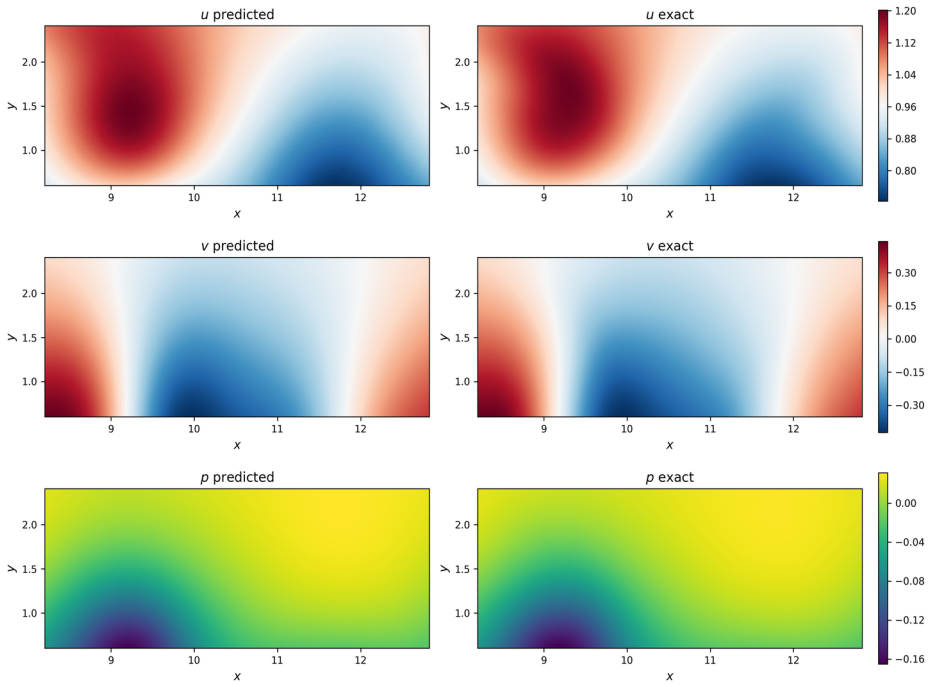
In a real experiment the measurement window is not necessarily centred on the wake. To test this situation, the PINN was trained on two smaller windows that lie off the wake axis: an upper window $[8, 13] \times [0.5, 2.5]$ in the mid wake and a lower window $[15, 20] \times [-2.5, -0.5]$ in the far wake (dashed in Figure 6.3). Each window has the size 5×2, compared with 7×4 and 10×4 for the centred mid-wake and far-wake windows, and it covers mainly one row of the vortex street. All other settings are those of Table 6.4; both windows were trained with clean data and with 1% noise. Table 7.4 lists the identified parameters, and Figure 7.7 compares the predicted and reference fields.

Table 7.4: Identified parameters and relative L^2 errors of the reconstructed fields at $Re = 100$ for the upper mid-wake window $[8, 13] \times [0.5, 2.5]$ and the lower far-wake window $[15, 20] \times [-2.5, -0.5]$, from clean data and from data with 1% noise (true values $\lambda_1 = 1$, $\lambda_2 = 0.01$). The field errors (6.4) are evaluated at $t = 10$; the pressure error after the constant shift of Section 6.5.3.

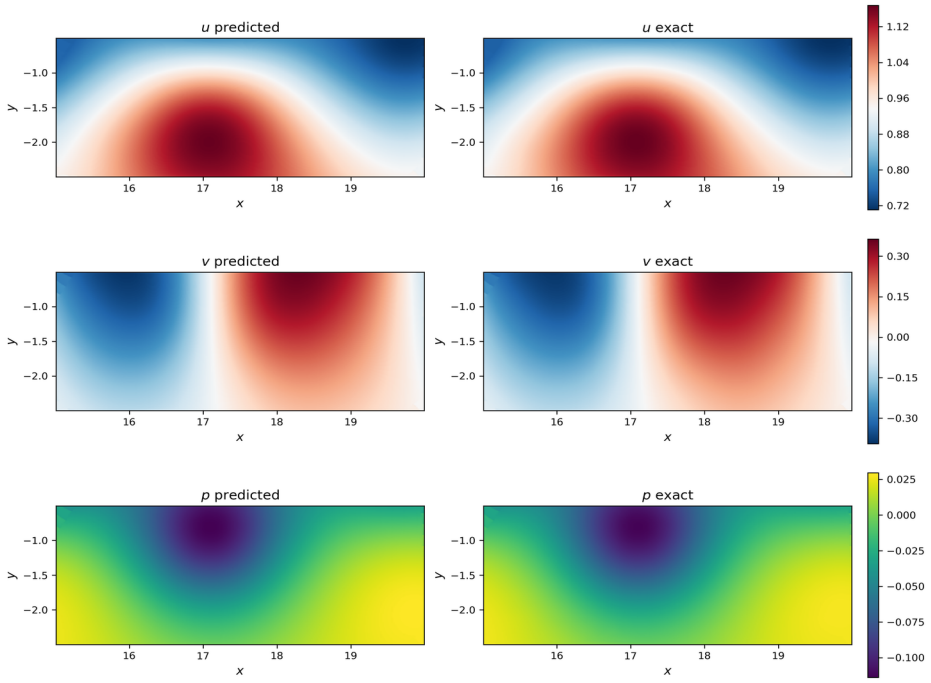
Window	$\hat{\lambda}_1$	e_{λ_1} [%]	$\hat{\lambda}_2$	e_{λ_2} [%]	e_u [%]	e_v [%]	e_p [%]
Mid wake, upper, clean	0.9994	0.06	0.01052	5.2	4.23	2.43	9.78
Mid wake, upper, 1% noise	1.00049	0.048	0.010751	7.51	–	–	–
Far wake, lower, clean	1.00160	0.160	0.009682	3.18	9.08	6.97	1.70
Far wake, lower, 1% noise	1.00169	0.169	0.009395	6.05	–	–	–

The convection coefficient is again identified very accurately, with errors of 0.06% in the upper and 0.16% in the lower window. The viscosity coefficient is less accurate than in the centred windows, clearly so in the mid wake and only slightly in the far wake: its error is 5.2% in the upper window ($\hat{\lambda}_2$ too large) and 3.18% in the lower window ($\hat{\lambda}_2$ too small), compared with 0.10% and 2.96% for the centred mid-wake and far-wake windows (Tables 7.2 and 7.3). Figure 7.7 shows that the main structures of the velocity field are reproduced in both windows, but the field errors are clearly larger than in the centred windows: $e_u = 4.23\%$ and 9.08%, compared with 0.32% and 0.34% (Tables 7.2 and 7.3). The pressure, which is not part of the training data, has the correct spatial pattern in both windows. In Figure 7.7a the predicted pressure is shown without the constant shift and is therefore offset from the reference, which is expected because only ∇p enters the residuals (4.47)–(4.48) [10].

The loss of accuracy in λ_2 is consistent with the argument of Section 4.5.5. The viscosity is identified only through the term $\lambda_2 \nabla^2 \mathbf{u}_\theta$, so a window contributes information about λ_2 only where the second derivatives of the velocity are large compared with the fitting error of the network. A centred window contains both rows of counter-rotating vortices and the strong shear between them on the wake axis, whereas an off-centre window contains mainly one row and the weaker variation towards the outer flow [22]. In addition, the off-centre windows are smaller, so they contain fewer vortex structures that constrain the viscous term. Both effects reduce the information about λ_2, while the convective balance, which determines λ_1, is still well represented. Since each window was trained only once, the difference between the two off-centre windows, and the opposite sign of the error in $\hat{\lambda}_2$, should not be over-interpreted. With 1% noise, λ_1 is still identified to within 0.17% in both windows, while the error of λ_2 is 7.51% in the upper and 6.05% in the lower window (Table 7.4).



(a) Upper mid-wake window $[8, 13] \times [0.5, 2.5]$ ($\hat{\lambda}_1 = 0.9994$, $\hat{\lambda}_2 = 0.01052$). The predicted pressure is shown without the constant shift.



(b) Lower far-wake window $[15, 20] \times [-2.5, -0.5]$ ($\hat{\lambda}_1 = 1.0016$, $\hat{\lambda}_2 = 0.00968$). The predicted pressure has been shifted by a constant.

Figure 7.7: Predicted (left) and reference (right) velocity components u, v and pressure p for the two off-centre windows at $Re = 100$ and $t = 10$. No pressure data were used in training.

7.2.4 Comparison of the Windows

In all windows λ_1 is identified more accurately than λ_2. This follows from the structure of the loss (Section 4.5.5). The gradients (4.50)–(4.51) show that λ_1 is multiplied by the convective terms $uu_x + vu_y$, whereas λ_2 multiplies the Laplacian of the velocity. Because $\lambda_2 = 1/Re$ is small, the viscous term $\lambda_2 \nabla^2 \mathbf{u}$ is much smaller than the convective term, so a small error in the fitted second derivatives gives a relatively large error in λ_2. Downstream the vortices are spread out by diffusion and the gradients are weaker [22], which may explain the larger error of λ_2 in the far wake (2.96%) than in the mid wake (0.10%); the outflow boundary next to the far window may also contribute (Section 7.2.2). The off-centre windows, however, show the opposite order (5.2% in the mid wake, 3.18% in the far wake), and each configuration was trained only once. The near-wake value (2.7%) cannot be compared directly with the others either, because that run used only a third of the Adam iterations (Section 7.1). The differences between the windows are therefore not large enough to rank them. What the results do show is that λ_2 is identified to within a few percent from every window, and that λ_1 is identified much more accurately than λ_2 in all of them.

For the placement of a measurement window, these results suggest that the viscosity is best identified from a window that is centred on the wake and contains both rows of vortices, while λ_1 can be identified from almost any part of the wake.

7.3 Higher Reynolds Number: $Re = 500$

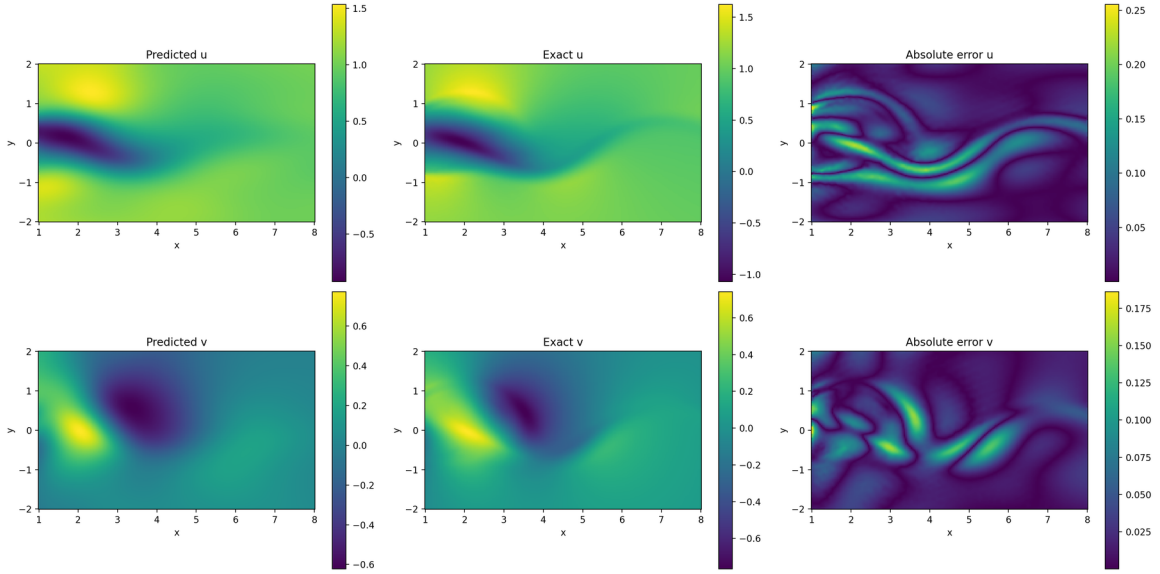
In the third experiment the Reynolds number was increased to $Re = 500$. The simulation used the refined mesh of Section 6.2 and a ten times smaller time step (Section 6.3), and the PINN was trained on the near-wake window $[1, 8] \times [-2, 2]$, as in the reproduction at $Re = 100$ (Table 6.5). The network and the loss are unchanged. Because the loss decreased much more slowly than at $Re = 100$, the training was made considerably longer: 300 000 Adam iterations, followed by L-BFGS-B with a limit of 100 000 iterations (Table 6.4). The true value $\lambda_2 = 1/500 = 0.002$ is used to evaluate the error, and the fields are evaluated at the snapshot $t = 20$, at the beginning of vortex shedding (Section 6.3.1).

Table 7.5: Identified parameters and relative L^2 errors of the reconstructed fields at $Re = 500$ in the near-wake window $[1, 8] \times [-2, 2]$ from clean data and from data with 1% noise (true values $\lambda_1 = 1$, $\lambda_2 = 0.002$). The field errors (6.4) are evaluated at $t = 20$; the pressure error after the constant shift of Section 6.5.3.

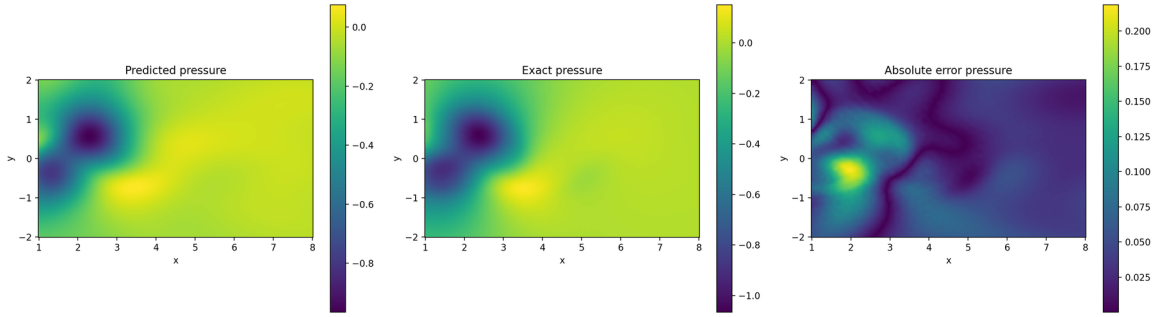
Data	$\hat{\lambda}_1$	e_{λ_1} [%]	$\hat{\lambda}_2$	e_{λ_2} [%]	e_u [%]	e_v [%]	e_p [%]
clean	0.96453	3.547	0.005554	177.69	6.14	16.00	17.37
1% noise	0.97136	2.864	0.004449	122.46	–	–	–

Table 7.5 lists the identified parameters and field errors at $Re = 500$; the result at $Re = 100$ in the same window is given in Table 7.1. The convection coefficient is still identified to within a few percent, with an error of 3.55%, although this error is about 30 times larger than at $Re = 100$ (0.11%). The identification of the viscosity, in contrast, fails: the error of λ_2 is 177.69%, so that the identified value is far from the true value. The identified value $\hat{\lambda}_2 = 0.005554$ is about 2.8 times the true value, i.e. the PINN sees an effective Reynolds number of only about $1/0.005554 \approx 180$.

The reconstructed fields in Figure 7.8 explain part of this result. The predicted velocity reproduces the main structures of the near wake: the region of reversed flow directly behind the cylinder, the bending of the wake behind it, and the alternating regions of positive and



(a) Velocity components u (top) and v (bottom).



(b) Pressure p (no pressure data were used in training).

Figure 7.8: Predicted (left) and reference Nektar++ (middle) fields and their absolute difference (right) at $Re = 500$ in the near-wake window $[1, 8] \times [-2, 2]$ at $t = 20$, clean data.

negative transverse velocity of the forming vortices. The relative errors (6.4) are, however, much larger than at $Re = 100$: $e_u = 6.14\%$ and $e_v = 16.0\%$, compared with $e_u = 4.51\%$ and $e_v = 1.25\%$ in the same window at $Re = 100$ (Table 7.1). The absolute error of u is largest, up to about 0.25, along the thin shear layers that bound the wake and along its curved centre line, and the error of v is largest, up to about 0.18, in the vortex cores between $x \approx 3$ and $x \approx 6$. These are exactly the regions with the steepest velocity gradients. The predicted fields are visibly smoother than the reference: the shear layers are wider and the wake behind $x \approx 4$ is more diffuse. A smoother fitted field has smaller second derivatives, so a larger λ_2 is needed to balance the viscous term in the residual, which is consistent with the overestimate of the viscosity. The pressure, which is not part of the training data, shows the correct pattern of low-pressure vortex cores near the cylinder, but its absolute error reaches about 0.22 in the formation region at $(x, y) \approx (2, -0.3)$, and the weak low-pressure core further downstream at about $(5, -0.5)$ is hardly visible in the prediction. After the constant shift of Section 6.5.3, the relative pressure error is $e_p = 17.37\%$, compared with 2.00% in the same window at $Re = 100$ (Tables 7.1 and 7.5).

The training history in Figure 7.9 differs clearly from that at $Re = 100$ (Figure 7.2). The loss decreases only slowly during the first 65 000 Adam iterations and drops by about a factor of three between 65 000 and 75 000 iterations. From about 80 000 iterations onwards, it shows frequent spikes of up to one order of magnitude, and at about 197 000 iterations a single spike

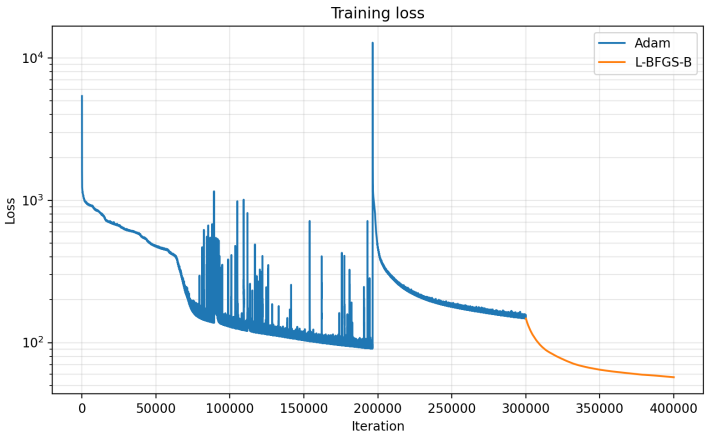


Figure 7.9: Training loss at $Re = 500$ in the near-wake window: 300 000 Adam iterations (blue) followed by L-BFGS-B (orange), which stopped at its iteration limit. The loss is the implemented loss $N_d\mathcal{L}$ (Section 6.5), as in Figure 7.2.

raises the loss by more than two orders of magnitude, from about 90 to about 1.3×10^4. Adam recovers from this only partly: at the end of the Adam stage the loss is about 150, higher than before the spike. L-BFGS-B then reduces it to about 57, but stops at its iteration limit without having converged. The final loss is about 57, compared with about 0.4 at $Re = 100$. These two values cannot be compared directly, because the size of the residuals depends on the velocity gradients of the flow, which are larger at $Re = 500$; the velocity errors of Table 7.5, however, which do not depend on this scaling, confirm that the fit is much worse than at $Re = 100$. At $Re = 500$ the network could not fit the data and satisfy the residuals at the same time, and a parameter identification from such a poorly converged state cannot be expected to be accurate. Spikes of this kind are typical of first-order training with a constant learning rate on an ill-conditioned loss: when the curvature of the loss becomes large in some direction, a step of the size of the learning rate overshoots [13, 31].

Training with 1% noise at $Re = 500$ gives errors of 2.86% for λ_1 and 122.46% for λ_2 (Table 7.5). Unlike the noisy runs at $Re = 100$ (Sections 7.2.1 and 7.2.2), this run was trained with both stages: 200 000 Adam iterations followed by L-BFGS-B with 50 000 iterations at most (Table 6.4), which again stopped at its iteration limit. The identification of λ_2 fails with and without noise; the somewhat smaller errors with noise should not be read as an improvement, because both runs are far from the true value and each was trained only once.

Two effects make the case $Re = 500$ harder. First, the viscous term is five times smaller relative to the convective term than at $Re = 100$, so the information about λ_2 in the residual is weaker, and a given absolute error in $\hat{\lambda}_2$ corresponds to a five times larger relative error. Second, the flow has steeper gradients and more compact vortices, which a network of fixed size fits less accurately; errors in the fitted second derivatives then enter λ_2 directly. Both effects are visible in the results: the largest velocity errors lie in the shear layers and vortex cores, and the error of λ_2 is much more affected than that of λ_1. This agrees with the observations in the literature that PINN training becomes harder as the flow becomes more multi-scale, because the gradients of the loss terms become more imbalanced [13, 14, 30]. Since the same network of 8×20 neurons was used as at $Re = 100$, a larger network, adaptive loss weights [13, 32] or a learning-rate schedule are the most obvious changes for a further attempt.

A third factor is the training data. At $Re = 500$ the snapshots cover $t \in [0, 100]$ and therefore include the start-up of the flow (Sections 6.3 and 6.3.1): before $t \approx 20$ no vortices are shed, and until about $t \approx 40$ the shedding amplitude still grows. Because the training points are drawn uniformly from all snapshots, about 40% of them lie in this non-periodic part, which the

network has to fit together with the periodic wake. At $Re = 100$, in contrast, only snapshots from the periodic state were used. The results at $Re = 500$ therefore do not show the effect of the Reynolds number alone. A training on the periodic part only, for example $t \geq 40$, would separate the two effects.

The simulation at $Re = 500$ is two-dimensional. A real cylinder wake becomes three-dimensional at about $Re \approx 190$ [22, 90], so the two-dimensional flow at $Re = 500$ is not a realistic model of a physical experiment. Since the data and the PINN both follow the two-dimensional equations, the experiment is nevertheless a consistent test of the method at a higher Reynolds number.

7.4 Discussion

Reproduction of the benchmark. The port from TensorFlow 1 to TensorFlow 2 and the new Nektar++ simulation reproduce the benchmark of Raissi et al. [10]: the identified parameters agree with the published values to within the differences that are to be expected from a different data set, different random training points and a different number of iterations.

Why the viscosity is harder to identify. The difference between λ_1 and λ_2, which appears in every experiment, can be understood from a pressure-free form of the residuals. Taking the derivative ∂_x of the residual (4.48), subtracting the derivative ∂_y of the residual (4.47) and using $\partial_x u + \partial_y v = 0$ eliminates the pressure and gives the vorticity transport equation [61, Ch. 5]

$$\omega_t + \lambda_1 \left(u\,\omega_x + v\,\omega_y\right) = \lambda_2 \left(\omega_{xx} + \omega_{yy}\right), \qquad \omega = v_x - u_y. \tag{7.1}$$

Here ω_t is the local change of the vorticity, $u\omega_x + v\omega_y$ its transport by the flow and $\omega_{xx} + \omega_{yy}$ its viscous diffusion. Equation (7.1) is linear in λ_1 and λ_2, so at every point of the window the data provide one linear relation between the two unknowns. The convective term is of order one wherever there are vortices, whereas the diffusion term is multiplied by $1/Re$ and is large only in the vortex cores and in the thin shear layers. In the outer flow, where $\omega \approx 0$, both terms vanish and the data carry no information about the parameters at all. This explains why λ_1 is found accurately in every window, and why windows that cover only one side of the vortex street and part of the outer flow tend to give larger errors in λ_2 (Section 7.2.3). The same equation was used by Rudy et al. [25] to identify the viscosity of this wake by sparse regression.

What the region study means for measurements. In an experiment the measurement window is usually limited, for example to the field of view of a camera. The results show that the window does not have to include the formation region behind the cylinder: data from the mid or far wake alone gave λ_2 to within a few percent (0.10% and 2.96%). A window centred on the wake axis should be preferred, because it contains the vortex cores of both rows, in which the viscous term of (7.1) is largest.

Noise. With 1% noise, λ_1 remained accurate in all windows. The error in λ_2 increased by 3.7 percentage points in the near wake and by 3.0 percentage points in the mid wake, while in the far wake the noisy run happened to give a smaller error than the clean run (0.63% instead of 2.96%), which shows the run-to-run variation of single runs. The physics residual prevents the network from following the noise, because a field that follows random noise cannot satisfy (7.1); this is the same regularising effect that was reported by Raissi et al. [10]. Sparse regression, which differentiates the data directly, is more sensitive to noise [25]. A

Bayesian PINN [39] would in addition give an uncertainty of the identified viscosity, which a single deterministic run cannot provide.

Higher Reynolds number. At $Re = 500$ both effects that make λ_2 difficult become stronger: the diffusion term in (7.1) is five times smaller relative to the convective term, and the steeper gradients are harder to fit with a network of fixed size. Even with about 400 000 training iterations, the velocity errors rose to 6.14% (u) and 16.0% (v), and the identification of the viscosity failed ($e_{\lambda_2} = 177.69\%$), while λ_1 was still found to within 3.55%. The identified value $\hat{\lambda}_2 \approx 0.0056$ and the visibly smoother predicted fields (Figure 7.8) are consistent with a network that cannot represent the thin shear layers and compensates with a larger viscosity. This agrees with the observation that PINN training becomes harder as flows become more multi-scale [13, 14, 30]. In addition, the training data at $Re = 500$ include the start-up of the flow, so part of the failure may be caused by the non-periodic data rather than by the Reynolds number (Section 7.3).

Comparison with classical approaches. For the present problem, with simulation data and a known solver, classical methods are a serious alternative: sparse regression of (7.1) [25] and adjoint-based optimisation [26] can identify the viscosity of this wake. The advantages of the PINN are that it needs neither a mesh of the measurement window nor an adjoint solver, that it works with scattered points, and that it reconstructs the pressure at the same time. Its disadvantages are the cost of training, 3–4 hours per run at $Re = 100$ and about three days at $Re = 500$ on the hardware of Table 6.4, and the lack of a guarantee that training converges.

Limitations. The results have several limitations. The data are simulation data, and the noise was added artificially with a single, small noise level of 1%, whereas real measurements, for example from particle image velocimetry, also contain systematic errors and missing data. The flow is two-dimensional, only one geometry and two Reynolds numbers were considered, and at $Re = 500$ the two-dimensional flow is not a realistic model of a physical wake. The network architecture and the training settings were taken from the original work and were not optimised for each case, and the same number of training points was used in windows of different size. Each configuration was trained only once, so the variation caused by the random choice of training points, the random initialisation and the noise realisation has not been quantified; the differences between the windows should be read with this in mind. The far-wake window reaches the outflow boundary, where the reference solution is influenced by the artificial outflow condition. Finally, the reference simulations differ from those of Raissi et al. in the lateral boundary conditions, they use the simple pressure boundary condition $\partial p/\partial n = 0$ instead of the high-order condition (5.8), and the polynomial order was not varied. At $Re = 100$ the shedding frequency and the forces are 4–5% above literature values, most likely because of the lateral boundaries (Section 6.3.1). At $Re = 500$, moreover, the training data include the start-up of the flow, whereas at $Re = 100$ only the periodic state was used.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This thesis investigated the identification of the convection coefficient λ_1 and the viscosity coefficient λ_2 of the incompressible Navier–Stokes equations with physics-informed neural networks, using velocity data only. Building on the cylinder-wake benchmark of Raissi et al. [10], a workflow from mesh generation in Gmsh and simulation in Nektar++ to PINN training was established, and the original implementation was ported to TensorFlow 2. With the author's own simulation data, the benchmark at $Re = 100$ was reproduced, including the reconstruction of the pressure field without pressure data.

The study of different regions of the wake showed that accurate identification does not require data directly behind the cylinder. Trained only on the mid-wake or the far-wake window, the PINN found λ_1 with errors of 0.009% and 0.11% and λ_2 with errors of 0.10% and 2.96%, respectively. Smaller windows that cover only one side of the vortex street still gave λ_1 to within 0.16%, but the error of λ_2 rose to 5.2% and 3.2%. The viscosity is therefore best identified from windows centred on the wake, because only there do the data contain the vortex cores of both rows, in which the viscous term is largest (Section 7.4).

With 1% noise in the training data, the error of λ_1 remained below 0.1% in the centred mid-wake and far-wake windows, while the error of λ_2 was 3.1% in the mid wake and 0.63% in the far wake. The physics residual acts as a regulariser that keeps the fitted field smooth. Because each configuration was trained only once, the effect of the noise cannot be separated in detail from the variation between individual runs.

At the higher Reynolds number $Re = 500$, the identification of the viscosity failed with the present network and training: λ_1 was still found with an error of 3.55%, but λ_2 only with an error of 177.69%, although the training was much longer than at $Re = 100$. This is in line with the known training difficulties of PINNs for more complex flows [13, 14]. Because the training data at $Re = 500$ also contain the start-up of the flow, this result does not isolate the effect of the Reynolds number.

In summary, in the experiments of this thesis PINNs identified the governing parameters of two-dimensional, time-periodic cylinder wakes from velocity data alone, also from a limited part of the flow and with a small amount of noise. The identification of the viscosity is the limiting part of the method: it becomes less accurate as the flow becomes more complex and, with the network, training and data used here, it did not succeed at $Re = 500$.

8.2 Future Work

Repeating each configuration with several random seeds would quantify the spread of the identified parameters and show which differences between the training windows are significant. Higher noise levels, repeated noise realisations and real measurements such as particle image

velocimetry data would show how robust the region-based identification is in practice. A classical baseline, for example a least-squares fit of λ_1 and λ_2 in the vorticity equation (7.1) with derivatives computed from the simulation data [25], would show more clearly what the PINN adds. The reference simulations could be strengthened by a variation of the polynomial order, a larger distance to the lateral boundaries and the use of the high-order pressure boundary condition. At $Re = 500$, a training on the periodic part of the simulation only, for example $t \geq 40$, would show how much of the failure is caused by the start-up transient in the present data, and a run with λ_2 fixed at its true value would show whether the failure is caused by the optimiser or by the limited size of the network. Training strategies such as adaptive loss weighting [13, 32] and causal training [34] could improve the identification of λ_2 at higher Reynolds numbers, and a systematic study between $Re = 100$ and $Re = 500$ would show where its accuracy begins to drop. Since the real wake becomes three-dimensional at about $Re \approx 190$ [22], physically realistic studies at higher Reynolds numbers would require three-dimensional data and networks. Finally, other geometries and domain-decomposition approaches such as XPINNs [36], which combine several training windows, are natural extensions of this work.

Bibliography

[1] Parviz Moin and Krishnan Mahesh. 'Direct numerical simulation: A tool in turbulence research'. In: *Annual Review of Fluid Mechanics* 30 (1998), pp. 539–578.

[2] Jeffrey Slotnick, Abdollah Khodadoust, Juan Alonso, David Darmofal, William Gropp, Elizabeth Lurie and Dimitri Mavriplis. *CFD Vision 2030 Study: A Path to Revolutionary Computational Aerosciences*. Tech. rep. NASA/CR-2014-218178. NASA Langley Research Center, 2014.

[3] Mark Asch, Marc Bocquet and Maëlle Nodet. *Data Assimilation: Methods, Algorithms, and Applications*. Philadelphia: SIAM, 2016.

[4] Dimitry P. G. Foures, Nicolas Dovetta, Denis Sipp and Peter J. Schmid. 'A data-assimilation method for Reynolds-averaged Navier–Stokes-driven mean flow reconstruction'. In: *Journal of Fluid Mechanics* 759 (2014), pp. 404–431.

[5] Steven L. Brunton, Bernd R. Noack and Petros Koumoutsakos. 'Machine learning for fluid mechanics'. In: *Annual Review of Fluid Mechanics* 52 (2020), pp. 477–508.

[6] Karthik Duraisamy, Gianluca Iaccarino and Heng Xiao. 'Turbulence modeling in the age of data'. In: *Annual Review of Fluid Mechanics* 51 (2019), pp. 357–377.

[7] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang and Liu Yang. 'Physics-informed machine learning'. In: *Nature Reviews Physics* 3.6 (2021), pp. 422–440.

[8] M. W. M. Gamini Dissanayake and Nhan Phan-Thien. 'Neural-network-based approximations for solving partial differential equations'. In: *Communications in Numerical Methods in Engineering* 10.3 (1994), pp. 195–201.

[9] Isaac E. Lagaris, Aristidis Likas and Dimitrios I. Fotiadis. 'Artificial neural networks for solving ordinary and partial differential equations'. In: *IEEE Transactions on Neural Networks* 9.5 (1998), pp. 987–1000.

[10] Maziar Raissi, Paris Perdikaris and George Em Karniadakis. 'Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations'. In: *Journal of Computational Physics* 378 (2019), pp. 686–707.

[11] Atılım Güneş Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul and Jeffrey Mark Siskind. 'Automatic differentiation in machine learning: A survey'. In: *Journal of Machine Learning Research* 18.153 (2018), pp. 1–43.

[12] Salvatore Cuomo, Vincenzo Schiano Di Cola, Fabio Giampaolo, Gianluigi Rozza, Maziar Raissi and Francesco Piccialli. 'Scientific machine learning through physics-informed neural networks: Where we are and what's next'. In: *Journal of Scientific Computing* 92.3 (2022), p. 88.

[13] Sifan Wang, Yujun Teng and Paris Perdikaris. 'Understanding and mitigating gradient flow pathologies in physics-informed neural networks'. In: *SIAM Journal on Scientific Computing* 43.5 (2021), A3055–A3081.

[14] Aditi Krishnapriyan, Amir Gholami, Shandian Zhe, Robert M. Kirby and Michael W. Mahoney. 'Characterizing possible failure modes in physics-informed neural networks'. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 34. 2021, pp. 26548–26560.

[15] Xiaowei Jin, Shengze Cai, Hui Li and George Em Karniadakis. 'NSFnets (Navier–Stokes flow nets): Physics-informed neural networks for the incompressible Navier–Stokes equations'. In: *Journal of Computational Physics* 426 (2021), p. 109951.

[16] Maziar Raissi, Alireza Yazdani and George Em Karniadakis. 'Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations'. In: *Science* 367.6481 (2020), pp. 1026–1030.

[17] Maziar Raissi, Zhicheng Wang, Michael S. Triantafyllou and George Em Karniadakis. 'Deep learning of vortex-induced vibrations'. In: *Journal of Fluid Mechanics* 861 (2019), pp. 119–137.

[18] Ehsan Haghighat, Maziar Raissi, Adrian Moure, Hector Gomez and Ruben Juanes. 'A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics'. In: *Computer Methods in Applied Mechanics and Engineering* 379 (2021), p. 113741.

[19] Shengze Cai, Zhicheng Wang, Sifan Wang, Paris Perdikaris and George Em Karniadakis. 'Physics-informed neural networks for heat transfer problems'. In: *Journal of Heat Transfer* 143.6 (2021), p. 060801.

[20] Georgios Kissas, Yibo Yang, Eileen Hwuang, Walter R. Witschey, John A. Detre and Paris Perdikaris. 'Machine learning in cardiovascular flows modeling: Predicting arterial blood pressure from non-invasive 4D flow MRI data using physics-informed neural networks'. In: *Computer Methods in Applied Mechanics and Engineering* 358 (2020), p. 112623.

[21] Amirhossein Arzani, Jian-Xun Wang and Roshan M. D'Souza. 'Uncovering near-wall blood flow from sparse data with physics-informed neural networks'. In: *Physics of Fluids* 33.7 (2021), p. 071905.

[22] Charles H. K. Williamson. 'Vortex dynamics in the cylinder wake'. In: *Annual Review of Fluid Mechanics* 28 (1996), pp. 477–539.

[23] Zhiping Mao, Ameya D. Jagtap and George Em Karniadakis. 'Physics-informed neural networks for high-speed flows'. In: *Computer Methods in Applied Mechanics and Engineering* 360 (2020), p. 112789.

[24] Hamidreza Eivazi, Mojtaba Tahani, Philipp Schlatter and Ricardo Vinuesa. 'Physics-informed neural networks for solving Reynolds-averaged Navier–Stokes equations'. In: *Physics of Fluids* 34.7 (2022), p. 075117.

[25] Samuel H. Rudy, Steven L. Brunton, Joshua L. Proctor and J. Nathan Kutz. 'Data-driven discovery of partial differential equations'. In: *Science Advances* 3.4 (2017), e1602614.

[26] Zhen Zhang, Alessandro Alla and George Em Karniadakis. *Adjoint method versus physics-informed neural networks in PDE-constrained inverse problems*. 2026. arXiv: `2606.12337`.

[27] Vladimir Parfenyev, Mark Blumenau and Ilia Nikitin. 'Inferring parameters and reconstruction of two-dimensional turbulent flows with physics-informed neural networks'. In: *JETP Letters* 120.8 (2024), pp. 599–607.

[28] Lu Lu, Xuhui Meng, Zhiping Mao and George Em Karniadakis. 'DeepXDE: A deep learning library for solving differential equations'. In: *SIAM Review* 63.1 (2021), pp. 208–228.

[29] Ameya D. Jagtap, Zhiping Mao, Nikolaus A. Adams and George Em Karniadakis. 'Physics-informed neural networks for inverse problems in supersonic flows'. In: *Journal of Computational Physics* 466 (2022), p. 111402.

[30] Shengze Cai, Zhiping Mao, Zhicheng Wang, Minglang Yin and George Em Karniadakis. 'Physics-informed neural networks (PINNs) for fluid mechanics: A review'. In: *Acta Mechanica Sinica* 37.12 (2021), pp. 1727–1738.

[31] Sifan Wang, Xinling Yu and Paris Perdikaris. 'When and why PINNs fail to train: A neural tangent kernel perspective'. In: *Journal of Computational Physics* 449 (2022), p. 110768.

[32] Levi D. McClenny and Ulisses M. Braga-Neto. 'Self-adaptive physics-informed neural networks'. In: *Journal of Computational Physics* 474 (2023), p. 111722.

[33] Jeremy Yu, Lu Lu, Xuhui Meng and George Em Karniadakis. 'Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems'. In: *Computer Methods in Applied Mechanics and Engineering* 393 (2022), p. 114823.

[34] Sifan Wang, Shyam Sankaran and Paris Perdikaris. 'Respecting causality for training physics-informed neural networks'. In: *Computer Methods in Applied Mechanics and Engineering* 421 (2024), p. 116813.

[35] Ameya D. Jagtap, Ehsan Kharazmi and George Em Karniadakis. 'Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems'. In: *Computer Methods in Applied Mechanics and Engineering* 365 (2020), p. 113028.

[36] Ameya D. Jagtap and George Em Karniadakis. 'Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations'. In: *Communications in Computational Physics* 28.5 (2020), pp. 2002–2041.

[37] Khemraj Shukla, Ameya D. Jagtap and George Em Karniadakis. 'Parallel physics-informed neural networks via domain decomposition'. In: *Journal of Computational Physics* 447 (2021), p. 110683.

[38] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang and George Em Karniadakis. 'Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators'. In: *Nature Machine Intelligence* 3 (2021), pp. 218–229.

[39] Liu Yang, Xuhui Meng and George Em Karniadakis. 'B-PINNs: Bayesian physics-informed neural networks for forward and inverse PDE problems with noisy data'. In: *Journal of Computational Physics* 425 (2021), p. 109913.

[40] Chris D. Cantwell, David Moxey, Andrew Comerford, Alessandro Bolis, Gabriele Rocco, Gianmarco Mengaldo, Daniele de Grazia, Sergey Yakovlev, Jean-Eloi Lombard, Dirk Ekelschot, Bruce Jordi, Hui Xu, Yumnah Mohamied, Claes Eskilsson, Bram Nelson, Peter Vos, Chiara Biotto, Robert M. Kirby and Spencer J. Sherwin. 'Nektar++: An open-source spectral/hp element framework'. In: *Computer Physics Communications* 192 (2015), pp. 205–219.

[41] Warren S. McCulloch and Walter Pitts. 'A logical calculus of the ideas immanent in nervous activity'. In: *The Bulletin of Mathematical Biophysics* 5.4 (1943), pp. 115–133.

[42] Frank Rosenblatt. 'The perceptron: A probabilistic model for information storage and organization in the brain'. In: *Psychological Review* 65.6 (1958), pp. 386–408.

[43] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: MIT Press, 1969.

[44] Ian Goodfellow, Yoshua Bengio and Aaron Courville. *Deep Learning*. Cambridge, MA: MIT Press, 2016.

[45] David E. Rumelhart, Geoffrey E. Hinton and Ronald J. Williams. 'Learning representations by back-propagating errors'. In: *Nature* 323.6088 (1986), pp. 533–536.

[46] George Cybenko. 'Approximation by superpositions of a sigmoidal function'. In: *Mathematics of Control, Signals and Systems* 2.4 (1989), pp. 303–314.

[47] Kurt Hornik, Maxwell Stinchcombe and Halbert White. 'Multilayer feedforward networks are universal approximators'. In: *Neural Networks* 2.5 (1989), pp. 359–366.

[48] Moshe Leshno, Vladimir Ya. Lin, Allan Pinkus and Shimon Schocken. 'Multilayer feedforward networks with a nonpolynomial activation function can approximate any function'. In: *Neural Networks* 6.6 (1993), pp. 861–867.

[49] Kurt Hornik. 'Approximation capabilities of multilayer feedforward networks'. In: *Neural Networks* 4.2 (1991), pp. 251–257.

[50] Tim De Ryck, Samuel Lanthaler and Siddhartha Mishra. 'On the approximation of functions by tanh neural networks'. In: *Neural Networks* 143 (2021), pp. 732–750.

[51] Anthropic. *Claude*. Version Claude Opus 5.5. Large language model; used for review, language editing and generating figures. 2026. URL: `https://claude.ai`.

[52] Xavier Glorot and Yoshua Bengio. 'Understanding the difficulty of training deep feedforward neural networks'. In: *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*. Vol. 9. Proceedings of Machine Learning Research. 2010, pp. 249–256.

[53] Yann LeCun, Léon Bottou, Genevieve B. Orr and Klaus-Robert Müller. 'Efficient BackProp'. In: *Neural Networks: Tricks of the Trade*. Ed. by Genevieve B. Orr and Klaus-Robert Müller. Vol. 1524. Lecture Notes in Computer Science. Berlin: Springer, 1998, pp. 9–50.

[54] Vinod Nair and Geoffrey E. Hinton. 'Rectified linear units improve restricted Boltzmann machines'. In: *Proceedings of the 27th International Conference on Machine Learning (ICML)*. 2010, pp. 807–814.

[55] Kunihiko Fukushima. 'Visual feature extraction by a multilayered network of analog threshold elements'. In: *IEEE Transactions on Systems Science and Cybernetics* 5.4 (1969), pp. 322–333.

[56] Xavier Glorot, Antoine Bordes and Yoshua Bengio. 'Deep sparse rectifier neural networks'. In: *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*. Vol. 15. Proceedings of Machine Learning Research. 2011, pp. 315–323.

[57] Prajit Ramachandran, Barret Zoph and Quoc V. Le. *Searching for activation functions*. 2017. arXiv: `1710.05941`.

[58] Ameya D. Jagtap, Kenji Kawaguchi and George Em Karniadakis. 'Adaptive activation functions accelerate convergence in deep and physics-informed neural networks'. In: *Journal of Computational Physics* 404 (2020), p. 109136.

[59] David Moxey, Chris D. Cantwell, Yan Bao, Andrea Cassinelli, Giacomo Castiglioni, Sehun Chun, Emilia Juda, Ehsan Kazemi, Kilian Lackhove, Julian Marcon, Gianmarco Mengaldo, Douglas Serson, Michael Turner, Hui Xu, Joaquim Peiró, Robert M. Kirby and Spencer J. Sherwin. 'Nektar++: Enhancing the capability and application of high-fidelity spectral/hp element methods'. In: *Computer Physics Communications* 249 (2020), p. 107110.

[60] George Em Karniadakis and Spencer J. Sherwin. *Spectral/hp Element Methods for Computational Fluid Dynamics*. 2nd ed. Oxford: Oxford University Press, 2005.

[61] George K. Batchelor. *An Introduction to Fluid Dynamics*. Cambridge: Cambridge University Press, 1967.

[62] Alexandre J. Chorin and Jerrold E. Marsden. *A Mathematical Introduction to Fluid Mechanics*. 3rd ed. Vol. 4. Texts in Applied Mathematics. New York: Springer, 1993.

[63] Morton E. Gurtin. *An Introduction to Continuum Mechanics*. New York: Academic Press, 1981.

[64] George G. Stokes. 'On the theories of the internal friction of fluids in motion, and of the equilibrium and motion of elastic solids'. In: *Transactions of the Cambridge Philosophical Society* 8 (1845), pp. 287–319.

[65] Rutherford Aris. *Vectors, Tensors, and the Basic Equations of Fluid Mechanics*. New York: Dover, 1989.

[66] Lev D. Landau and Evgeny M. Lifshitz. *Fluid Mechanics*. 2nd ed. Vol. 6. Course of Theoretical Physics. Oxford: Pergamon Press, 1987.

[67] Roger Temam. *Navier–Stokes Equations: Theory and Numerical Analysis*. Providence, RI: AMS Chelsea Publishing, 2001.

[68] Osborne Reynolds. 'An experimental investigation of the circumstances which determine whether the motion of water shall be direct or sinuous, and of the law of resistance in parallel channels'. In: *Philosophical Transactions of the Royal Society of London* 174 (1883), pp. 935–982.

[69] Andreas Griewank and Andrea Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. 2nd ed. Philadelphia: SIAM, 2008.

[70] Diederik P. Kingma and Jimmy Ba. 'Adam: A method for stochastic optimization'. In: *International Conference on Learning Representations (ICLR)*. 2015. arXiv: `1412.6980`.

[71] TensorFlow Developers. *tf.keras.optimizers.Adam: API documentation*. 2026. URL: `https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/Adam` (visited on 26/10/2026).

[72] Richard H. Byrd, Peihuang Lu, Jorge Nocedal and Ciyou Zhu. 'A limited memory algorithm for bound constrained optimization'. In: *SIAM Journal on Scientific Computing* 16.5 (1995), pp. 1190–1208.

[73] Dong C. Liu and Jorge Nocedal. 'On the limited memory BFGS method for large scale optimization'. In: *Mathematical Programming* 45.1–3 (1989), pp. 503–528.

[74] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. 2nd ed. New York: Springer, 2006.

[75] Richard H. Byrd, Jorge Nocedal and Robert B. Schnabel. 'Representations of quasi-Newton matrices and their use in limited memory methods'. In: *Mathematical Programming* 63.1–3 (1994), pp. 129–156.

[76] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C. J. Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt and SciPy 1.0 Contributors. 'SciPy 1.0: Fundamental algorithms for scientific computing in Python'. In: *Nature Methods* 17 (2020), pp. 261–272.

[77] Ciyou Zhu, Richard H. Byrd, Peihuang Lu and Jorge Nocedal. 'Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization'. In: *ACM Transactions on Mathematical Software* 23.4 (1997), pp. 550–560.

[78] Spencer J. Sherwin and George Em Karniadakis. 'A triangular spectral element method; applications to the incompressible Navier–Stokes equations'. In: *Computer Methods in Applied Mechanics and Engineering* 123.1–4 (1995), pp. 189–229.

[79] Christoph Schwab. *p- and hp-Finite Element Methods: Theory and Applications in Solid and Fluid Mechanics*. Oxford: Oxford University Press, 1998.

[80] Susanne C. Brenner and L. Ridgway Scott. *The Mathematical Theory of Finite Element Methods*. 3rd ed. Vol. 15. Texts in Applied Mathematics. New York: Springer, 2008.

[81] George Em Karniadakis, Moshe Israeli and Steven A. Orszag. 'High-order splitting methods for the incompressible Navier–Stokes equations'. In: *Journal of Computational Physics* 97.2 (1991), pp. 414–443.

[82] Nektar++ Development Team. *Nektar++: Spectral/hp Element Framework – User Guide*. Imperial College London and University of Utah. 2026. URL: `https://www.nektar.info/` (visited on 30/09/2026).

[83] Christophe Geuzaine and Jean-François Remacle. 'Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities'. In: *International Journal for Numerical Methods in Engineering* 79.11 (2009), pp. 1309–1331.

[84] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu and Xiaoqiang Zheng. 'TensorFlow: A system for large-scale machine learning'. In: *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 2016, pp. 265–283.

[85] M. Behr, D. Hastreiter, S. Mittal and T. E. Tezduyar. 'Incompressible flow past a circular cylinder: dependence of the computed flow field on the location of the lateral boundaries'. In: *Computer Methods in Applied Mechanics and Engineering* 123.1–4 (1995), pp. 309–316.

[86] Christophe Geuzaine and Jean-François Remacle. *Gmsh Reference Manual*. URL: `https://gmsh.info/doc/texinfo/gmsh.html` (visited on 22/10/2026).

[87] James Ahrens, Berk Geveci and Charles Law. 'ParaView: An end-user tool for large data visualization'. In: *The Visualization Handbook*. Ed. by Charles D. Hansen and Chris R. Johnson. Burlington, MA: Elsevier, 2005, pp. 717–731.

[88] Charles H. K. Williamson. 'Defining a universal and continuous Strouhal–Reynolds number relationship for the laminar vortex shedding of a circular cylinder'. In: *Physics of Fluids* 31.10 (1988), pp. 2742–2744.

[89] J. Park, K. Kwon and H. Choi. 'Numerical solutions of flow past a circular cylinder at Reynolds numbers up to 160'. In: *KSME International Journal* 12.6 (1998), pp. 1200–1205.

[90] George Em Karniadakis and George S. Triantafyllou. 'Three-dimensional dynamics and transition to turbulence in the wake of bluff objects'. In: *Journal of Fluid Mechanics* 238 (1992), pp. 1–30.

[91] Suchuan Dong, George Em Karniadakis and Chryssostomos Chryssostomidis. 'A robust and accurate outflow boundary condition for incompressible flow simulations on severely-truncated unbounded domains'. In: *Journal of Computational Physics* 261 (2014), pp. 83–105.

Appendix A

Mesh and Simulation Set-up

This appendix lists the input files of the reference simulation at $Re = 500$ (Sections 6.2 and 6.3), so that the data can be regenerated. Only the parts that determine the result are shown; the mesh coordinates themselves are omitted. Comments in the files have been shortened; all commands and values are those used in the simulation.

A.1 Gmsh Geometry and Size Fields

The mesh was generated with Gmsh [83]. The element size is controlled by size fields [86, Sec. 'Specifying mesh element sizes']: a `Distance` field measures the distance to the cylinder, a `Threshold` field turns it into an element size that grows from `lc_cyl` at the wall to `lc_far` at a distance of 6, a `Box` field defines the band of finer elements in the wake, and a `Min` field combines them. A `BoundaryLayer` field adds quadrilateral layers around the cylinder. The domain is triangulated with the Frontal–Delaunay algorithm (`Mesh.Algorithm = 6`). The file below is the one used for $Re = 500$; the mesh for $Re = 100$ (2665 triangles, Table 6.1) was generated in the same way without the boundary-layer field and with larger element sizes.

```
// Cylinder wake, Re = 500
// Domain [-15, 25] x [-8, 8], cylinder of diameter D = 1 at the origin
lc_cyl  = 0.05;   // target element size at the cylinder
lc_wake = 0.35;   // target element size in the wake band
lc_far  = 1.5;    // target element size in the far field

// Domain corners
Point(1) = {-15, -8, 0, lc_far};
Point(2) = { 25, -8, 0, lc_far};
Point(3) = { 25,  8, 0, lc_far};
Point(4) = {-15,  8, 0, lc_far};

// Cylinder (radius 0.5)
Point(5) = { 0,    0,   0, lc_cyl};
Point(6) = { 0.5,  0,   0, lc_cyl};
Point(7) = { 0,    0.5, 0, lc_cyl};
Point(8) = {-0.5,  0,   0, lc_cyl};
Point(9) = { 0,   -0.5, 0, lc_cyl};

// Outer boundary
Line(1) = {1, 2};   // bottom (y = -8)
Line(2) = {2, 3};   // outflow (x = 25)
Line(3) = {3, 4};   // top (y = 8)
Line(4) = {4, 1};   // inflow (x = -15)

// Cylinder arcs
Circle(5) = {6, 5, 7};
Circle(6) = {7, 5, 8};
Circle(7) = {8, 5, 9};
Circle(8) = {9, 5, 6};
```

```
Curve Loop(1) = {1, 2, 3, 4};
Curve Loop(2) = {5, 6, 7, 8};
Plane Surface(1) = {1, 2};

// Physical groups (become the composites C[1], ..., C[4] and the domain)
Physical Curve("inlet",    1) = {4};
Physical Curve("outlet",   2) = {2};
Physical Curve("walls",    3) = {1, 3};      // lateral boundaries y = +-8
Physical Curve("cylinder", 4) = {5, 6, 7, 8};
Physical Surface("fluid", 100) = {1};

// Quadrilateral boundary layer around the cylinder
Field[5] = BoundaryLayer;
Field[5].CurvesList = {5, 6, 7, 8};
Field[5].hwall_n    = 0.005;   // height of the first layer at the wall
Field[5].ratio      = 1.15;    // growth ratio from layer to layer
Field[5].thickness  = 0.15;    // total thickness of the layer region
Field[5].Quads      = 1;       // quadrilaterals instead of triangles
BoundaryLayer Field = 5;

// Element size growing with the distance from the cylinder
Field[1] = Distance;
Field[1].CurvesList = {5, 6, 7, 8};
Field[1].Sampling   = 100;
Field[2] = Threshold;
Field[2].InField = 1;
Field[2].SizeMin = lc_cyl;
Field[2].SizeMax = lc_far;
Field[2].DistMin = 0.0;
Field[2].DistMax = 6.0;

// Refined band in the wake
Field[3] = Box;
Field[3].XMin = -0.5;  Field[3].XMax = 25.0;
Field[3].YMin = -2.5;  Field[3].YMax =  2.5;
Field[3].VIn  = lc_wake;
Field[3].VOut = lc_far;

Field[4] = Min;
Field[4].FieldsList = {2, 3};
Background Field = 4;

Mesh.Algorithm    = 6;   // Frontal-Delaunay
Mesh.RecombineAll = 0;
Mesh.Smoothing    = 5;
Mesh.ElementOrder = 1;
```

The mesh was converted to the Nektar++ format with `NekMesh mesh.msh mesh.xml` [40]. NekMesh stores triangles and quadrilaterals in separate composites, `C[100]` and `C[101]`, which is why the expansion in Appendix A.2 is defined on `C[100,101]`.

A.2 Nektar++ Session File

The session file defines the expansion, the solver, the parameters and the boundary conditions. Its keywords are described in the Nektar++ user guide [82, Ch. 'Incompressible Navier–Stokes Solver']. The file below is the one used for $Re = 500$; for $Re = 100$ the same settings were used with `Kinvis` $= 0.01$, `TimeStep` $= 0.005$, `NumSteps` $= 100\,000$ and a snapshot every 20 time steps (Table 6.2). The expansion uses a modified modal basis with $P = 10$ (`NUMMODES` $= P + 1 = 11$), the velocity correction scheme [81] with second-order implicit–explicit time

integration. At $Re = 500$ a snapshot is written every 400 time steps (0.2 time units) by the checkpoint filter, and the forces on the cylinder are written every 10 time steps.

```
<NEKTAR>
<CONDITIONS>
<!-- Expansion: modified modal basis, NUMMODES = P + 1 = 11 (P = 10);
     C[100] = triangles, C[101] = quadrilaterals of the mesh -->
<EXPANSIONS>
<E COMPOSITE="C[100,101]" NUMMODES="11" TYPE="MODIFIED" FIELDS="u,v,p"/>
</EXPANSIONS>
<SOLVERINFO>
<I PROPERTY="SolverType"    VALUE="VelocityCorrectionScheme"/>
<I PROPERTY="EQTYPE"        VALUE="UnsteadyNavierStokes"/>
<I PROPERTY="AdvectionForm" VALUE="Convective"/>
<I PROPERTY="Projection"    VALUE="Continuous"/>
<I PROPERTY="Dealiasing"    VALUE="True"/>
</SOLVERINFO>
<TIMEINTEGRATIONSCHEME>
<METHOD> IMEX </METHOD>
<ORDER> 2 </ORDER>
</TIMEINTEGRATIONSCHEME>
<PARAMETERS>
<P> TimeStep     = 0.0005 </P>
<P> NumSteps     = 200000 </P>  <!-- 200000 x 0.0005 = 100 time units -->
<P> FinalTime    = 100.0  </P>
<P> Checkpoint   = 400    </P>
<P> IO_InfoSteps = 2000   </P>  <!-- console output every time unit -->
<P> Kinvis       = 0.002  </P>  <!-- nu = 1/Re, Re = 500 -->
</PARAMETERS>
<VARIABLES>
<V ID="0"> u </V>
<V ID="1"> v </V>
<V ID="2"> p </V>
</VARIABLES>
<BOUNDARYREGIONS>
<B ID="0"> C[1] </B>  <!-- inflow x = -15             -->
<B ID="1"> C[2] </B>  <!-- outflow x = 25             -->
<B ID="2"> C[3] </B>  <!-- lateral boundaries y = +-8 -->
<B ID="3"> C[4] </B>  <!-- cylinder                   -->
</BOUNDARYREGIONS>
<BOUNDARYCONDITIONS>
<!-- Inflow: uniform velocity u = 1, v = 0 -->
<REGION REF="0">
<D VAR="u" VALUE="1.0"/>
<D VAR="v" VALUE="0.0"/>
<N VAR="p" VALUE="0.0"/>
</REGION>
<!-- Outflow: zero normal velocity derivatives, p = 0 -->
<REGION REF="1">
<N VAR="u" VALUE="0.0"/>
<N VAR="v" VALUE="0.0"/>
<D VAR="p" VALUE="0.0"/>
</REGION>
<!-- Lateral boundaries: free-stream velocity (Dirichlet) -->
<REGION REF="2">
<D VAR="u" VALUE="1.0"/>
<D VAR="v" VALUE="0.0"/>
<N VAR="p" VALUE="0.0"/>
</REGION>
<!-- Cylinder: no slip -->
<REGION REF="3">
<D VAR="u" VALUE="0.0"/>
<D VAR="v" VALUE="0.0"/>
<N VAR="p" VALUE="0.0"/>
</REGION>
</BOUNDARYCONDITIONS>
<!-- Initial condition: uniform free stream -->
<FUNCTION NAME="InitialConditions">
<E VAR="u" VALUE="1.0"/>
<E VAR="v" VALUE="0.0"/>
<E VAR="p" VALUE="0.0"/>
</FUNCTION>
</CONDITIONS>
```

```
<FILTERS>
<!-- snapshot every 400 steps = 0.2 time units -->
<FILTER TYPE="Checkpoint">
<PARAM NAME="OutputFile">cyl_chk</PARAM>
<PARAM NAME="OutputFrequency">400</PARAM>
</FILTER>
<!-- drag and lift on the cylinder, every 10 steps -->
<FILTER TYPE="AeroForces">
<PARAM NAME="OutputFile"> cylinder_forces </PARAM>
<PARAM NAME="OutputFrequency"> 10 </PARAM>
<PARAM NAME="Boundary"> B[3] </PARAM>
<PARAM NAME="Density"> 1.0 </PARAM>
<PARAM NAME="SurfaceArea"> 1.0 </PARAM>
</FILTER>
</FILTERS>
</NEKTAR>
```

The simulation was run in parallel with MPI, for example with

```
mpirun -np 8 IncNavierStokesSolver mesh.xml session.xml.
```

Appendix B

Data Format and Extraction

B.1 From Nektar++ Output to Training Data

The data pipeline of Section 6.4 has three steps:

1. Nektar++ writes one field file per snapshot (`.chk`). These are converted to the VTU format with the utility FieldConvert, for example `FieldConvert mesh.xml session.xml cyl_100.chk cyl_100.vtu`, which evaluates the high-order expansion at the quadrature points of every element [82, Ch. 'FieldConvert'].

2. A Python script reads the VTU files with the VTK library, keeps the points inside the chosen window, and removes points that occur more than once at element interfaces.

3. The arrays are written to a MATLAB file with `scipy.io.savemat`, together with the snapshot times measured from the first snapshot used (Section 6.3).

Before training, the script checks the scaling of the data (Section 6.4): the cylinder diameter in the mesh coordinates must be $D = 1$ and the mean inflow velocity $U_\infty = 1$.

B.2 Structure of the Data Files

Table B.1 lists the arrays in the data set of Raissi et al. [10] and in the files produced for this thesis. N_x is the number of spatial points in the window and N_t the number of snapshots.

Table B.1: Arrays in the MATLAB data files. Only u and v are used for training; the pressure is used only to evaluate the reconstruction.

Quantity	Raissi et al. [10]	This thesis	Shape
coordinates	`X_star`	`x_star`, `y_star`	$N_x \times 2$ / $N_x \times 1$ each
times	`t_star`	`t_star`	$N_t \times 1$
velocity	`U_star`	`U_star`, `V_star`	$N_x \times 2 \times N_t$ / $N_x \times N_t$ each
pressure	`p_star`	`P_star`	$N_x \times N_t$

The sizes of the files used in this thesis are: near wake $N_x = 5000$ and $N_t = 200$, mid wake $N_x = 4432$ and $N_t = 400$, far wake $N_x = 5000$ and $N_t = 400$, upper mid-wake window $N_x = 1349$ and $N_t = 400$, lower far-wake window $N_x = 1567$ and $N_t = 400$ snapshots. The file at $Re = 500$ (near-wake window) contains $N_x = 8934$ points and $N_t = 501$ snapshots for $t \in [0, 100]$. The data set of Raissi et al. has $N_x = 5000$ and $N_t = 200$ [10].

Appendix C

The L-BFGS Two-Loop Recursion

Algorithm 1 computes the product $\mathbf{r} = \mathbf{H}_k \mathbf{g}_k$ of Section 4.5.6 from the last m pairs $(\mathbf{s}_i, \mathbf{y}_i)$, $i = k-m, \ldots, k-1$, without forming the matrix $\mathbf{H}_k$ [74, Sec. 7.2]. The first loop runs backwards through the stored pairs and removes their contribution from the gradient; the initial matrix $\vartheta_k \mathbf{I}$ is then applied; and the second loop runs forwards and adds the corrections back in. The scalars a_i and β and the vector $\mathbf{q}$ are auxiliary quantities used only inside the algorithm. As noted in Section 4.5.6, the L-BFGS-B code computes the same product with the equivalent compact representation [75].

Algorithm 1 L-BFGS two-loop recursion, adapted from [74, Alg. 7.4]. All vectors are in $\mathbb{R}^{n_\Theta}$.

$\mathbf{q} \leftarrow \mathbf{g}_k$
for $i = k-1, k-2, \ldots, k-m$ **do**
    $\varrho_i \leftarrow 1/(\mathbf{y}_i^\top \mathbf{s}_i)$
    $a_i \leftarrow \varrho_i \, \mathbf{s}_i^\top \mathbf{q}$
    $\mathbf{q} \leftarrow \mathbf{q} - a_i \, \mathbf{y}_i$
end for
$\mathbf{r} \leftarrow \vartheta_k \, \mathbf{q}$, with $\vartheta_k = \mathbf{s}_{k-1}^\top \mathbf{y}_{k-1} / (\mathbf{y}_{k-1}^\top \mathbf{y}_{k-1})$
for $i = k-m, \ldots, k-1$ **do**
    $\beta \leftarrow \varrho_i \, \mathbf{y}_i^\top \mathbf{r}$
    $\mathbf{r} \leftarrow \mathbf{r} + (a_i - \beta) \, \mathbf{s}_i$
end for
return $\mathbf{r}$